Graph Theory

Introduction

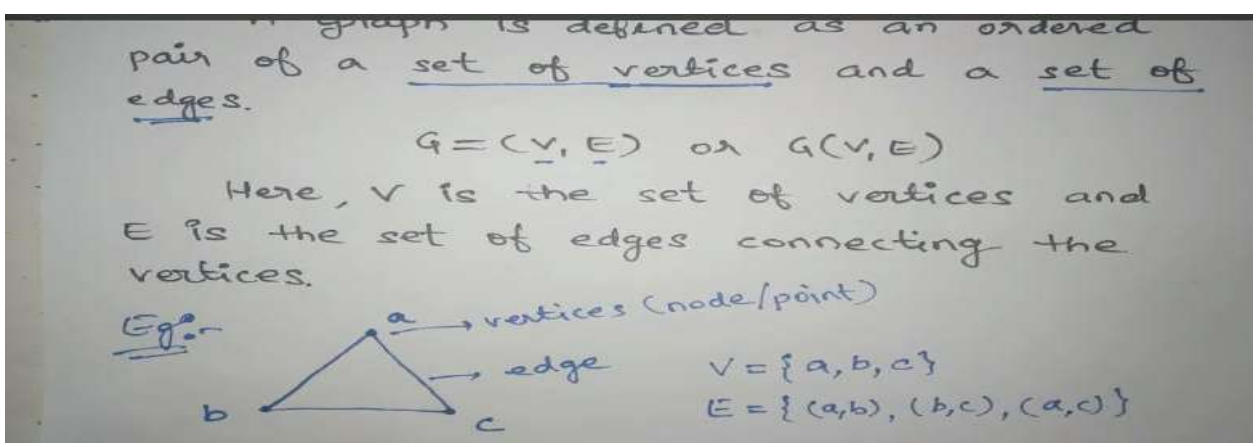
Many situations that occur in computer science, physical science, communication science, Economics and many others areas can be analyzed by using techniques found in a relatively new area of mathematics called graph theory. Graphs can be used to represent almost any problem involving discrete arrangements of objects where concern lies not with the internal properties of these objects but with relationships among them

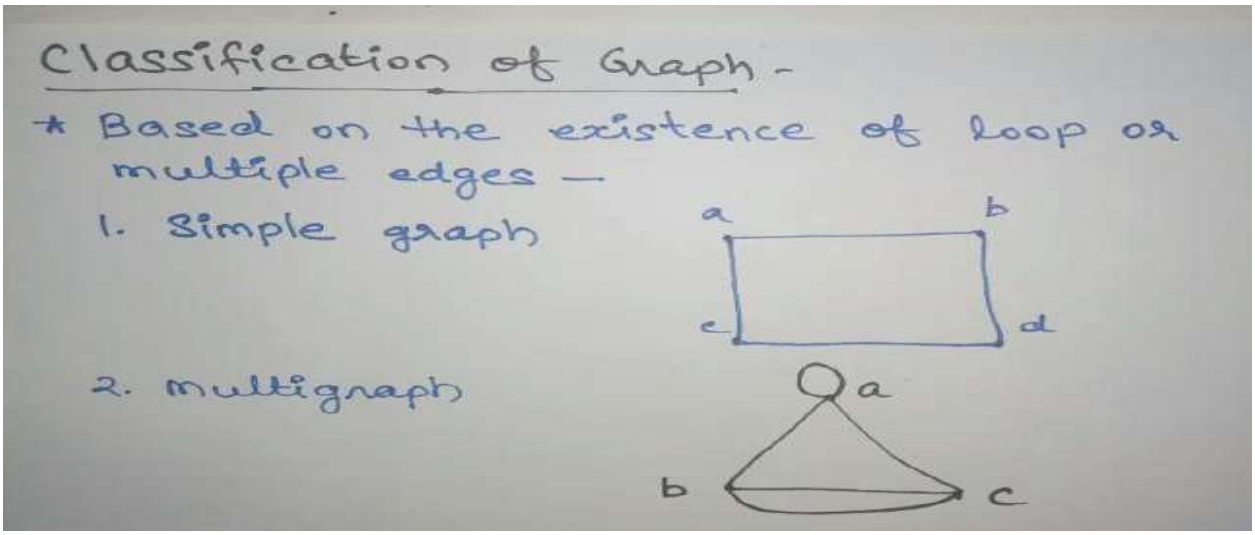
Graph is a discrete structure consisting of vertices and edges connecting the vertices. A graph G = (V,E) is a mathematical model consists of two non empty sets: $V = \{v_1, v_2, , v_3 \ldots\ldots v_n\}$ is called set of vertices and $E = \{e_1, e_2, e_3 \ldots e_n\}$ of ordered or unordered pairs of distinct vertices called edges. We usually write G = (V,E) and say V is the vertex and E is the edge set of G

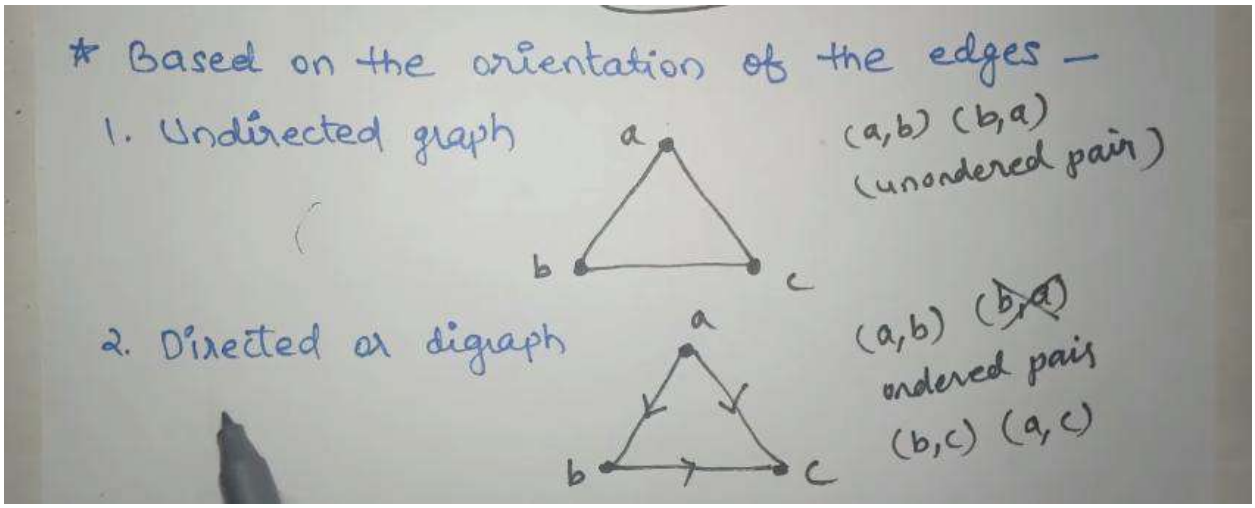
Applications of Graph

Graphs are used to solve problems in many fields. Some of them are as follows:

- Graphs are used to model the geographic maps of cities in which each place in city can be represented by the node and the road connecting such places are represents the edge
- Graphs are used to model the computer network in which each node is a machine and link between them represent the edge
- They are used to analyze the electric circuits, project planning, genetics etc
- Any structured problem can be modeled by graphs. Then can help to solve typical problems those connected with finding shortest path of most economical route between two vertices, or the smallest set of edges which connect all the vertices in a graph
- Graphs are used to study the structure of WWW
- It is used to find the number of different combination of flight between two cities in an airline network
- It can be used to distinguish between two chemical compounds with the same molecular formula but different structure
- It is also used to assign channels to television stations







Simple Graph

We define a simple graph as 2-tuple consisting of a non- empty set of vertices V and a set of unordered pairs of distinct elements of vertices called edges, having neither loop not parallel edges. A graph G = (V,E) is said to be simple if G has no loops and no parallel edges.

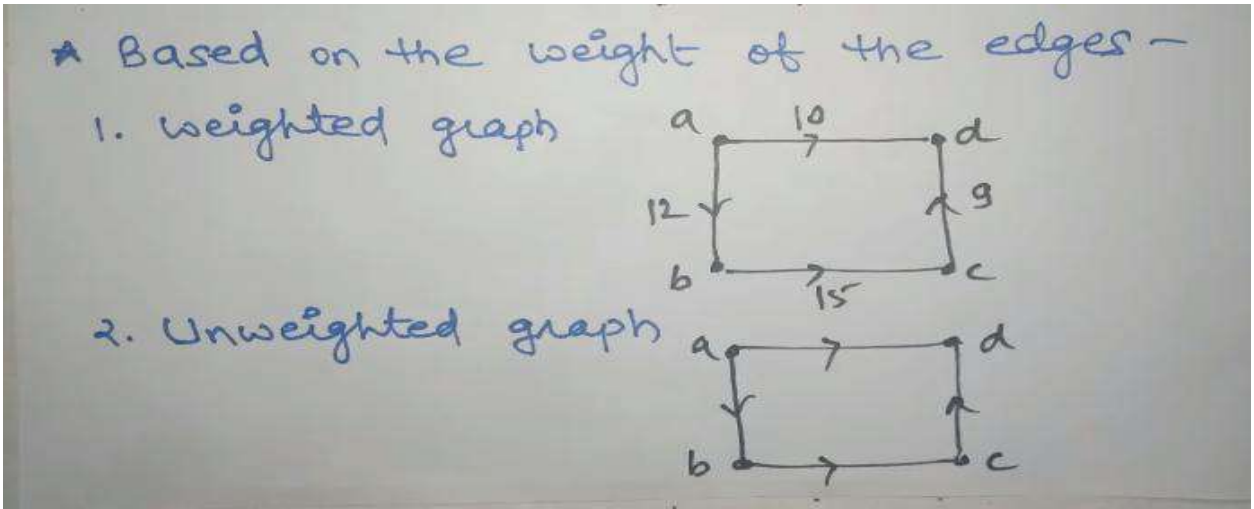
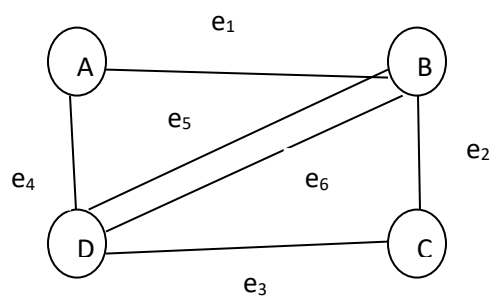


Figure Simple Graph

Multi-Graph: A computer network may contain multiple links between data centers, to model such network we need graphs that have more than one edge connecting the same pair of vertices. To or more edges between same pair of vertices are called parallel edges or two or more edges having same end points are called parallel edges

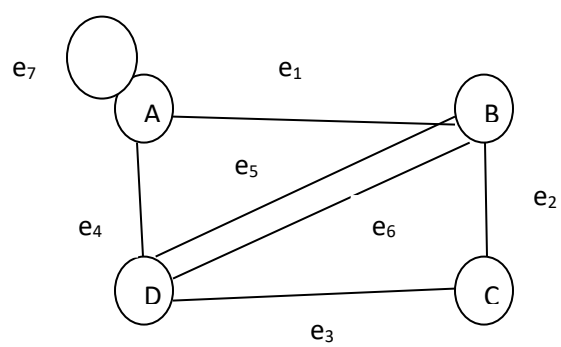


A graph G = (V,E) consists of a set of vertices V and set of edges E such that some of edges are parallel edges is called multi-graph

In above graph edge e_5 and e_6 are parallel. So this graph is multi-graph.

Pseudograph

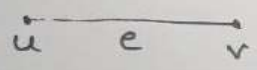
Sometimes a communication link connects a data center with itself, perhaps a feedback loop for diagnostic purpose, to model such a network we need to include edges that connects a vertex to itself such a edge is called a loop



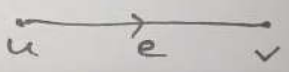
Important Definitions -

(1) Adjacent Vertex —

Vertex u and v are said to be adjacent if there is an edge = {u, v}



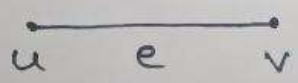
{u, v} is adjacent with e = {u, v}



u is adjacent to v, and v is adjacent from u

(2) Incident Edge -

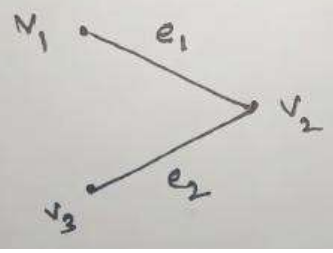
The edge 'e' that joins the nodes u and v is said to be incident on each of its end points u and v.



↓ incident edge

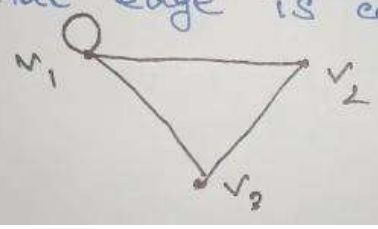
(3) Adjacent Edges -

Two edges are said to be adjacent if they are incident on a common vertex.



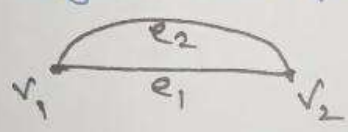
(4) Self loops –

If there is an edge from v_i to v_i then that edge is called self loop.



(5) Parallel Edges –

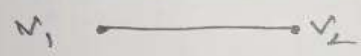
If two edges have same end points then the edges are parallel edges.



e_1, e_2 is parallel edges.

(6) Isolated vertex –

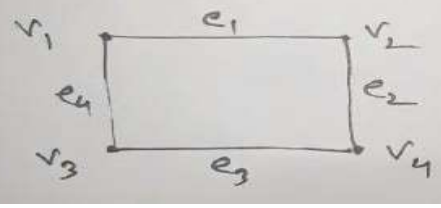
A vertex having no edge incident on it is an isolated vertex.



$\cdot v_3 \rightarrow$ isolated vertex

(7) Simple graph –

A graph which has neither self loops nor parallel edge is simple graph.



(8) Multigraph –

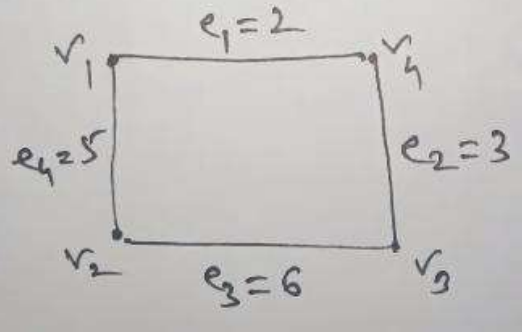
A graph which has more than one edge between a pair of vertices is multigraph.

(9) Pseudograph –

A graph in which loops and parallel edges/ multiple edges are allowed is called pseudograph.

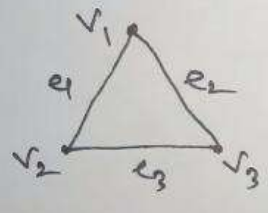
(10) Weighted graph –

A graph, in which weights are assigned to every edge is called weighted graph.



(11) Finite graph –

A graph $G=\{V,E\}$ in which both V and E are finite set is called a finite graph.

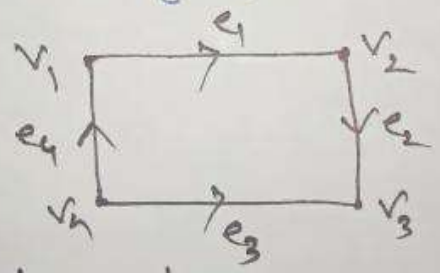


$V=\{v_1, v_2, v_3\}$

$E=\{e_1, e_2, e_3\}$

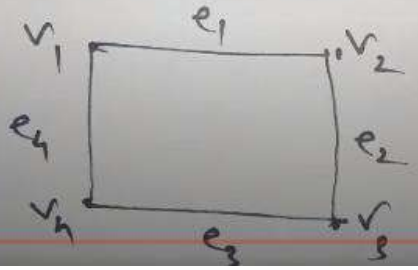
(12) Directed graph / Digraph –

A graph in which every edge is directed is digraph.



(13) Undirected graph –

A graph in which an edge is associated with an unordered pair of vertices is undirected graph.



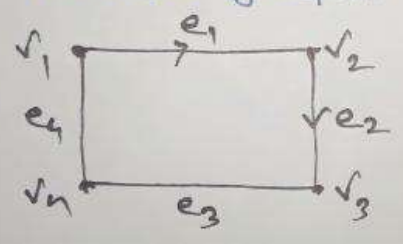
$e_1 = (v_1, v_2)$

or

(v_2, v_1)

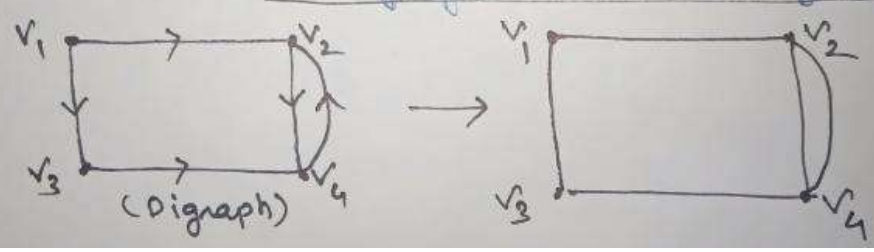
(14) Mixed Graph –

If some edges are directed and some are undirected in a graph, the graph is called mixed graph.



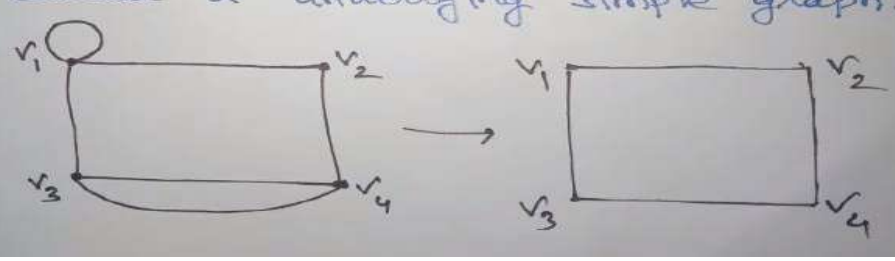
(15) Underlying undirected graph.

A graph obtained by ignoring the direction of edges in a directed graph is called underlying undirected graph.



(16) Underlying simple graph –

A graph obtained by deleting all loops and parallel edges from a graph is called a underlying simple graph.



Graph

- undirected
  - degree of a vertex
- directed
  - Indegree
  - Outdegree

Degree of a vertex –

The number of edges incident on a vertex v, with self-loops counted twice, is called the degree of the vertex v and is denoted by $deg_G(v)$ or $deg(v)$ or $d(v)$.

Isolated Vertex –

A vertex having no incident edge is called an isolated vertex. In other words, isolated vertices are th with zero degree.

Pendant Vertex –

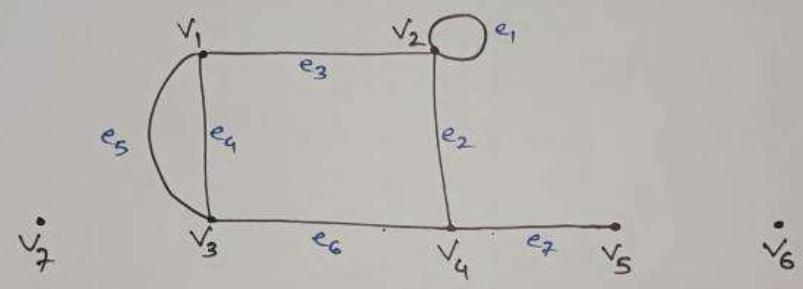
A vertex of degree 1, is called a pendant vertex or an end vertex.

Like & Subscribe

Internal Vertex –

A vertex, which is neither a pendant vertex nor an isolated vertex, is called an internal vertex or an intermediate vertex.

Qu:- Find the degree of all vertex. ~~and also def~~



Soln:-

$\deg(v_1) = 3$
$\deg(v_2) = 4$
$\deg(v_3) = 3$
$\deg(v_4) = 3$
] internal vertex

$\deg(v_5) = 1$] pendant vertex

$\deg(v_6) = 0$
$\deg(v_7) = 0$
] isolated vertex

Like & Subscribe

Indegree of a Vertex –

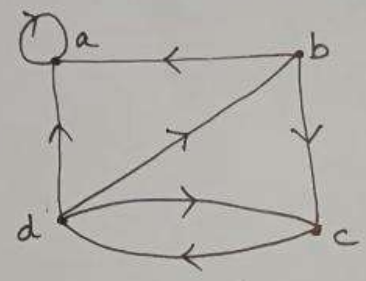
The indegree of vertex v in a directed graph is the number of edges which are coming into the vertex v (that is, the number of incoming edges) and is denoted b $d^-(v)$, $\deg^-(v)$ or $\text{Indeg}(v)$.

Out degree of a Vertex –

The out degree of vertex v in a directed grap is the number of edges which are going out from the vertex v (that is, the number of outgoing edges) and is denoted by $d^+(v)$, $\deg^+(v)$ or $\text{Outdeg}(v)$.

Like & Subscribe

Qu:- Find the degrees of vertex of the directed graph.



vertex	$d^-(v)$ Indegree	$d^+(v)$ outdegree	Total degree
a	3	1	4
b	1	2	3
c	2	1	3
d	1	3	4

Graph Models

Graph Terminologies

- **Order and Size of Graph:** If G = (V,E) be a finite graph then the number of vertices in graph G is called order of graph G and the number of edges in G is called size of G
- **Degree of Vertex:** Let G = (V,E) be a graph and v be a vertex of G. The degree of v is denoted by d(v) is the number of edges incident on v
- **Isolated and pendant Vertex:** In a graph G = (V,E), a vertex having degree zero (0) is called isolated vertex and vertex having degree one (1) is called pendant vertex

Degree sequence of a Graph: If $v_1, v_2, v_3 \ldots\ldots\ldots\ldots v_n$ are n vertices of G then the sequence $(d_1,d_2,d_3 \ldots d_n)$ where $d_i = \deg(v_i)$ is the degree sequence of G. In general, we order the vertices so that the degree sequence is monotonically increasing.

That is

$d(G) = d_1 \leq d_2 \leq d_3 \ldots \leq d_n = \Delta(G)$

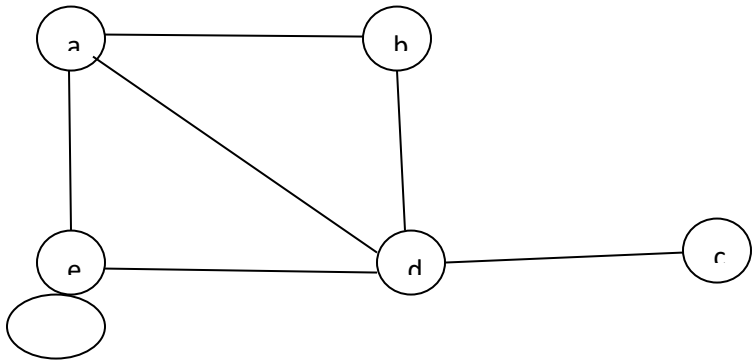
where $d(G) = \min (\deg v: v \in V(G)\}$

and $\Delta(G) = \max(\deg v: v \in V(G)\}$

Adjacent vertices and adjacent edges

Two vertices u and v in an unordered graph are called adjacent if there is an edge between u and v.

Two edges e_1 and e_2 in an undirected graph are said to be adjacent if they share a common vertex.



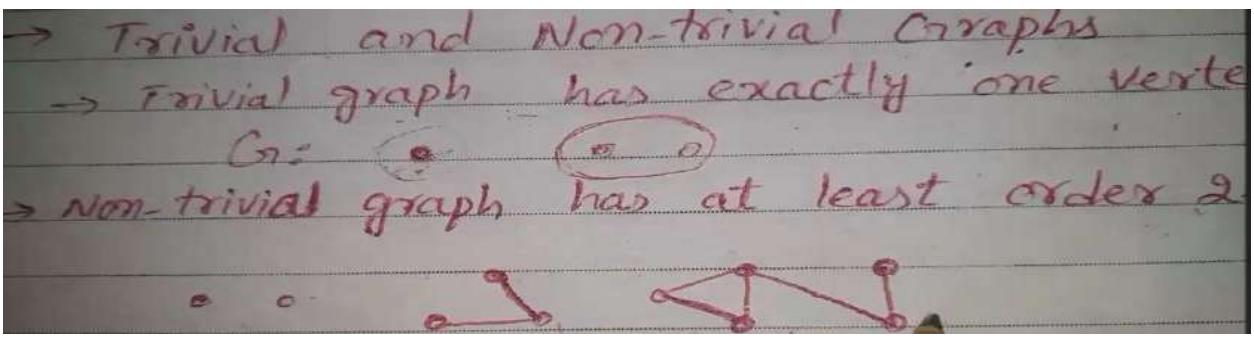
In this graph, deg(a) = 3, deg(b) = 2, deg(c) = 1 deg(d)=4 and deg(e)=4

Therefore, degree sequence of G is (1, 2, 3, 4)

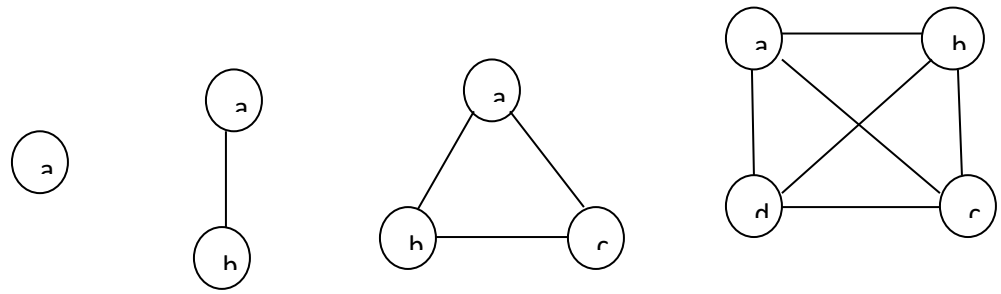
Simple and Special Graph

Some important types of graphs are introduced here. These graphs are often used as examples and arise in many applications.

Trivial Graph: A graph with one vertex and no edges is called a trivial graph



Complete Graph: A graph is said to be complete if there exists exactly one edge between any pair of vertices in G. The complete graph with n vertices is denoted by K_n.



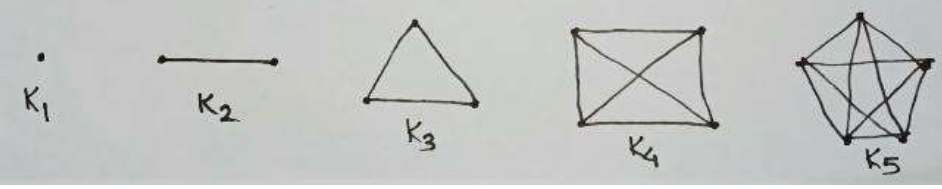
K_1 K_2 K_3 K_4

Complete Graph –

A simple graph G is said to be complete if every ve in G is connected with every other vertex i,e if G contains exactly one edge between each pair of distinct vertices.

A complete graph on n vertices is denoted by K_n.

Eg:-



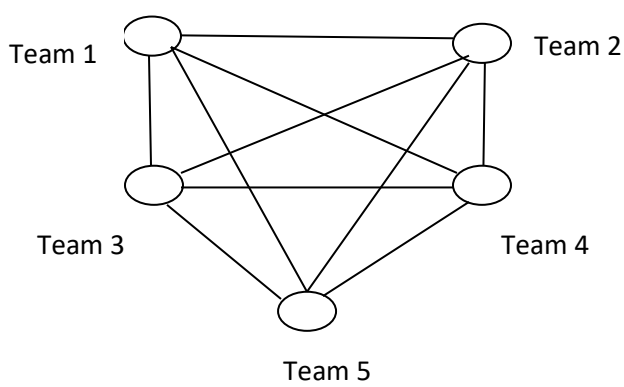
Note: ① A complete graph K_n has $\frac{n(n-1)}{2}$ edges.

Eg:- $K_4 = \frac{n(n-1)}{2} = \frac{4(4-1)}{2} = \frac{\cancel{4}^{2} \times 3}{\cancel{2}} = 6$ edges

② The size of every graph of order n is at most $\frac{n(n-1)}{2}$

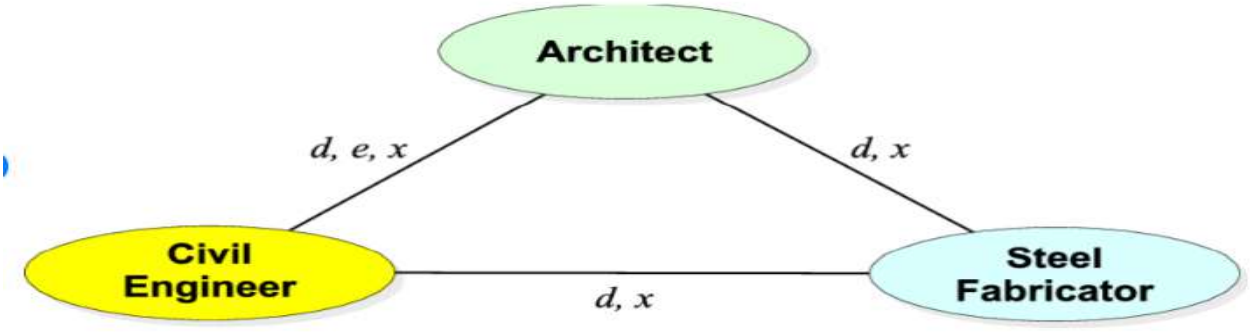
$\boxed{K_n \text{ has exactly } \frac{n(n-1)}{2} \text{ edges}}$

Round Robin Tournament: A game tournament where each team plays with each other exactly once is called round-robin tournament. Such tournaments can be modeled using complete graph where there exists exactly one edge between each possible parts of vertices except itself. The following figure shows the tournament between 5 teams in Round-Robin fashion. Here, in the graph edge between vertices represent the matches between the teams and vertices represent teams.



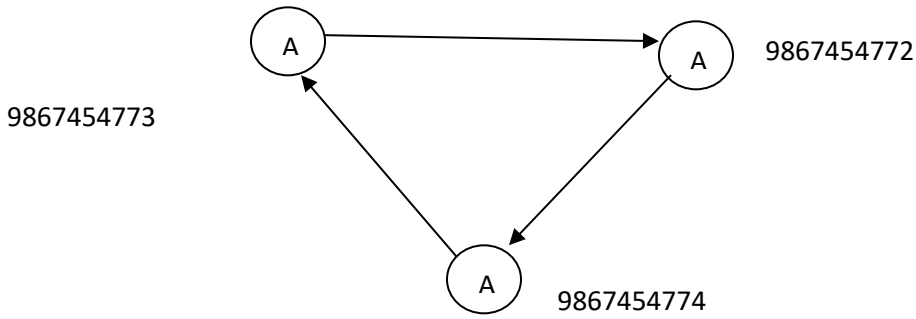
Collaborative Graph

A simple graph that can be used to model joint authorship of academic papers, where vertices represent people and edges between two vertices represent joint authorship if they have jointly written paper.

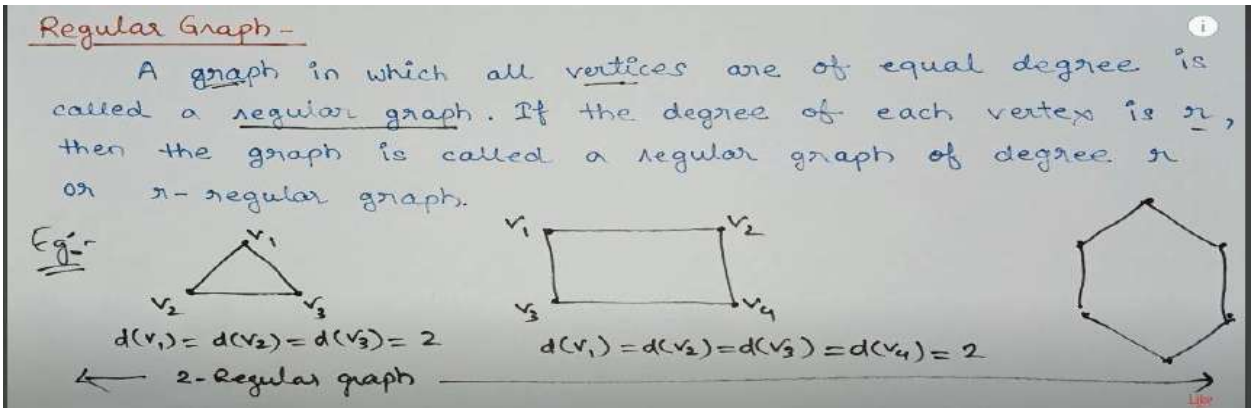


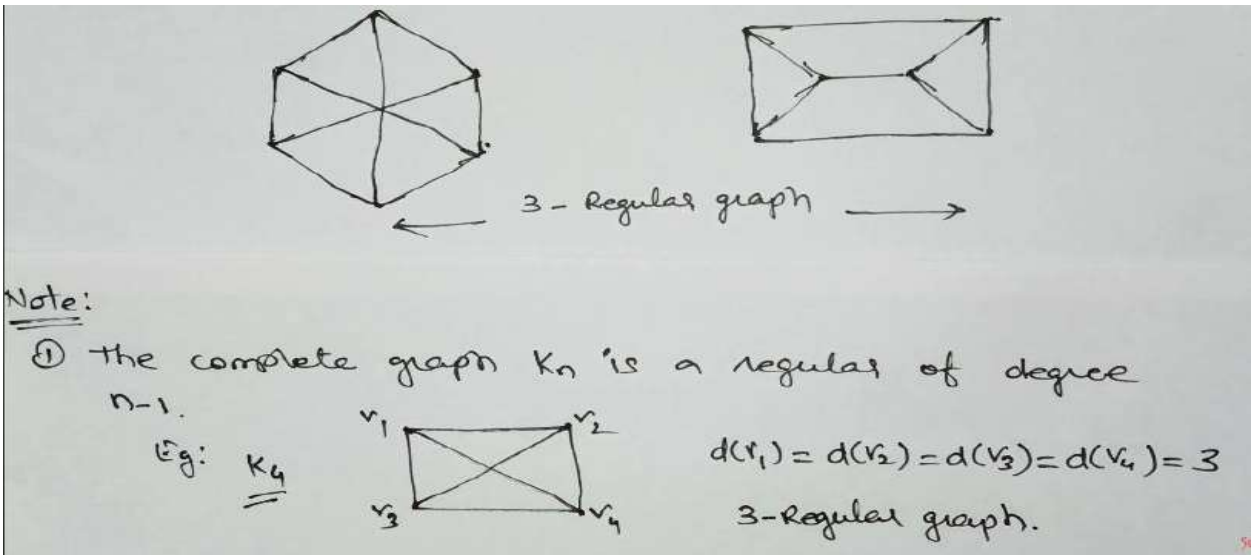
3: Collaboration graph example.

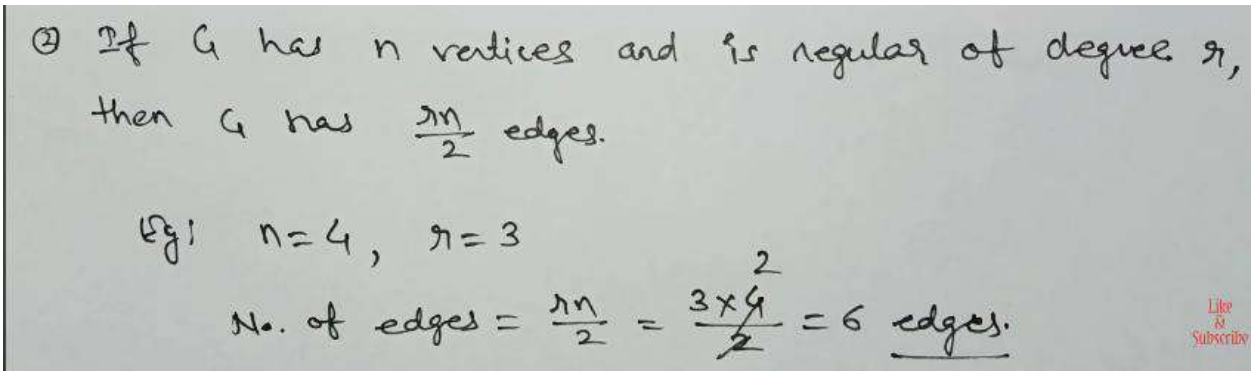
Call Graph: Graph can be used to model telephone calls in a telecommunication network. In such graph, each telephone number is represented by a vertex and each telephone call is represented by edge. The source vertex represents a start call (sender) and the vertex where edge ends, represents the end of call (receiver)



Regular Graph: A graph in which all vertices are of same (equal) degree is called a regular graph. If the deg ree of each vertex is r then the graph is a r-regular graph.



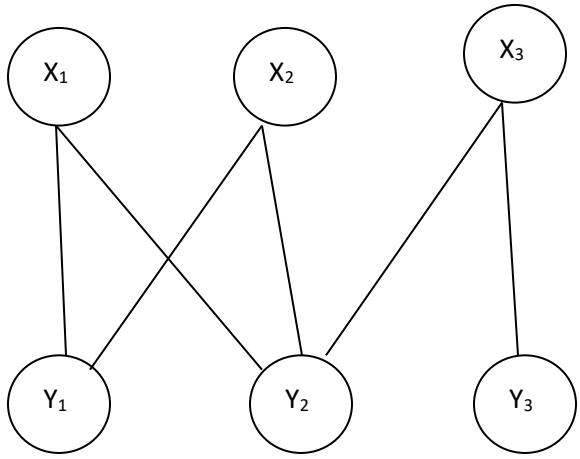




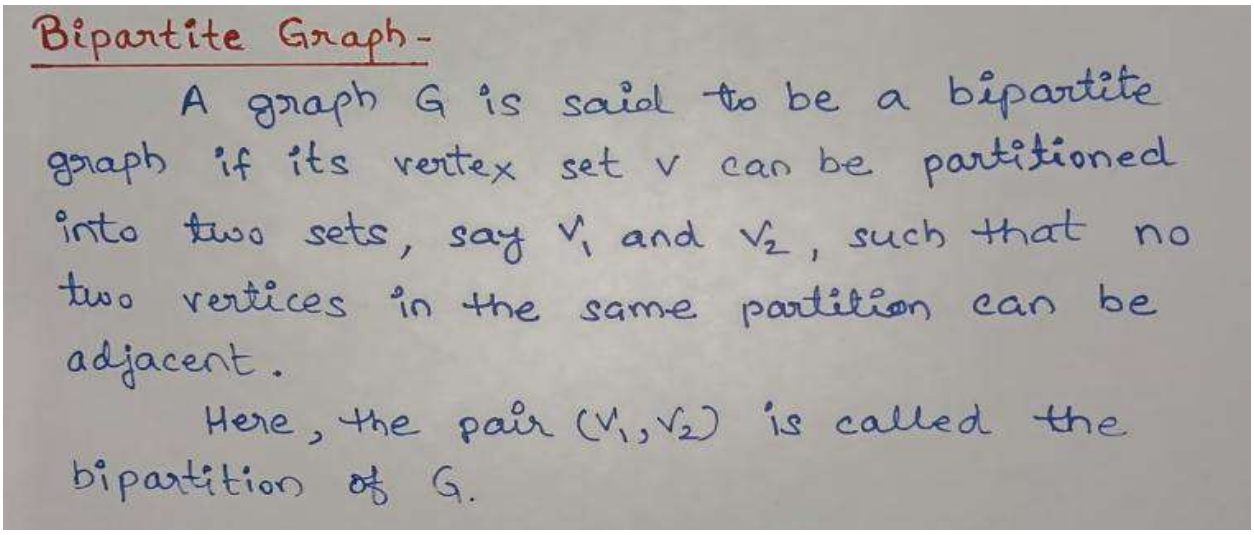
Bipartite Graph: A graph G = (V,E) is said to be bipartite if V is the union of two no-empty disjoint subsets V1 and V2 of V such that each edge in E is incident on one vertex in V1 and one vertex is in V2

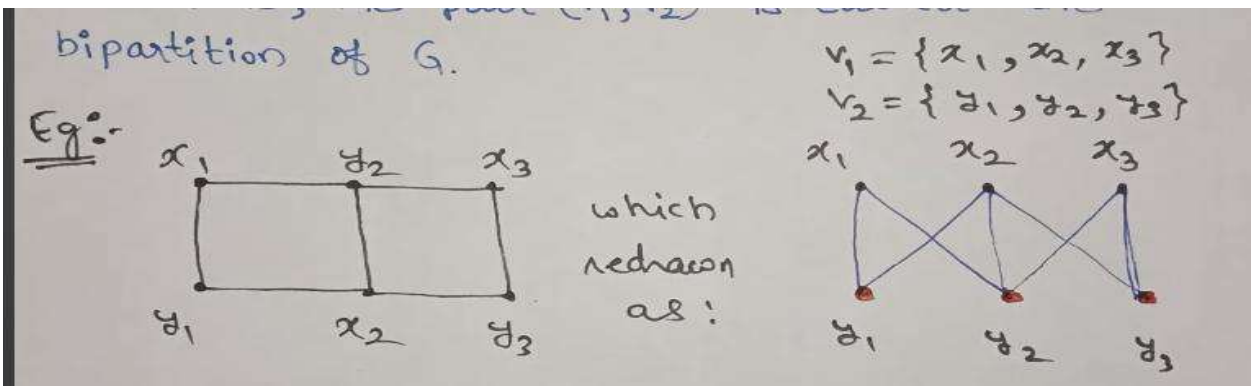
Or

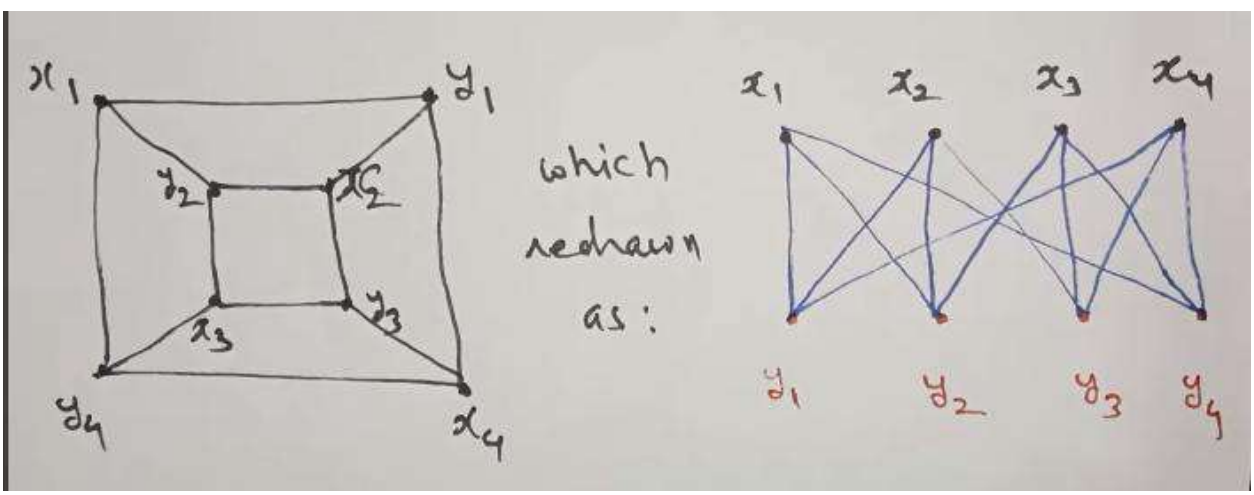
A graph G = (V,E) is said to be bipartite if its vertex set V can be partitioned into two subsets V_1 and V_2 such that each edge of G connect the vertex of V_1 to vertex of V_2 so that no edge in G connect two vertices in V_1 or two vertices in V_2.



In this figure, bipartite sets are { X_1, X_2, X_3} and {Y_1, Y_2, Y_3}







Complete Bipartite Graph : A bipartite graph is said to be complete if each vertex of first bipartite set is connected to every vertices of another bipartite set. A complete bipartite graph with n number of vertices in first partite set and n number of vertices in second bipartite set is denoted $K_{m,n}$

A complete bipartite graph $K_{m,n}$ is a graph that has its vertex set partitioned into two subsets of m and n vertices, respectively with an edge between every pair of vertices if and only if one vertex in the pair is in the first subset and the other vertex is in the second subset.

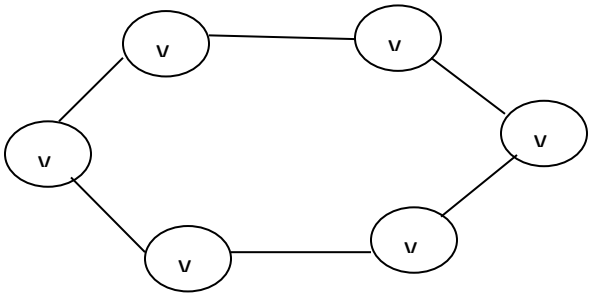
Eg:-

$K_{2,3}$

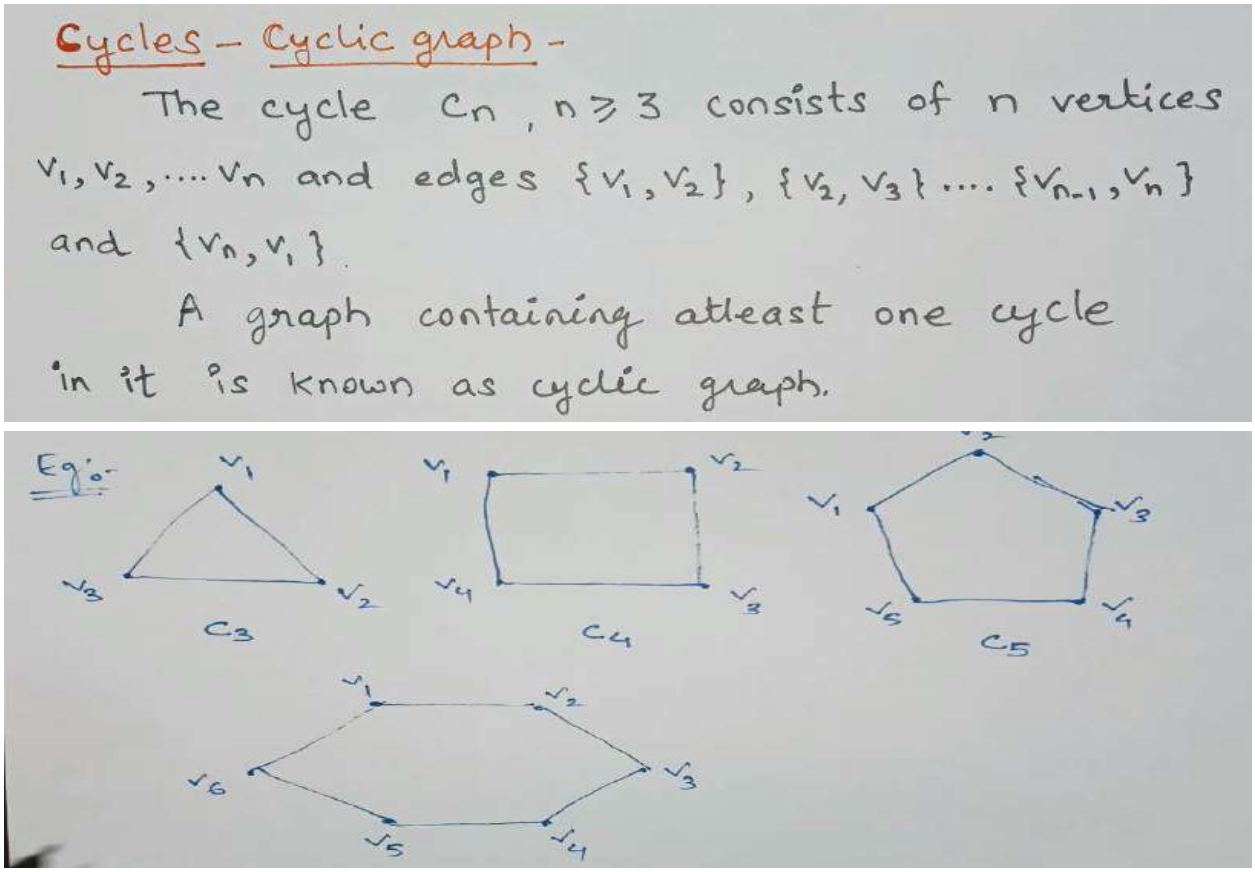
$K_{3,3}$

$K_{3,5}$

Cycle Graph: A graph V = (G,E) with vertices n≥3 in which every vertex is connected with two other vertices one on either side of it and last vertex is connected with first one to form a closed path, is called cycle graph.

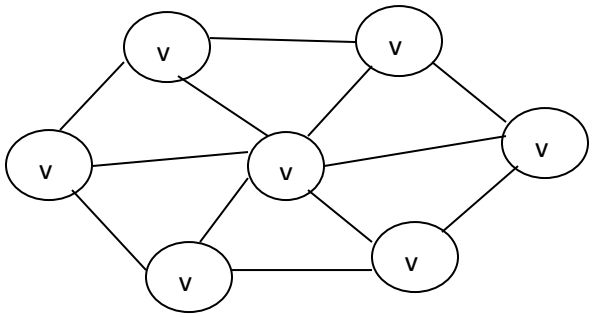


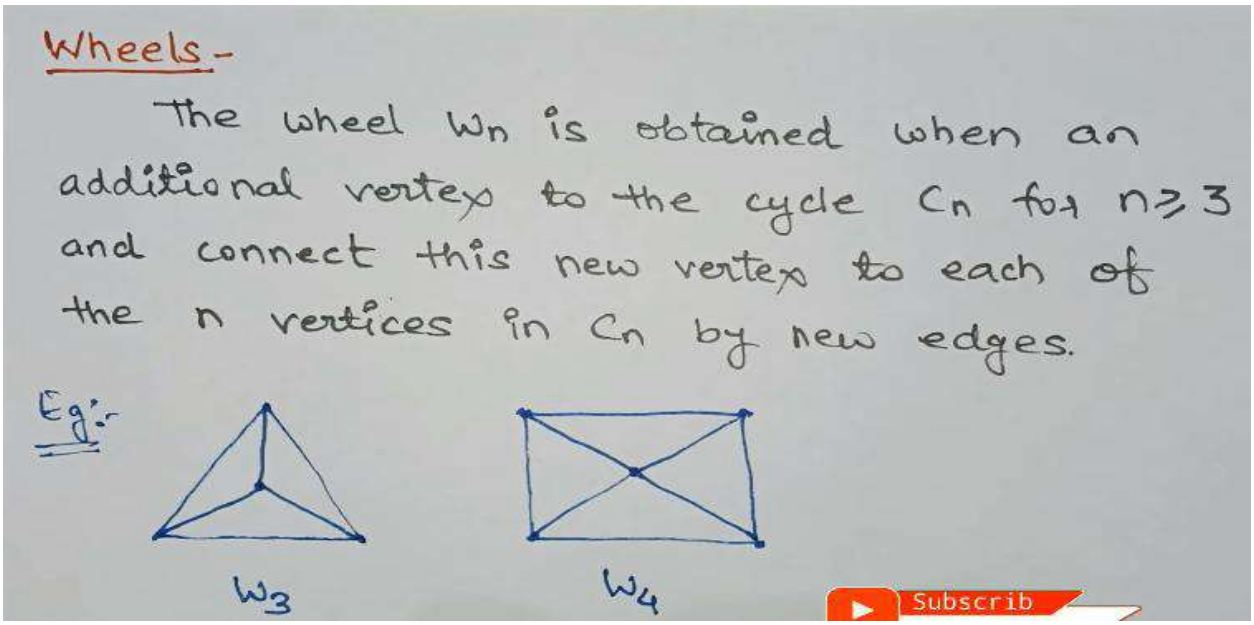
The cycle graph C_n of length n consists of n vertices $v_1, v_2, v_3 \ldots v_n$ and edges $\{v_1,v_2\}, \{v_2,v_3\}, \{v_3,v_4\} \ldots\ldots \{v_{n-1}, v_n\}$ and $(v_n, v_1\}$

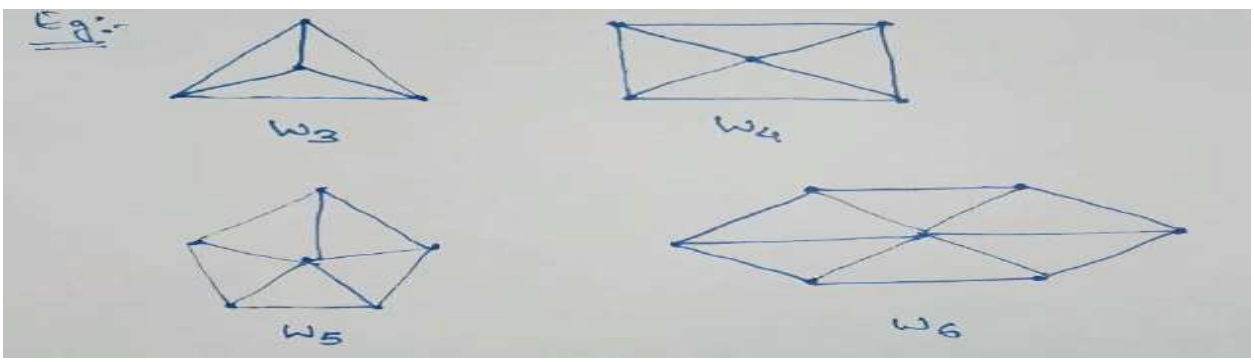


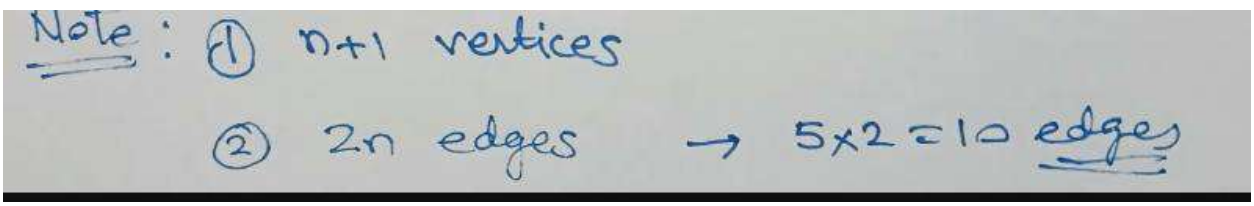
Wheel Graph

The wheel W_n, for $n \geq 3$, is an union of Cn and additional vertex where the new vertex is connected to each vertex of cycle. That is, a graph which is obtained by an additional vertex to the cycle graph C_n for $n \geq 3$ and connect new vertex to each of n vertices in Cn by new edge is called wheel graph.



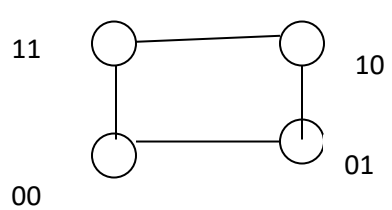






N-cube Graph

The n-dimensional cube or n-cube denoted by Q_n, is the graph that has vertices representing the 2nd bits strings of length n. Here, two vertices are adjacent if and only if the bit string that they represent differ in exactly one bit positions.



N- cube –

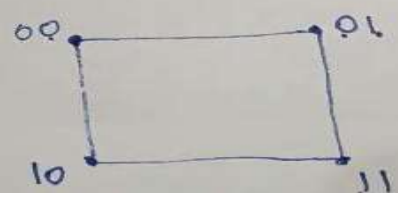
The N- cube denoted by Q_n, is the graph that has vertices representing the 2^n bit strings of length n.

Eg:- Q_1 i.e $n=1$, No. of vertex $2^n \Rightarrow 2^1 = 2$

Q_2 i.e $n=2$ No of vertex $2^n \Rightarrow 2^2 = 4$

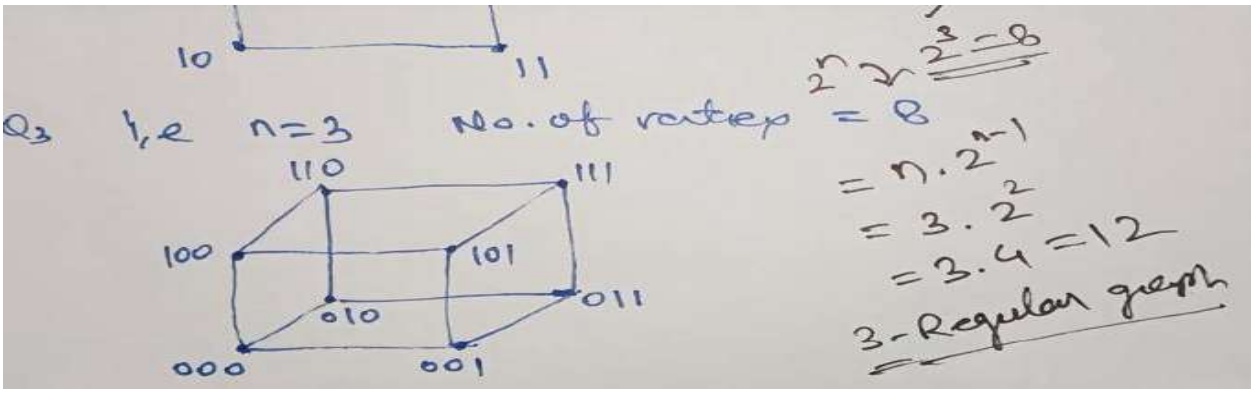
Eg:- Q_1 i.e $n=1$, No. of vertex $2^n \Rightarrow 2^1 = 2$

Q_2 i.e $n=2$ No of vertex $2^n \Rightarrow 2^2 = 4$



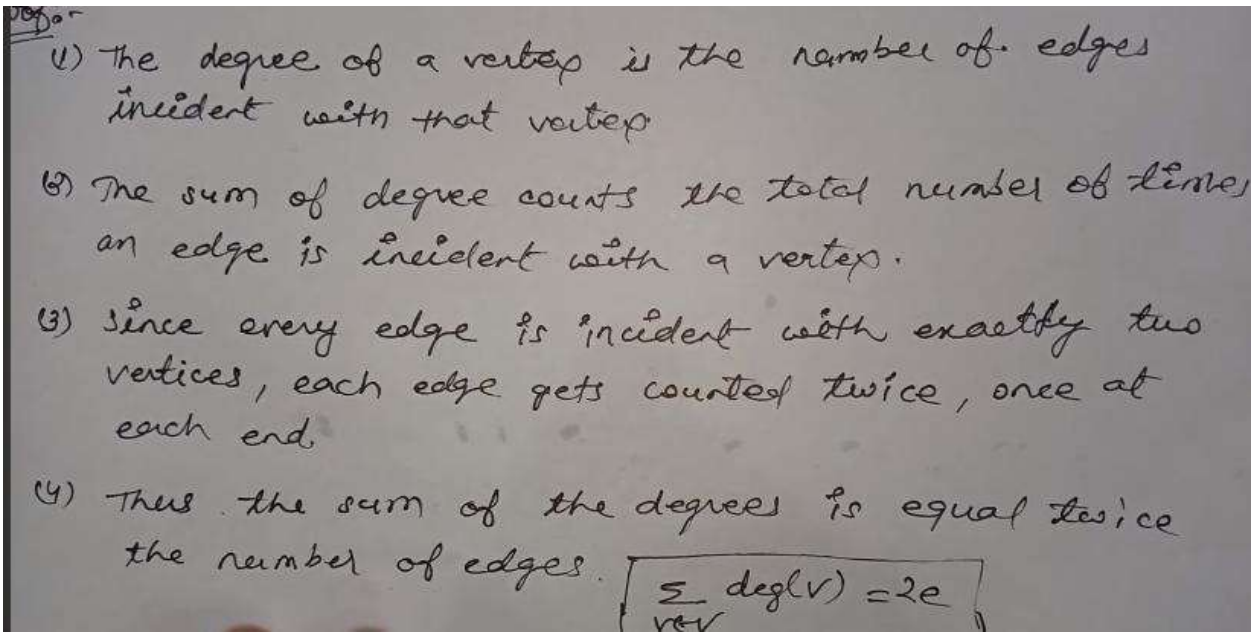
Note :-

1. Two vertices are adjacent if and only if the bit strings that they represent differ in exactly one bit position.
2. the Q_n has 2^n vertices and $n.2^{n-1}$ edges, and is regular of degree n.

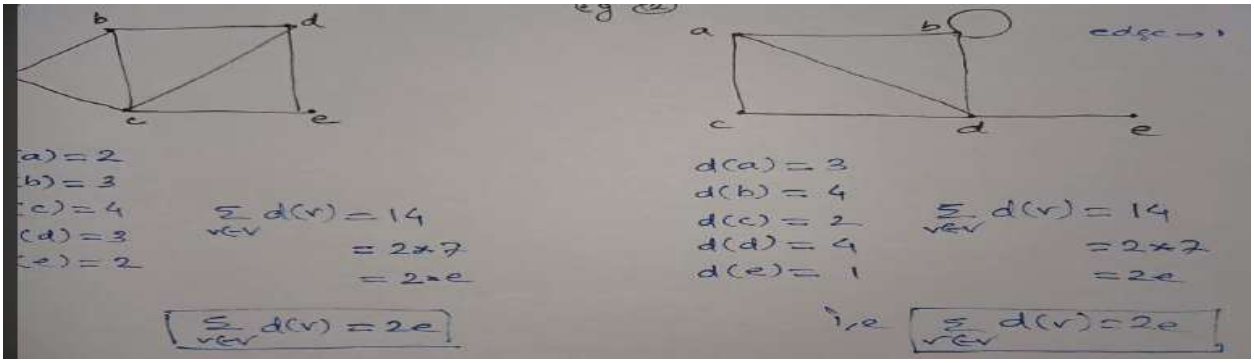


Theorem (The Handshaking Theorem)

The sum of the degrees of the vertices of a graph is equal to twice the number of edges



Proof: Consider a graph G = (V,E) with n vertices v1,v2,v3…vn and |E|, the number of edges in G. Since each edge contributes a degree of 2, the sum of the degrees of all the vertices in G is twice the number of edges in G. That is,$\sum_{i=1}^{n} d(vi) = 2|E|$



Theorem: An undirected graph has an even number of vertices of odd degree.

<u>Theorem</u>:- In a undirected graph, the total number of odd degree vertices is even.

<u>Proof</u>:-

Let $G=(V,E)$ a undirected graph. Let U denote the set of <u>even</u> degree vertices in G and W denote the set of <u>odd</u> degree vertices.

Then

$$\sum_{v_i \in V} \deg(v_i) = \sum_{v_i \in U} \deg(v_i) + \sum_{v_i \in W} \deg(v_i)$$

$$\Rightarrow \underbrace{2e}_{\text{even}} - \underbrace{\sum_{v_i \in U} \deg(v_i)}_{\text{even}} = \sum_{v_i \in W} \deg(v_i) \quad \text{——— (1)}$$

Now $\sum_{v_i \in W} \deg(v_i)$ is also even.

Therefore, from (1)

$\sum_{v_i \in W} \deg(v_i)$ is even.

$\therefore$ the number of odd vertices in G is even.

Proof: Let G be a graph. If G contains no odd vertices, then the result follows immediately. Suppose that G contains K number of odd vertices which are v_1,v_2,v_3……v_k and n number of even vertices which are u1,u2,u3,,,,,un. Clearly, d(u1)+d(u2)+…+d(un) = even

By handshaking theorem,

$[d(v_1)+d(v_2)+\ldots\ldots+d(v_k)]+[d(u_1)+d(u_2)+\ldots\ldots+(d(u_n)] = 2|E|$

Where |E| is the total number of edges in G

Then,

$d(v_1)+d(v_2)+\ldots\ldots+d(v_k) = 2|E|- d(u_1)+d(u_2)+\ldots\ldots+(d(u_n)$

$= 2|E|$-even

=even

Here, sum of the degrees of odd vertices is even, to hold this condition K must be even. That is, G has an even number of odd vertices. If G has no even vertices then we have

d(v1)+d(v2)+……..+d(vk) = 2|E|, is even

From which we again conclude that K is even. This completes the proof of the theorem.

Example 1 : Find the sum of the degrees of the vertices of the graph G(V,E) where V(G) = {v1,v2,v3,v4,v5} and E(G) = {(v1,v2),(v1,v4),(v1,v5),(v2,v5),(v2,v3),(v3,v4)}

Solution:

Number of edges in G = |E| = 6

Sum of degrees of vertices in G = $\sum_{i=1}^{5} d(vi) = 2|E| = 2*6 = 12$

Example 2 : How many vertices do the following graphs have if they contain

a) 16 edges and all vertices of degree 2
b) 21 edges, 3 vertices of degree 4 and all other vertices of degree 3

Solution: Let S be the sum of degrees of vertices, |E| be the number of edges and n be the number of vertices in G. Then,

a) |E| = 16, let there are n vertices of degree 2. Since S = 2*|E|, then
   Number of vertices * equal degree of each vertex = 2*|E|
   Or n*2 = 2*16
   Or n = 16
b) Since S = 2|E|, then
   3*4+(n-3)*3 = 2*21
   Or 12+3n-9 = 42
   Or 3n = 39
   Or n = 13

Example 3 : Show that the maximum number of edges in a simple graph with n vertices is $\frac{1}{2}n(n+1)$

Solution:

By handshaking theorem,

$\sum_{i=1}^{n} d(vi) = 2|E|$

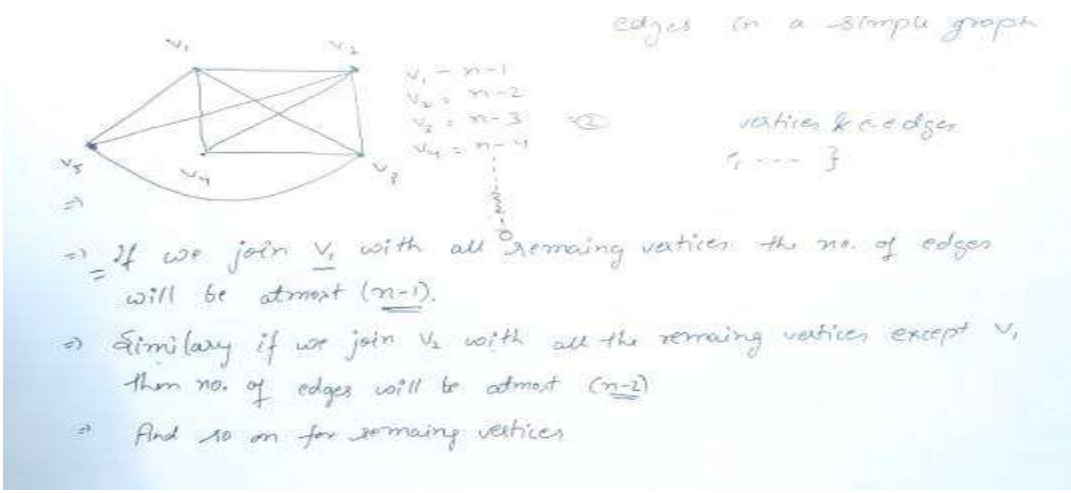
Where |E| is the number of edges in G with n vertices

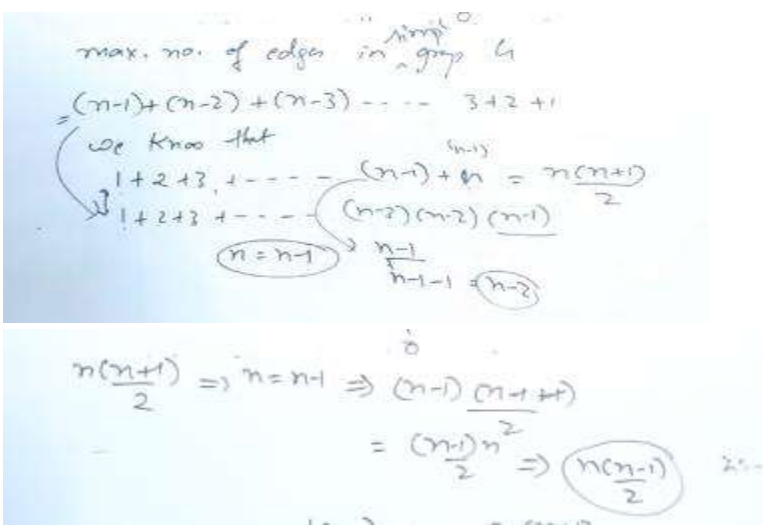
Or $d(v_1)+d(v_2)+.....+d(v_n) = 2|E|$

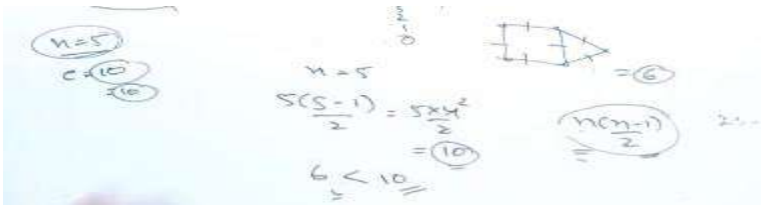
Since, the maximum degree of each vertex in the graph G can be (n-1) then,

(n-1)+(n-2)+..... ..... to n terms = 2|E|

$n(n-1) = \frac{1}{2}n(n-1)$

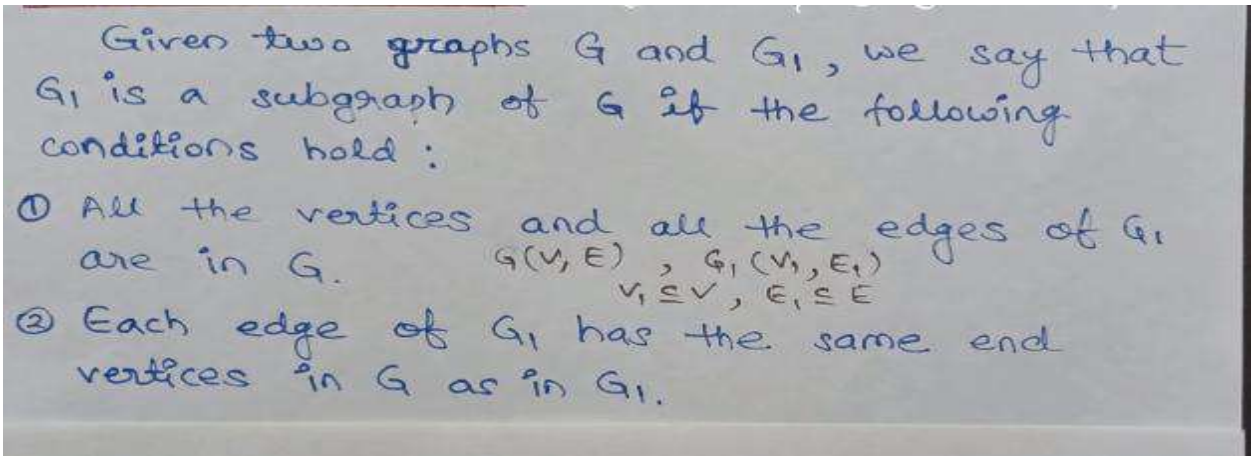


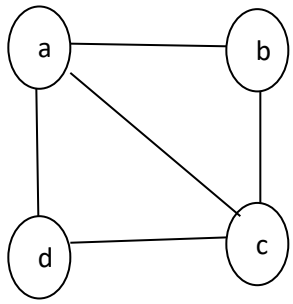




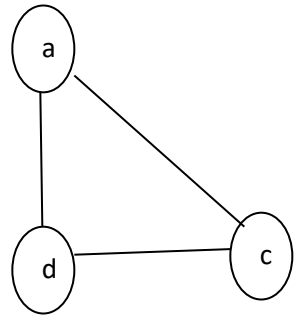
Subgraph: Let G = (V.E) be a graph with vertex set V(G) and edges set E(G) and H = (V,E) be a graph with vertex set V(H) and edge set E(H) then H is said to be sub-graph of G written as V(H) ⊆ V(G) and E(H) ⊆ E(G) . If H is a Subgraph of G , then

a. All the vertices of H are in G
b. All the edges of H are in G
c. Each edge of H has the same end points in H as in G



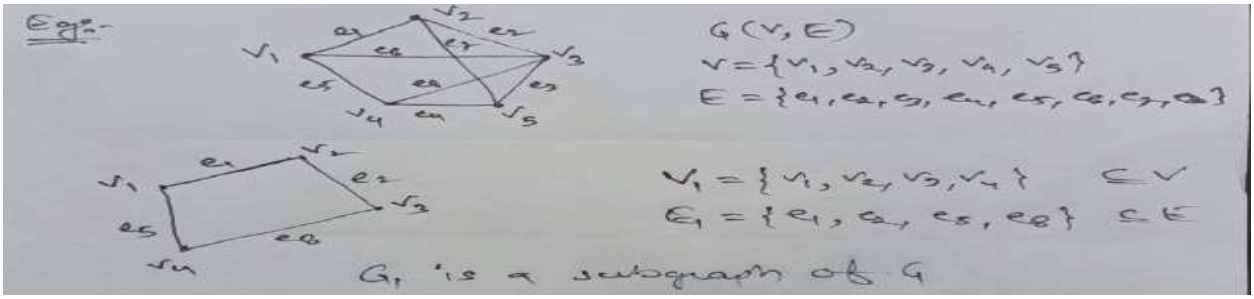


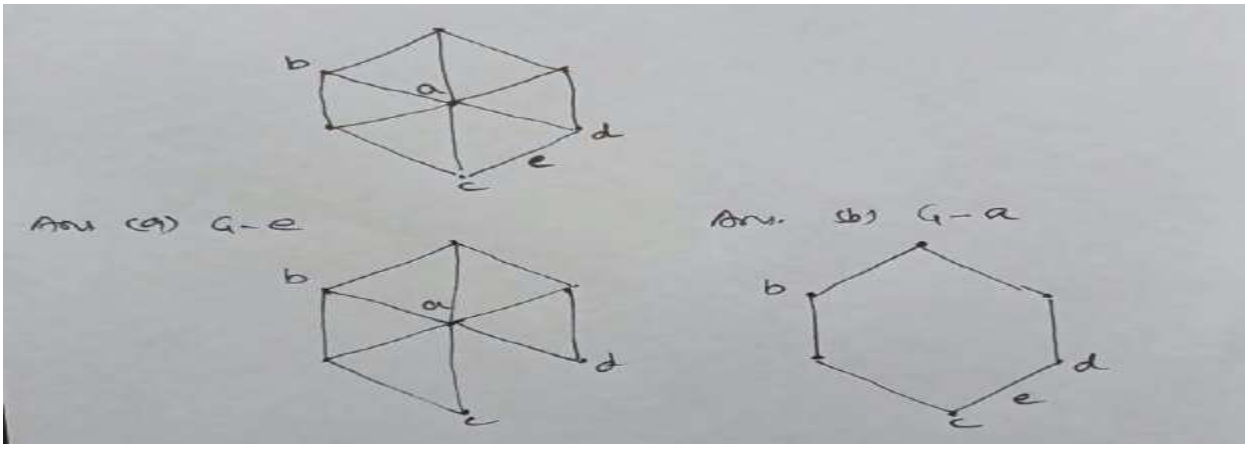
Graph G_1



Graph G_2

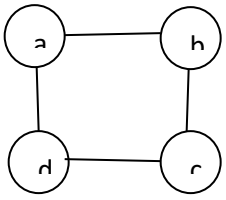
In the above figures, G_2 is subgraph of G_1



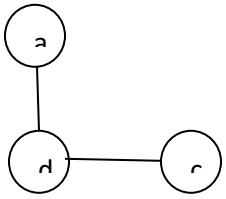


Spanning Subgraph

The Sub-graph H of graph G is said to be spanning sub-graph if V(H) = V(G). that is H and G has exactly same vertex set

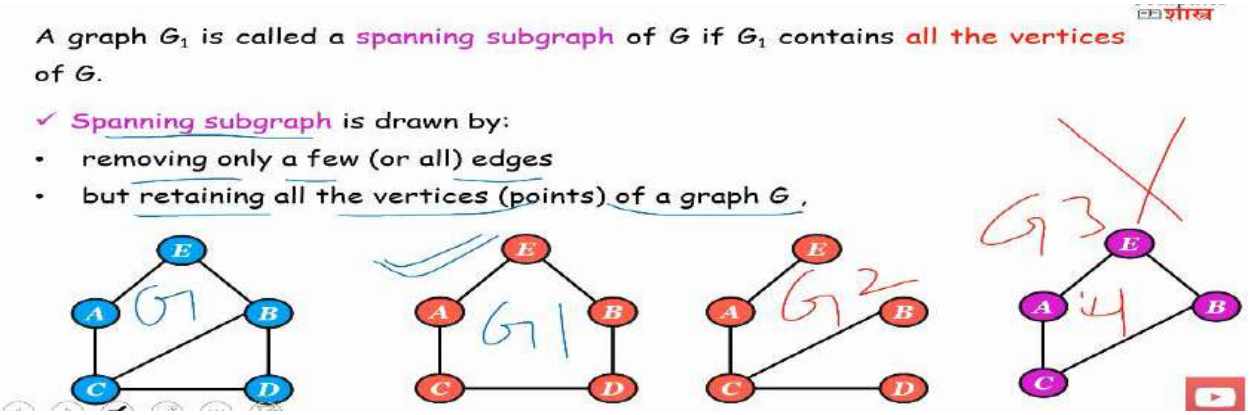


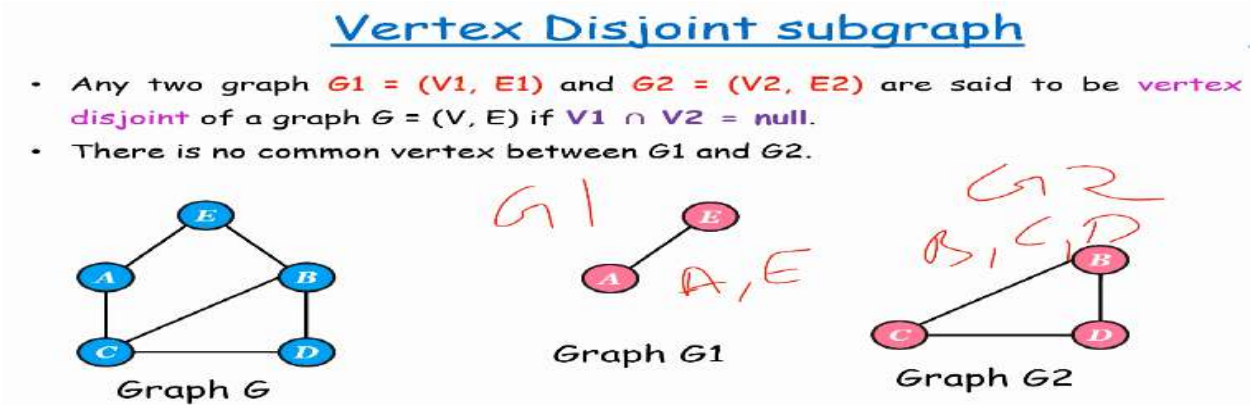
Graph G_1

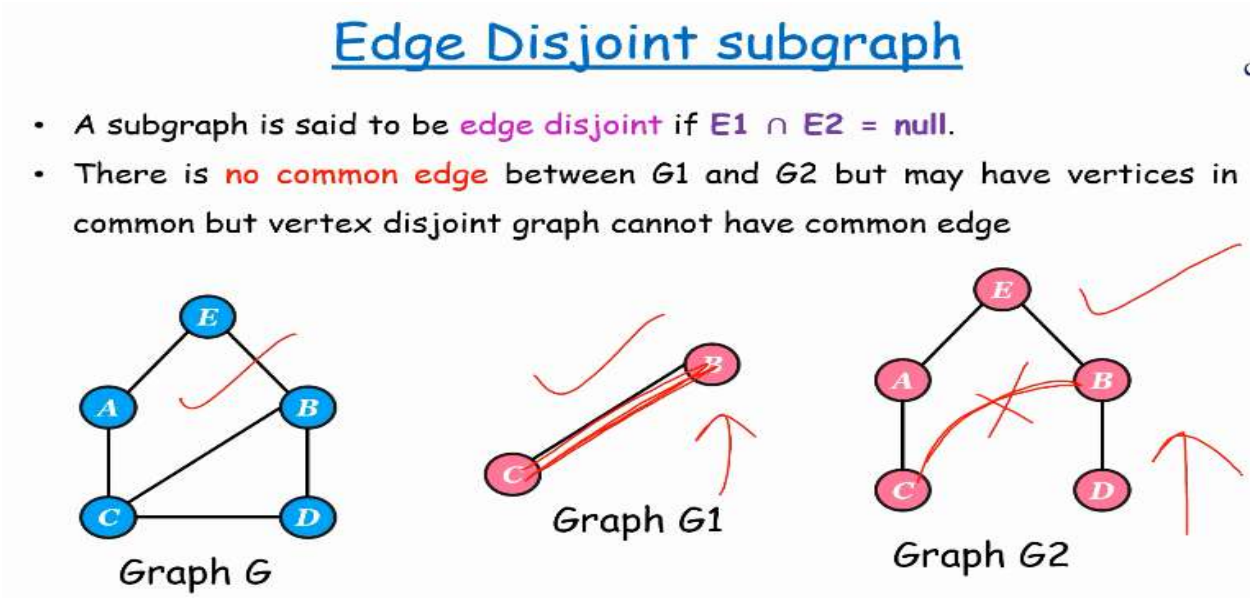


Graph G_2

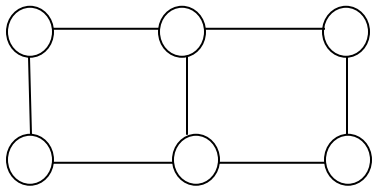
Graph G_2 is spanning subgraph of G_1



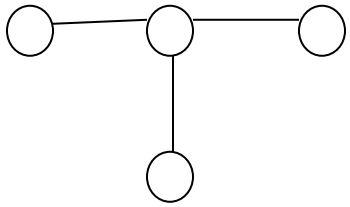




Induced Sub-graph: If G is a graph with vertex set V and U is a subset of v, then the Subgraph G(U) with the vertex set U and with edge set consisting of those edges of G that have both ends in U, is called the induced Subgraph of G induced by the vertices in U.



Graph G_1



Graph G_2

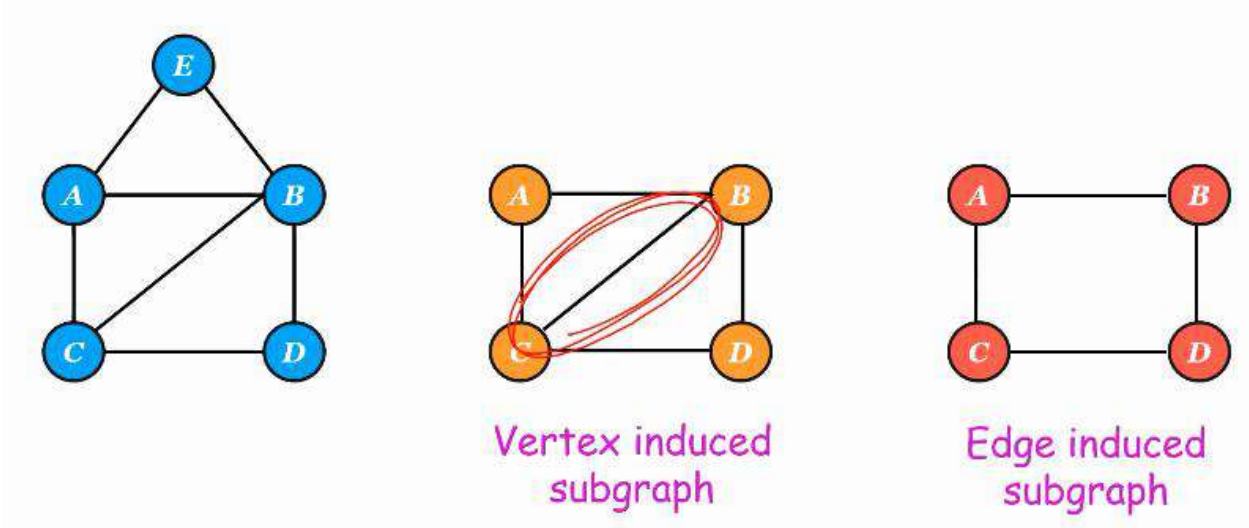
Here G_2 is an induced subgraph of G

Vertex Induced Subgraph

A *vertex-induced subgraph* is one that consists of some of the vertices of the original graph and all of the edges that connect them in the original graph.

Edge Induced Subgraph

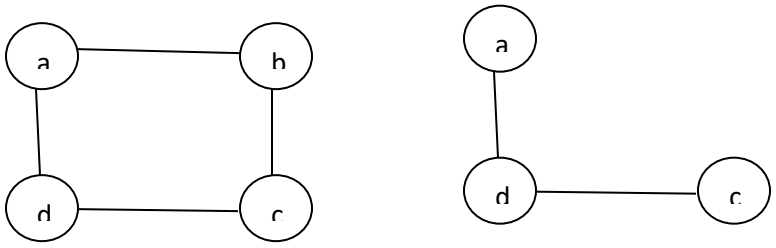
An *edge-induced subgraph* consists of some of the edges of the original graph and the vertices that are at their endpoints.



Vertex deleted sub-graph

Let G = (V,E) be a Subgraph and S be a non-empty subset of v. The induced Subgraph denoted by G-S is a sub-graph obtained by deleting vertices in S. In this method, two form a Subgraph

1. Removal of all vertices from V(G) which are in S
2. Remove all the edges which are incident on vertices in S

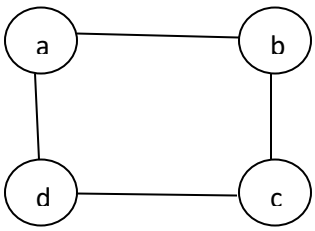


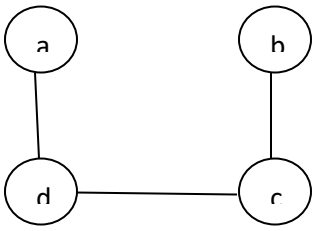
Graph G Graph G-b

Edge deleted Subgraph

If a subset F of E is deleted from G then G-E denoted the Subgraph of G with vertex V and edge E-F, then G-F is called an edge deleted Subgraph

Example: For the graph G shown below, draw the subgraphs





Graph G **Graph G-(a,b)**

Union and Intersection of Graphs

Given two graphs G_1 and G_2, their union will be a graph such that

$V(G_1 \text{ U } G_2) = V(G_1) \text{ U } V(G_2)$

And

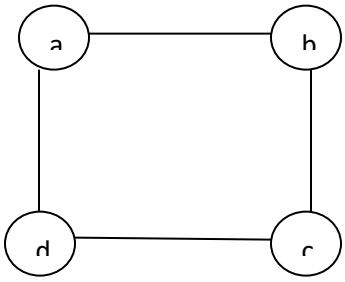
$E(G1 \text{ U } G2) = E(G_1) \text{ U } E(G_2)$

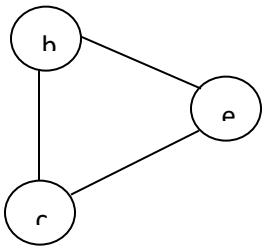
Intersection

Given two graphs G_1 and G_2, their intersection will be a graph such that

$V(G_1 \cap G_2) = V(G_1) \cap V(G_2)$

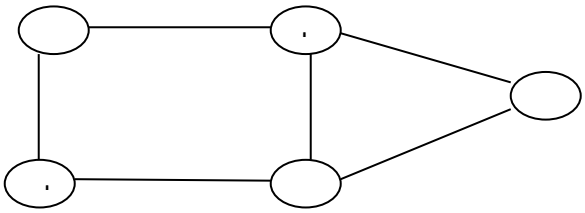
$E(E_1 \cap E_2) = E(G_1) \cap E(G_2)$



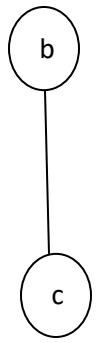


Graph G_1 Graph G_2

Now G_1 U G_2 is the graph

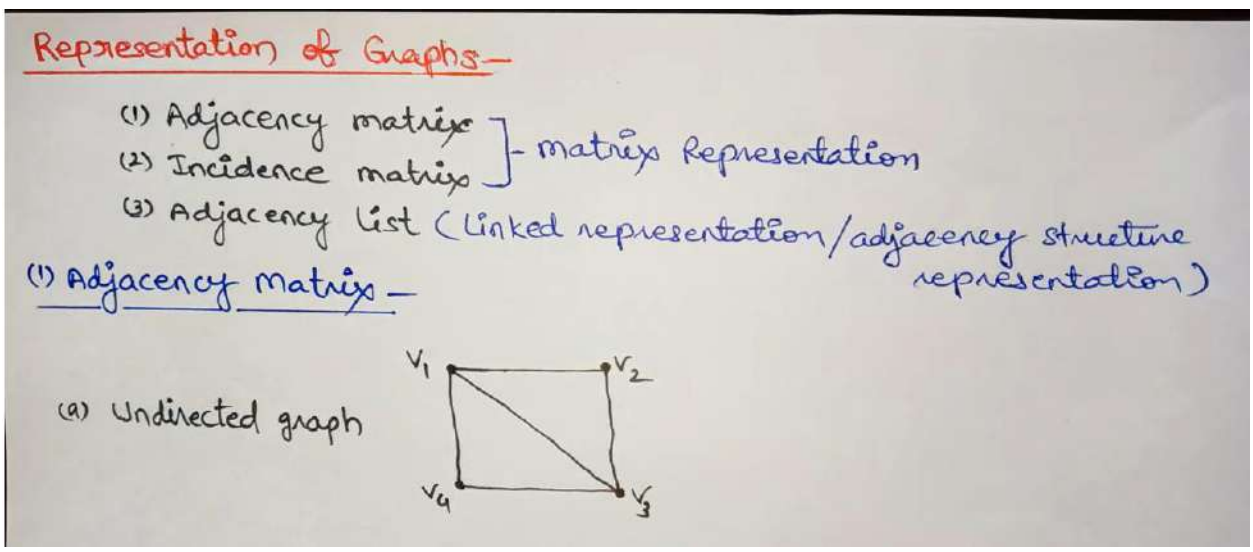


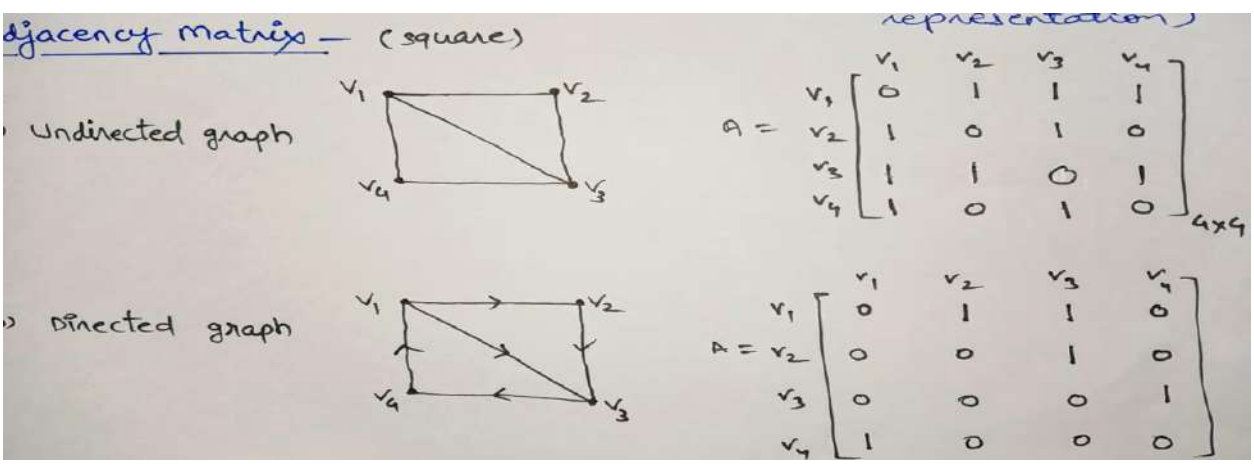
And G1 ∩ G2 is the graph



Representation of Graph

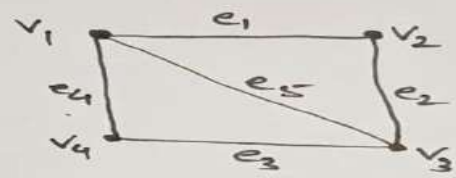
Graphs can be represented in many ways. One of the representation methods of graph is Matrix representation based on adjacency of vertices called adjacency matrix representation and another is based on vertices and edges called incidence matrix. The other representation of graph is adjacency list, based on the list of adjacent vertices.



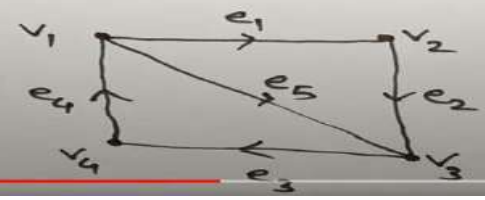


(2) Incidence Matrix –

(a) Undirected graph –



(b) Directed graph –

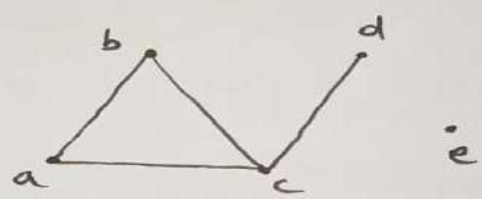


$$\begin{array}{c|ccccc} & e_1 & e_2 & e_3 & e_4 & e_5 \\ \hline v_1 & 1 & 0 & 0 & 1 & 1 \\ v_2 & 1 & 1 & 0 & 0 & 0 \\ v_3 & 0 & 1 & 1 & 0 & 1 \\ v_4 & 0 & 0 & 1 & 1 & 0 \end{array}$$

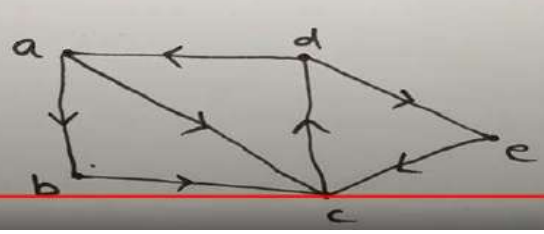
$$\begin{array}{c|ccccc} & e_1 & e_2 & e_3 & e_4 & e_5 \\ \hline v_1 & 1 & 0 & 0 & -1 & 1 \\ v_2 & -1 & 1 & 0 & 0 & 0 \\ v_3 & 0 & -1 & 1 & 0 & -1 \\ v_4 & 0 & 0 & -1 & 1 & 0 \end{array}$$

(3) Adjacency List –

(a) Undirected graph –



(b) Directed graph –



9:08 / 12:00

vertex	adjacency list
a	b, c
b	a, c
c	a, b, d
d	c
e	ϕ

vertex	adjacency list
a	b, c
b	c
c	d
d	a, e
e	c

Adjacency Matrix Representation of Graph

Let G = (V,E) be a graph with n vertices $v_1, v_2, v_3 \ldots v_n$. The adjacency matrix of G with respect to given ordered list of vertices is a n*n matrix denoted by $A(G) = (a_{ij})m \times n$ such that

$a_{ij} =$ 0 if there is no edge between the vertices v_i and v_j

=1 if there is edge between the vertices v_i and v_j

= k if there is k edges between the vertices v_i and v_j

Example: Find the adjacency matrix to represent the graph shown in figure given below

Incidence Matrix: Let G be a graph with vertices $v_1, v_2, v_3 \ldots\ldots v_m$ and edges $e_1, e_2, e_3 \ldots\ldots e_n$. The incidence matrix I(G) of graph G is a m*n matrix with $I(G) = (m_{ij})m*n$ where

$m_{ij} = 1$ if e_j is incident with vertex v_i

= 0 if e_j is not incident with v_i

= 2 if v_i is end of the loop

Example: Find the incidence matrix to represent the graph shown in figure below

Adjacency List

One of the ways of representing a graph without multiple edges is by listing its edges. This type of representation is suitable for the undirected graphs without multiple edges and directed graphs. This representation looks as in the tables below

Vertex	Adjacent Vertices
A	b,c
B	a,d
C	a,d
D	b,c

Graph Connectivity

Walk -

A walk in a graph G is a finite sequence

$$W = v_0 e_1 v_1 e_2 v_2 e_3 \cdots\cdots v_{K-1} e_K v_K$$

whose terms are alternately vertices and edges such that, for $1 \leq i \leq K$, the edge e_i has ends v_{i-1} and v_i

Trivial Walk -

A trivial walk is one containing no edges.

Closed and open walk -

Given two vertices u and v of a graph G, a $u-v$ walk is called closed or open depending on whether $u=v$ or $u \neq v$.

Trail -

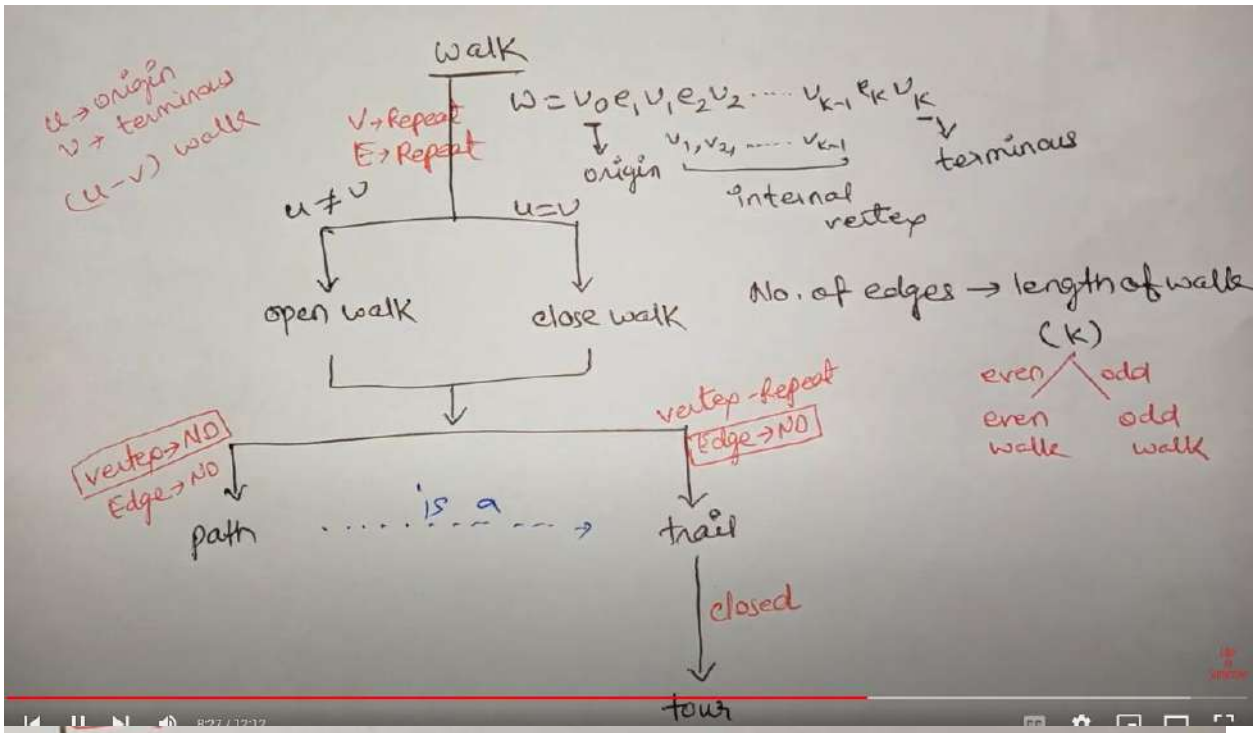
If the edges $e_1, e_2, \cdots, e_K$ of the walk $W = v_0 e_1 v_1 e_2 v_2 \cdots e_K v_K$ are distinct then w is called a trail. | A closed trail is called a tour.

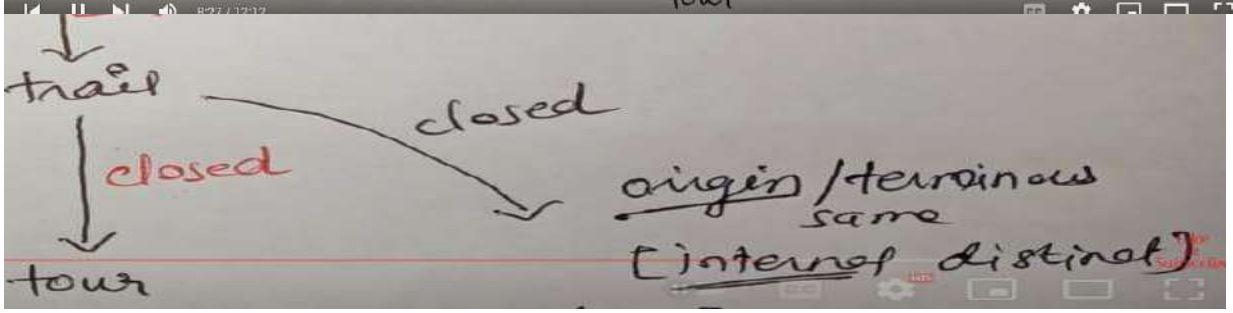
path -

If the vertices $v_0, v_1, \cdots, v_K$ of the walk $W = v_0 e_1 v_1 e_2 v_2 \cdots e_K v_K$ are distinct then w is called a path.

Cycle (circuit) -

A nontrivial closed trail in a graph G is called a cycle if its origin and internal vertices are distinct.





Walk : A walk in a graph G is a finite ordered set W $= (v_0,e_1,v_1,e_2,v_2,e_3,\ldots\ldots v_{k-1}\ e_k\ ,\ v_k\ )$ whose elements are alternately vertices and edges such that $1 \leq i \leq k$, the edge e1 has ends v_{i-1} and v_1

This walk W is a V_0-V_k walk or walk from V_0 to V_k. The number of edges appearing in the sequence of the path is called its length. If the length of the walk is zero that is, the walk has no edges, it contains only a single vertex and is called a trivial walk. A walk is closed if its starts and ends at the same point, otherwise the walk is open.

Trail

A walk W(U,V) in which all the edges are distinct, is called a trail

Path

A walk in which all the vertices and edges are distinct, is called a path

Circuit

A closed trail which contains at least three edges is called a circuit

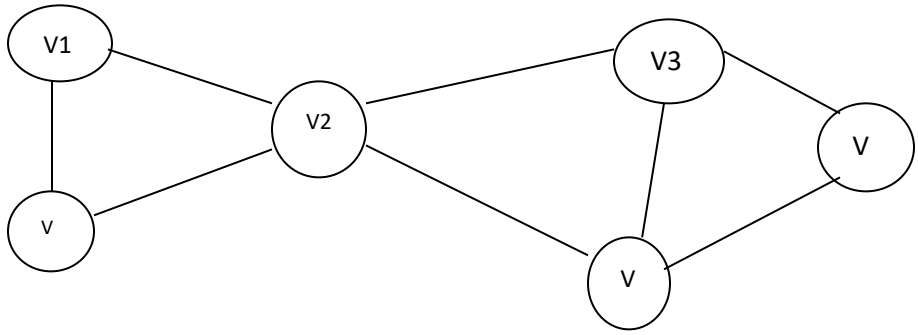
Cycle

A circuit which does not repeat any vertices (except the initial and final vertex) is called a cycle. Thus, a cycle is non-intersecting circuit and must have length three or more. A cycle of length K is called K-cycle. It should be noted that while every cycle is a circuit, the converse is not always true. A circuit may have repeated vertices other than the end vertices, but in a cycle the only repeated vertices are the first and last.

The definitions are summarized in the following table

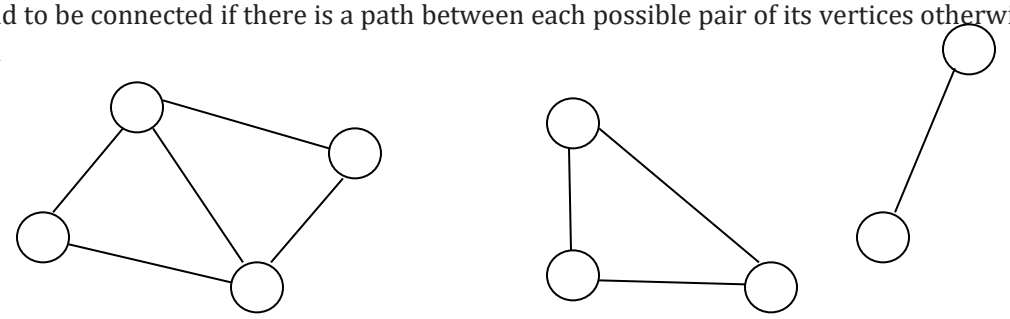
Term	Repeated Edge	Repeated vertex
Path	No	No
Trail	No	Allowed
Walk	Allowed	Allowed
Circuit	No	Allowed
Cycle	No	First and last only

Example: Consider the graph in the figure below, the closed trail (v_1,v_2,v_3,v_4,v_5,v_2,v_6, v_1} is a circuit but not a cycle, while the closed trail (v_2,v_3,v_4,v_5,v_2} is a cycle as well as a circuit.



Connected Graph

A graph G is said to be connected if there is a path between each possible pair of its vertices otherwise G is disconnected



Connected Graph Undirected Graph

Connected Graph – [undirected]

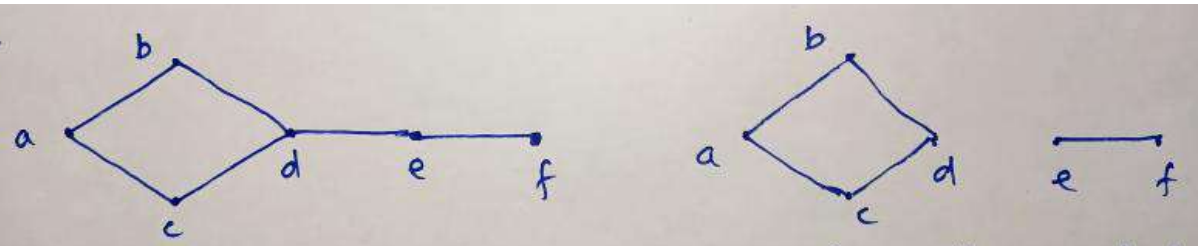
Two vertices u and v are said to be connected if there exists a path between them.

If there is a path between two vertices u and v, then u is said to be reachable from v and vice versa.

A graph G is said to be connected if there exist paths between any two vertices in G.

A graph that is not connected is called disconnected.

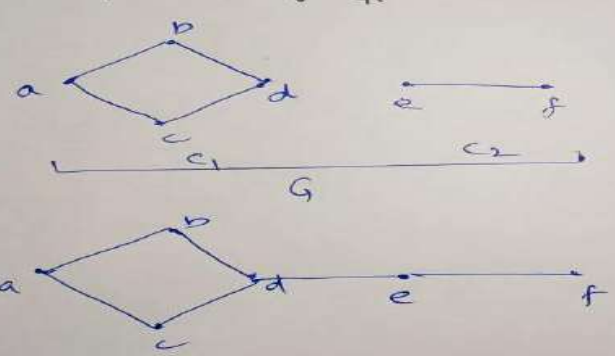
Eg:



G_1 : connected G_2 : disconnected

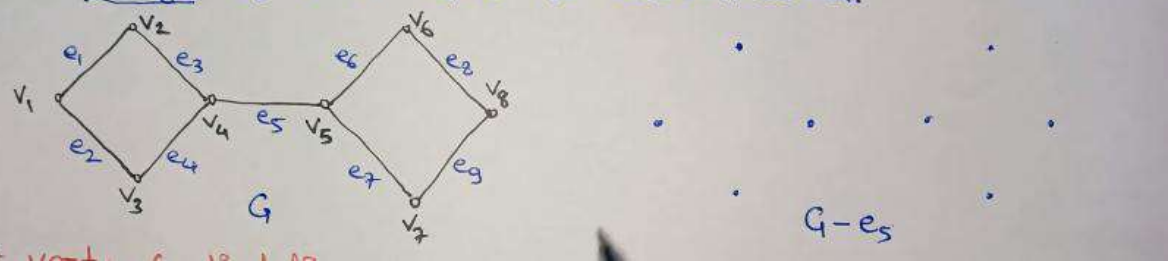
A maximal connected subgraph of G is a component of G.

Eg:-



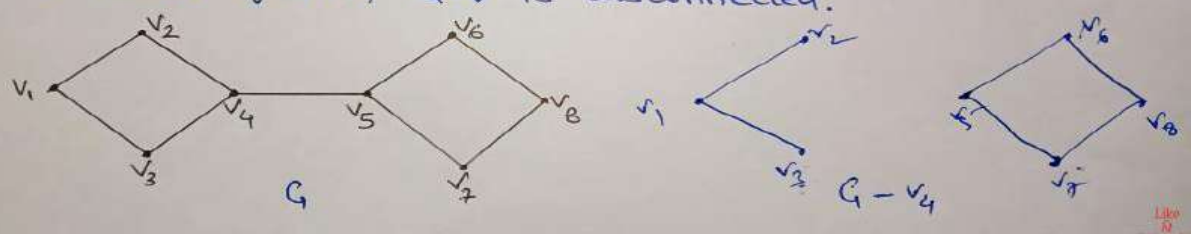
Cut-Edge -

An edge e of a graph G is said to be a cut-edge or a bridge of G if G-e is disconnected.



Cut-vertex (articulation point / cut-point) -

A vertex v of a graph G is said to be a cut vertex of G if G-v is disconnected.

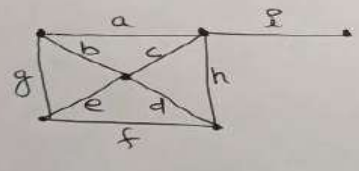


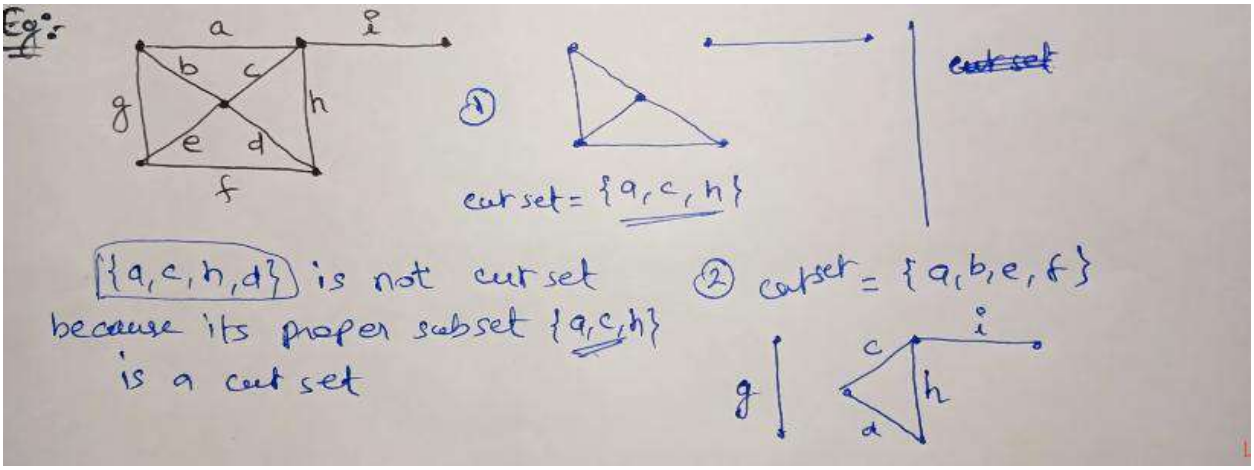
Cut set -

The set of all minimum number of edges of G whose removal disconnects a graph G is called a cut-set of G. Thus a cut set S of a satisfy the following :-

1. S is a subset of the edge set E of G
2. Removal of edges from a connected graph G, disconnected G.
3. No proper subset of G satisfy the condition.

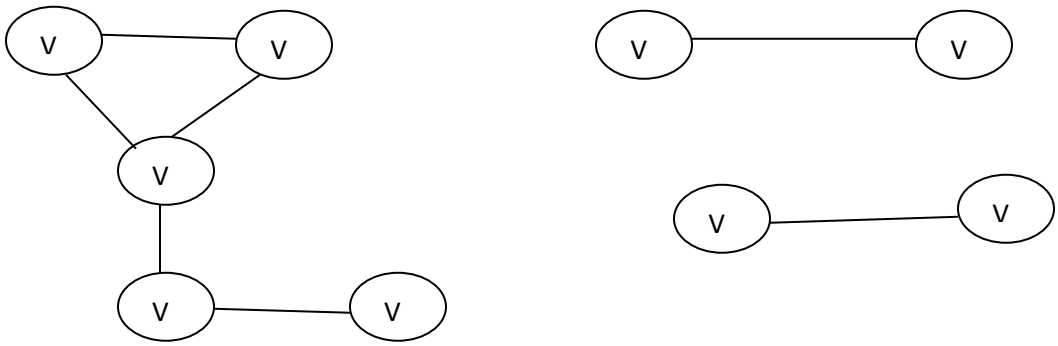
Eg:-





Cut vertex

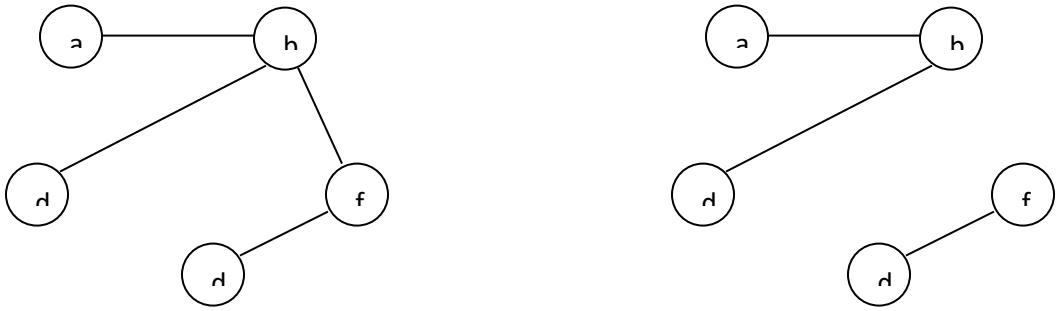
A vertex v of a connected graph is called a cut –vertex (or cut-point) if G-v is disconnected. Any vertex in a graph is said to be cut vertex if graph becomes disconnected after removal of this vertex from the graph.



In the above figure, after removal of vertex v_3, graph G becomes disconnected. So, v_3 is cut vertex. Similarly, vertex v_4 is also cut vertex

Cut edge or bridge

Any edge in graph after whose removal becomes disconnected, is called bridge or cut edge. An edge e is a bridge for G if G-e is disconnected



In the above figure, after removal of an edge e = {b,f}, the connected graph G becomes disconnected. So e = {b,f} is a cut edge

Connected Component

A graph that is not connected is the union of two or more connected subgraphs, each pair of which has no vertex in common. These disjoint connected subgraphs are called the connected components of G. In other word, a connected component H or an undirected graph G is maximal connected sub graph. By, 'maximal' we mean that G contains no other subgraph that is both connected and properly contains H

Theorem 4: There is a simple path between every pair of distinct vertices of a connected undirected graph

Proof: Suppose a and b are two distinct vertices of the connected undirected graph G = (V,E). We know that G is connected so by definition there is at least one path between a and b. Let x_0 , x_1 , x_2, x_3…… xn where x=a and x_n = b, be the vertex sequence of a path of a least length. Now if this path of least length is not simple then we have xi = xj for some I and j with $0 \leq i \leq j$. This implies that there is a path from a to b of shorter length with the vertex sequence x_0,, x_1, x_2,….. x_{j+1},..x_n obtained by removing the edges corresponding to the vertex sequence x_{i+1},……x_j. This show that there is a simple path

Isomorphism of Graphs

Two graphs $G_1(V_1, E_1)$ and $G_2(V_2, E_2)$ are isomorphic if there exists a bijective mapping ϕ between $V_1(G_1)$ and $V_2(G_2)$ such that {u,v} is in E_1 if and only if $(\phi(u), \phi(v))$ is in E_2. The function ϕ is called isomorphism.

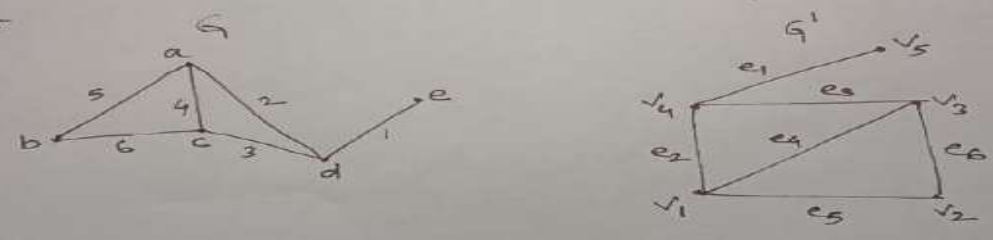
If two graphs G_1 and G_2 are isomorphic then they must have

i. They must have same number of vertices. That is $|V_1| = |V_2|$ where $|V_1|$ is number of vertices in G_1 and $|V_2|$ is number of vertices in G_2
ii. They must have same number of edges. That is, $|E_1| = |E_2|$ where $|E_1|$ is number of edges in G_1 and $|E_2|$ is number of edges in G_2
iii. If $(u,v) \in E_1 \rightarrow (\phi(u), \phi(v)) \in E_2$
iv. If deg (u) = k in G1 then $\deg(\phi(u)) = k$ in G_2
v. $A(G_1) = A(G_2)$ with the respect to orders of vertices v_1, v_2, v_3, v_4…..v_n and $\phi(v_1), \phi(v_2)$,…. $\phi(v_n)$.

Isomorphic Graph -

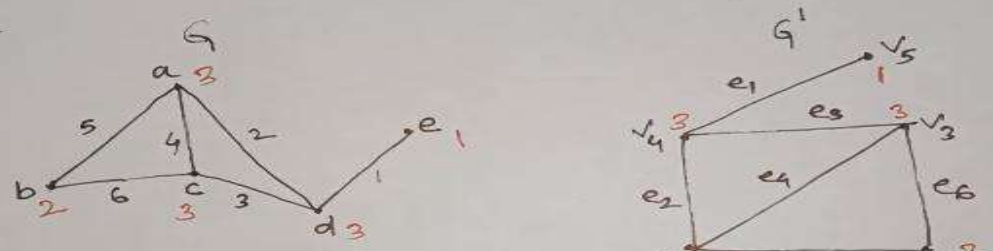
Two graphs G and G' are said to be isomorphic to each other if there is a one-to-one correspondence (bijection) between their vertices and between their edges such that the incidence relationship is preserved.

Eg:-



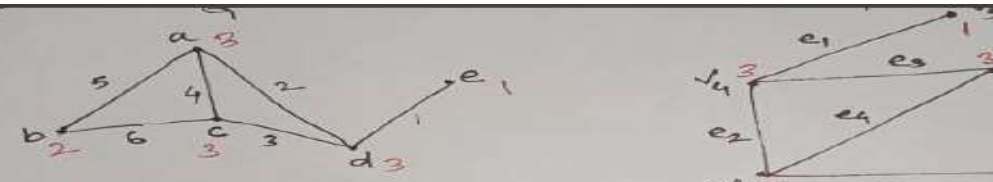
that the incidence relationship is preserved.

Eg:-



degSeq(G) = 3, 3, 3, 2, 1

Eg:-



degSeq(G) = 3, 3, 3, 2, 1

degSeq(G') = 3, 3, 3, 2, 1

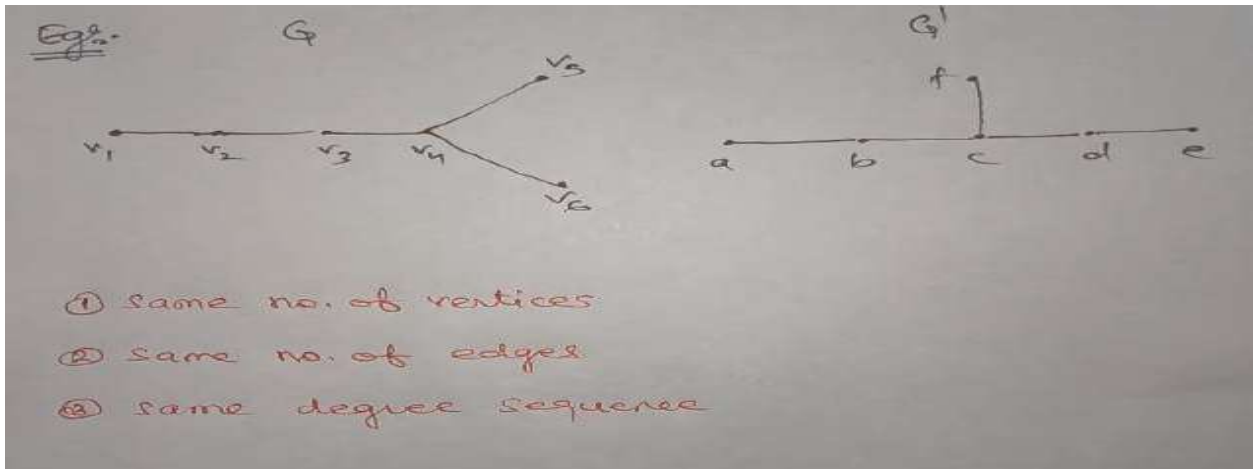
Vertex

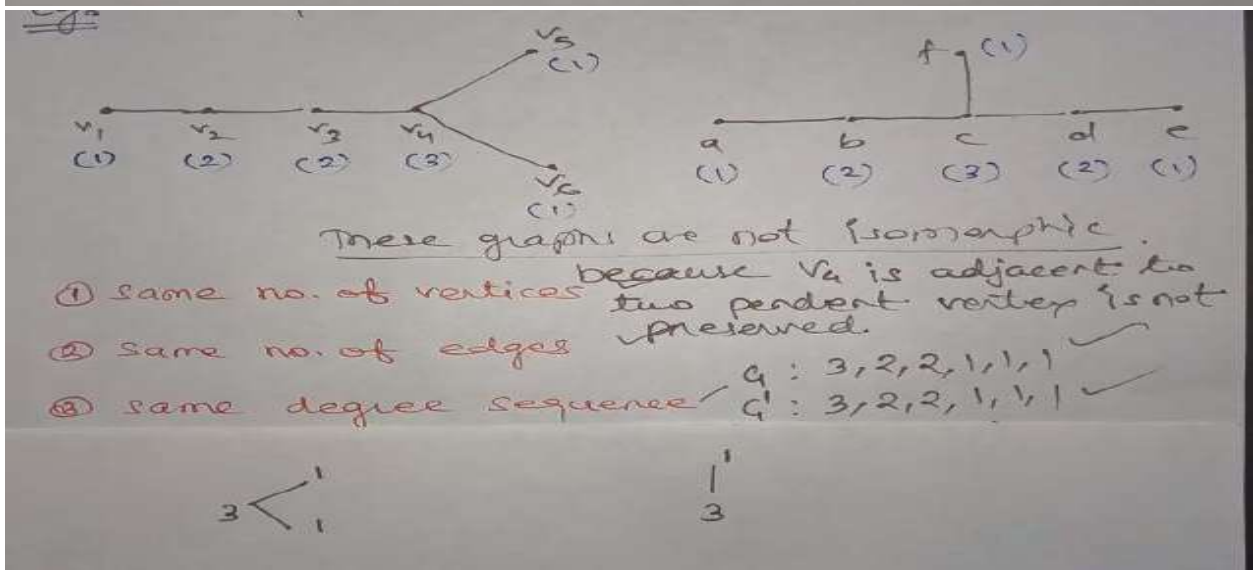
$f(a) = v_1$
$f(b) = v_2$
$f(c) = v_3$
$f(d) = v_4$
$f(e) = v_5$

edges

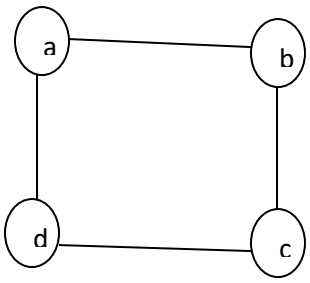
$f(1) = e_1$
$f(2) = e_2$
$f(3) = e_3$
$f(4) = e_4$
$f(5) = e_5$
$f(6) = e_6$

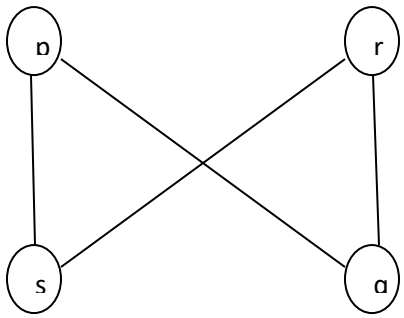
Adjacency also preserved. Therefore G and G' are said to be isomorphic.





Example: Show that the graphs G_1 and G_2 are isomorphic





Graph G_1 Graph G_2

Solution:

The isomorphic invariants for two graphs are:

1. No of vertices in $G_1 = |V(G_1)| = 4$ and no of vertices in $G_2 = |V(G_2)|=4$
2. No of edges in $E_1 = |E(G_1)| = 4$ and no of edges in $G_2 = |V(G_2)| = 4$
3. The degree sequence of vertices (a , b , c , d) in graph $G_1 = (2 , 2 , 2 , 2)$ and degree sequence of vertices (p , q , r , s) in G_2 is (2,2,2,2).
4. Let $\phi: V(G_1) \rightarrow V(G_2)$ be defined as :
   $\phi(a) = p$, $\phi(b) = q$, $\phi(c) = r$ and $\phi(d) = s$
   Then ϕ is one to one and onto because
   $(a ,b) \in E(G_1) \rightarrow (\phi(a), \phi(b)) = (p, q) \in E(G_2)$
   $(b, c) \in E(G_1) \rightarrow (\phi(b), \phi(c)) = (q , r) \in E(G_2)$
   $(c, d) \in E(G_1) \rightarrow (\phi(c), \phi(d)) = (r, s) \in E(G_2)$
   $(d, a) \in E(G_1) \rightarrow (\phi(d), \phi(a)) = (s, p) \in E(G_2)$
5. Adjacency Matrix of G_1 is

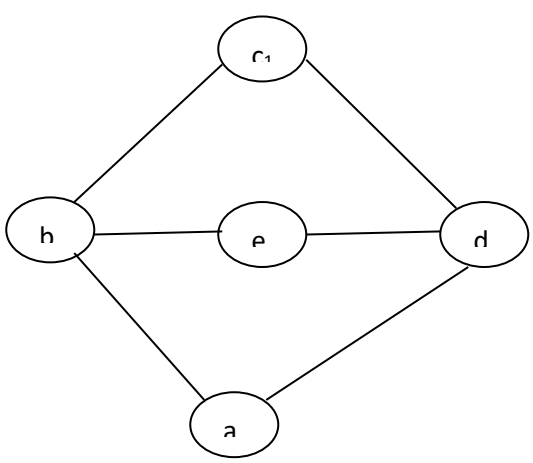
$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

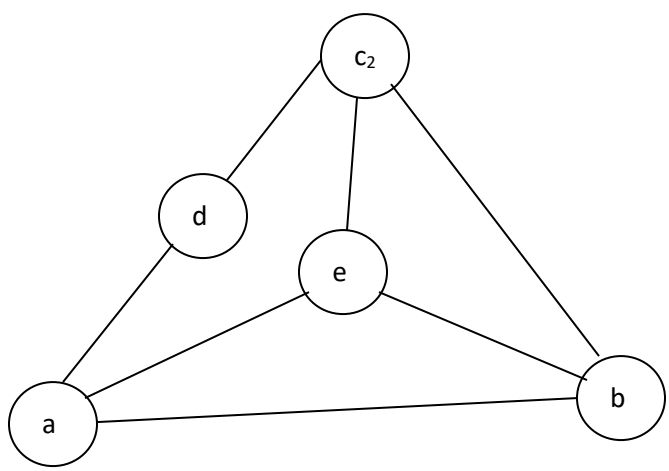
And adjacency Matrix of graph G_2 is

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Since adjacency matrixes of both graphs are same, both graphs are isomorphic

Example: Show that the following graphs G1 and G2 are isomorphic by showing their corresponding adjacency matrices are equal.





Graph G_1 Graph G_2

Solution:

Consider the map ϕ: $G_1 \rightarrow G_2$ defined as $\phi(a_1) = d_2$, $\phi(b_1) = a_2$, $\phi(c_1) = b_2$, $\phi(d_1) = c_2$ and $\phi(e_1) = e_2$. The adjacency matrix of G1 for the ordering a_1, b_1, c_1, d_1 and e_1 is

$A(G_1) =$

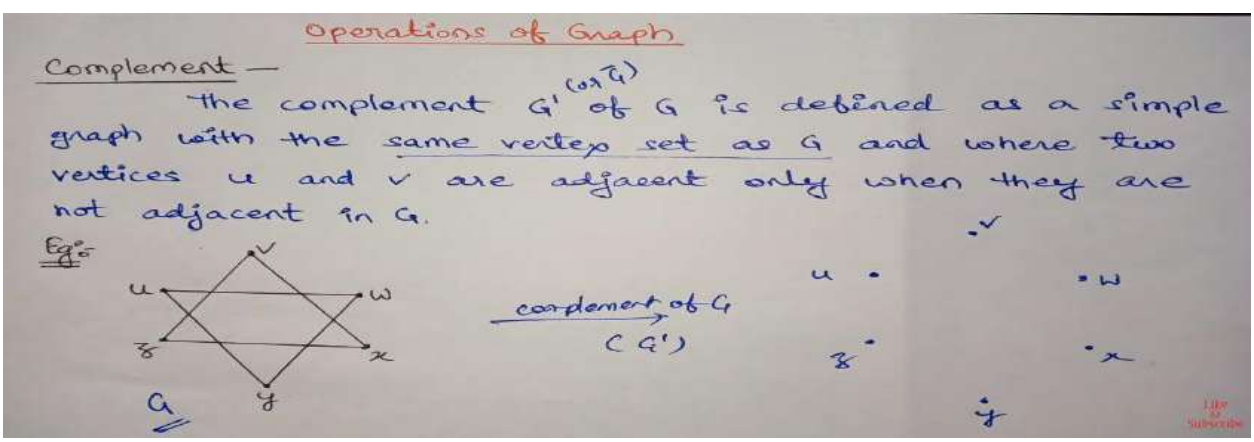
$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

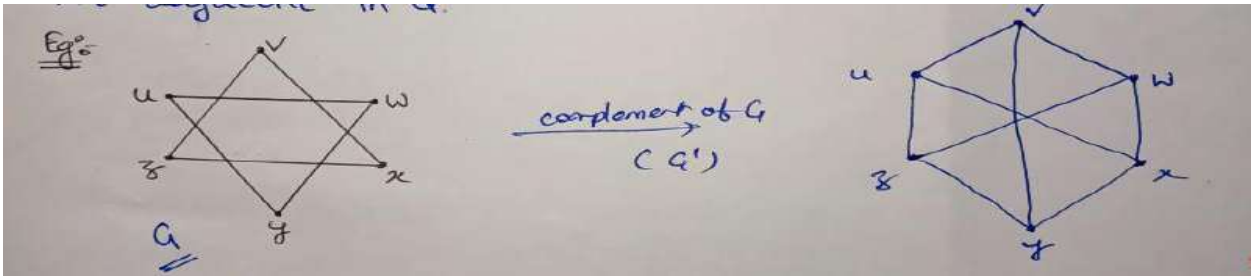
And A(G2) = $\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$

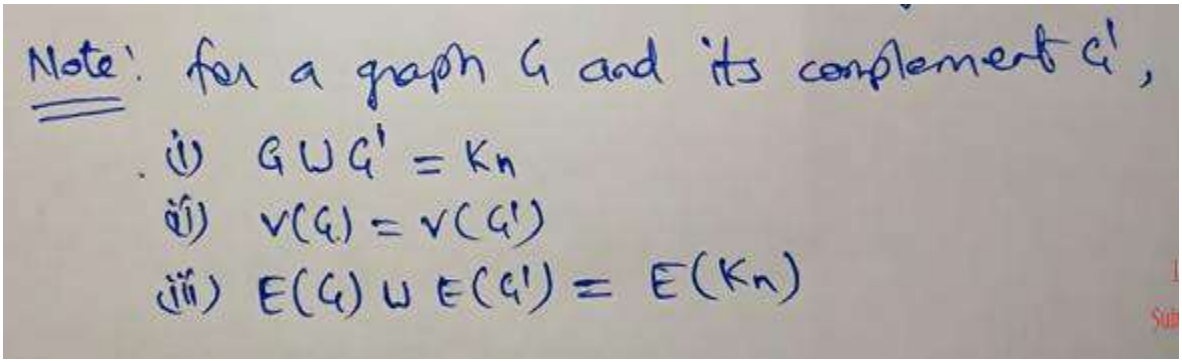
Therefore G1 and G2 are isomorphic graph

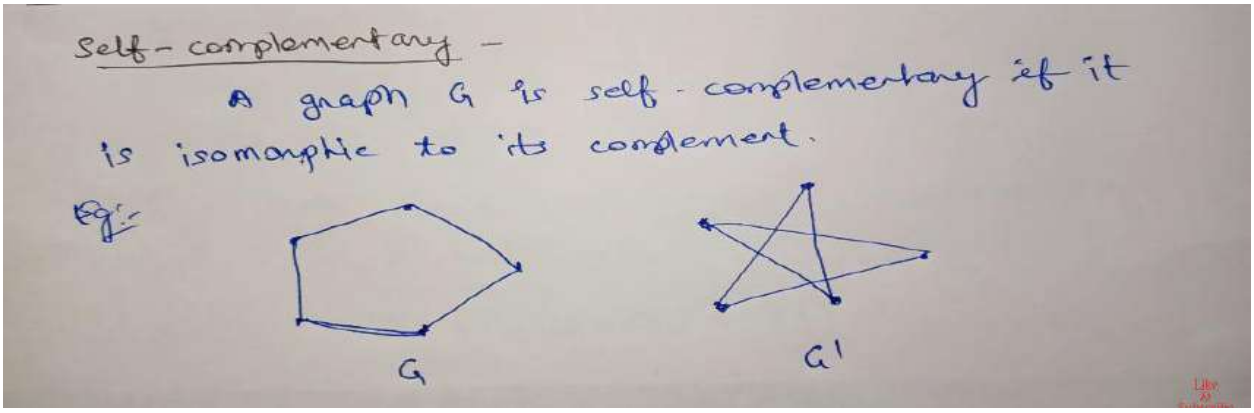
Self Complementary Graph

A graph G = (V,E) is said to be self complementary if G is isomorphic to its complement graph

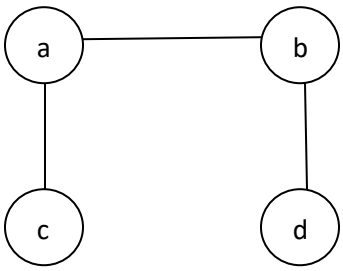




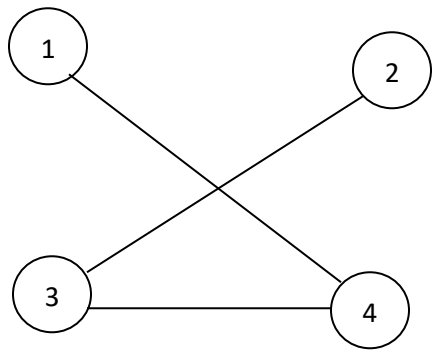




Example: Show that the following graph is self complementary.



Solution: A graph is self complementary if it is isomorphic to its complement. We now find complement G' of the graph G



Here, V(G) = {a ,b, c, d} E(G) = {(a,b),(a, c),(b, d)} and E(G') = {(1,4),(2,3),(3,4)}

Define a function $\phi : V(G) \rightarrow V(G')$ as

$\phi\ (c) = 1, \phi\ (d) = 2, \phi(a) = 4$ and $\phi(b) = 3$

Then ϕ is one to one and onto mapping

$(a, c) \in E(G)$ we have $(\phi(a), \phi(c)) = (4, 1) \in E(G')$

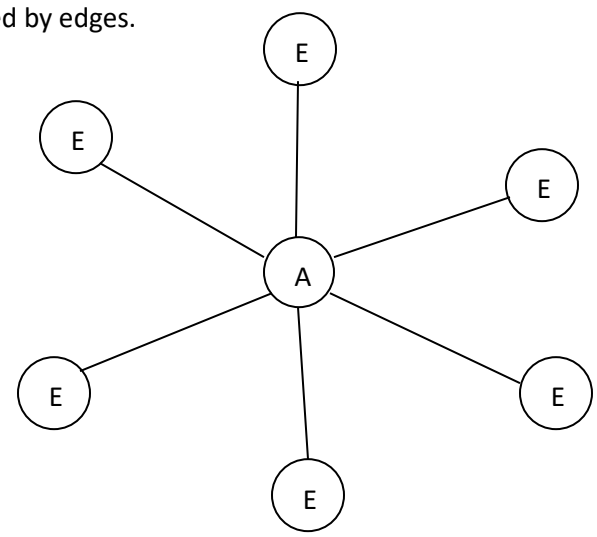
$(a, b) \in E(G)$ we have $(\phi(a), \phi(b)) = (4,3) \in E(G')$

$(b, d) \in E(G)$ we have $(\phi(a), \phi(c)) = (3,2) \in E(G')$

Since G is isomorphic to its complement G', it is self complementary.

Application of Graph on Local Area Network

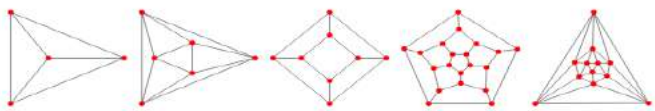
In a local area network, there are many computer connected to each other, which form a special geometrical structure, called network topology. To model local area in the form of topology, graph can be used in which computers or devices are represented by vertices and connections between these devices are represented by edges.



This topology can be modeled using complete bipartite graph $K_{1,6}$

Platonic Graph: The graph formed by the vertices and edges of five regular (platonic) solids- the tetrahedron; octahedron, cube, dodecahedron and icosahedrons are called the platonic graphs.

Wolfram Notebook



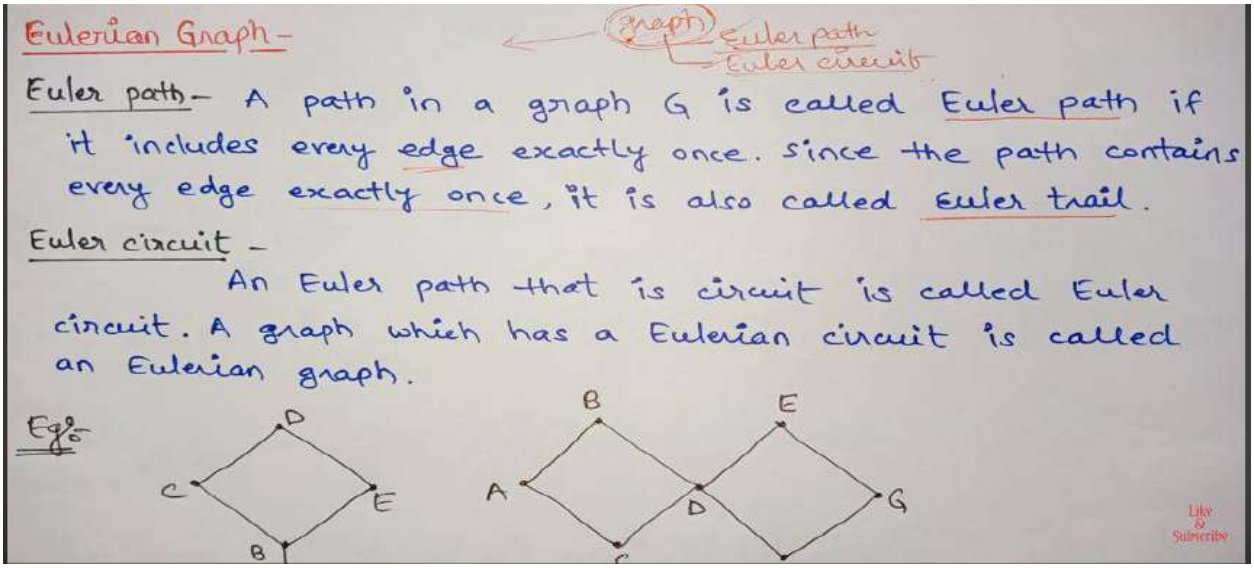
A polyhedral graph corresponding to the skeleton of a Platonic solid. The five platonic graphs, the tetrahedral graph, cubical graph, octahedral graph, dodecahedral graph, and icosahedral graph, are illustrated above. They are special cases of Schlegel graphs.

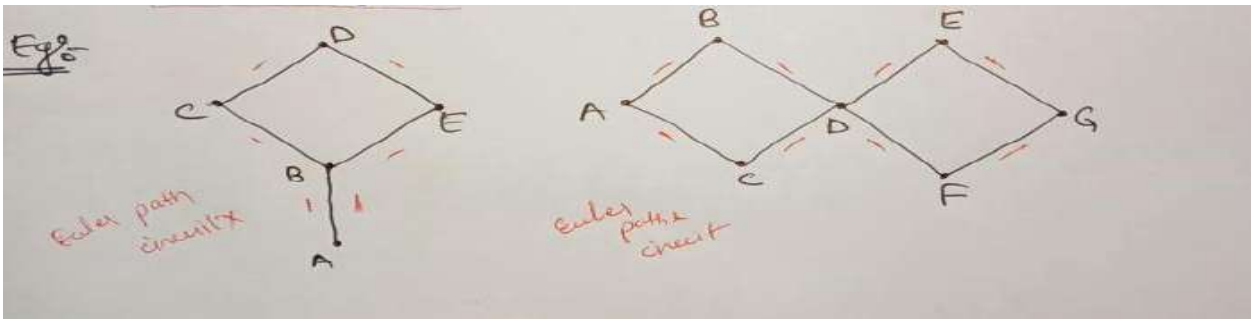
Platonic graphs are graceful (Gardner 1983, pp. 158 and 163-164).

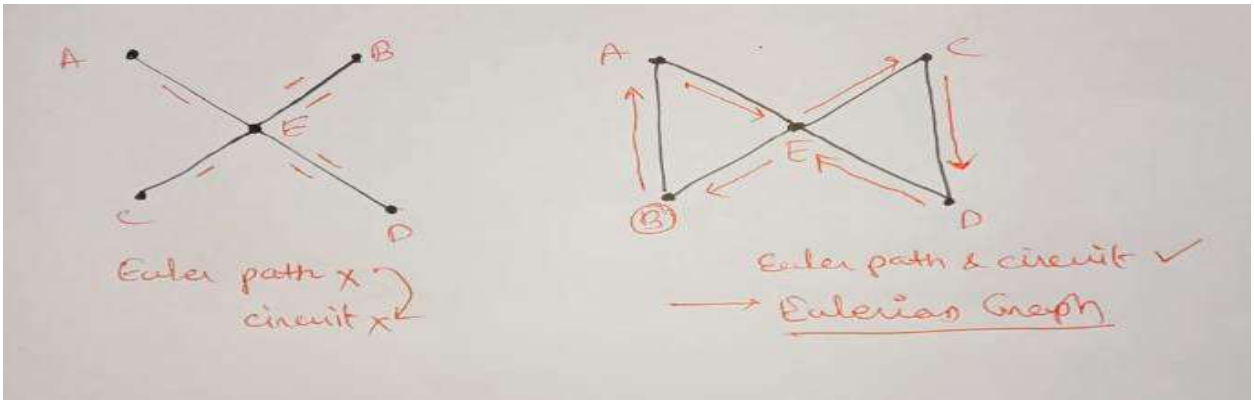
The following table summarizes the Platonic graphs and some of their properties.

graph G	regularity	V	E	$\lvert\mathrm{Aut}\,(G)\rvert$	Hamiltonian	Eulerian	vertex-transitive	edge-transitive
cubical graph	cubic	8	12	48	yes	no	yes	yes
dodecahedral graph	cubic	20	30	120	yes	no	yes	yes
icosahedral graph	quintic	12	30	120	yes	no	yes	yes
octahedral graph	quartic	6	12	48	yes	yes	yes	yes
tetrahedral graph	cubic	4	6	24	yes	no	yes	yes

Euler and Hamiltonian Graphs: The development of the concept of the Euler graph due to the solution of the famous Konigsberg bridge problem by Swiss Mathematician Leonard Euler (1707-1783) in 1936.







The Konigsberg Bridge Problem

- The eighteenth century city of Konigsberg included two islands and seven bridges crossing the Pregel river with and areas denoted by A, B , C and D as shown in the following figure.
- It was asked whether it would be possible to walk around the city by crossing each the bridges exactly once.
- Euler proved in 1736 that such a walk is impossible. He replaced the land area and islands by points and the bridges by curves as shown below.

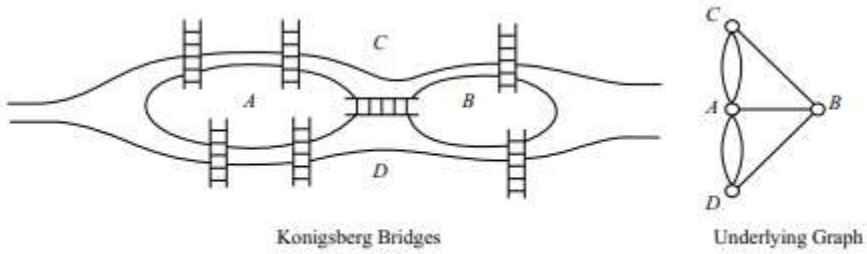


Fig. 3.3

Euler Trail

A trail in a graph G is called an Eulerian trail if it includes every edge of G and then G is called a traversable graph

Eulerian Circuit: A graph containing an Eulerian circuit is called an Eulerian graph or simply Eulerian.

Eulerian Graph: A graph containing an Eulerian circuit is called an Eulerian graph or simply Eulerian.

We summarize the above definitions using a table as

Term	**Initial and Terminal Vertices the same**	**Must include every edge**	**Repeated vertices allowed**
Eulerian Circuit	Yes	Yes	Yes
Eulerian Trail	No	Yes	Yes

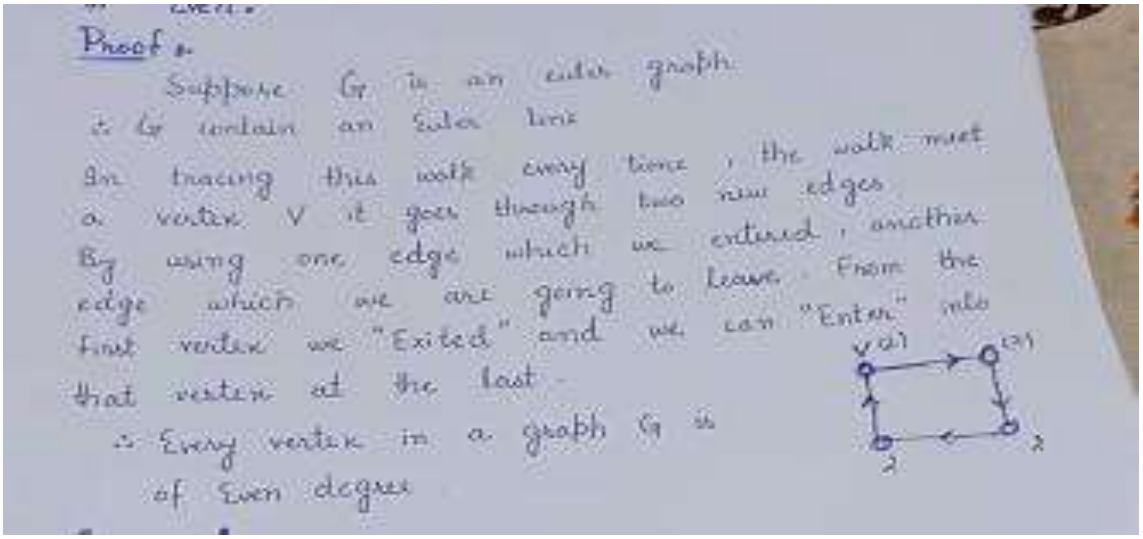
Theorem

A connected graph (multi-graph) G is Eulerian if and only if each vertex has even degree.

Proof:

Take a connected multi-graph G = (V,E) where V and E are finite. We can prove the theorem in two parts. First, we prove that if a connected multi-graph has an Euler circuit, then all the vertices have even degree. For this, take a vertex v, where the Euler circuit begins. There is some edge that is incident to v and some other vertex say u then we have an edge {u,v}. This edge {u,v} contributes one to the degree of v and u both. Again there must be some edge other than {u,v} that is incident to u and some other vertex. In this case, the total degree of the vertex u becomes even, so whenever in the circuit the vertex is met the degree of that vertex is even since every time entering and leaving the vertex gives even degree to all the vertices other than the initial vertex. However, since the circuit must terminate in the vertex v and the edge that is terminating the circuit contributes one to the degree of the initial vertex v the total degree of vertex v is also even. Now, we have every time the vertex is entered and left it gives

the even degree and the initial vertex also gives even degree, we can conclude that if a graph has Euler circuit the all the vertices have even degree.



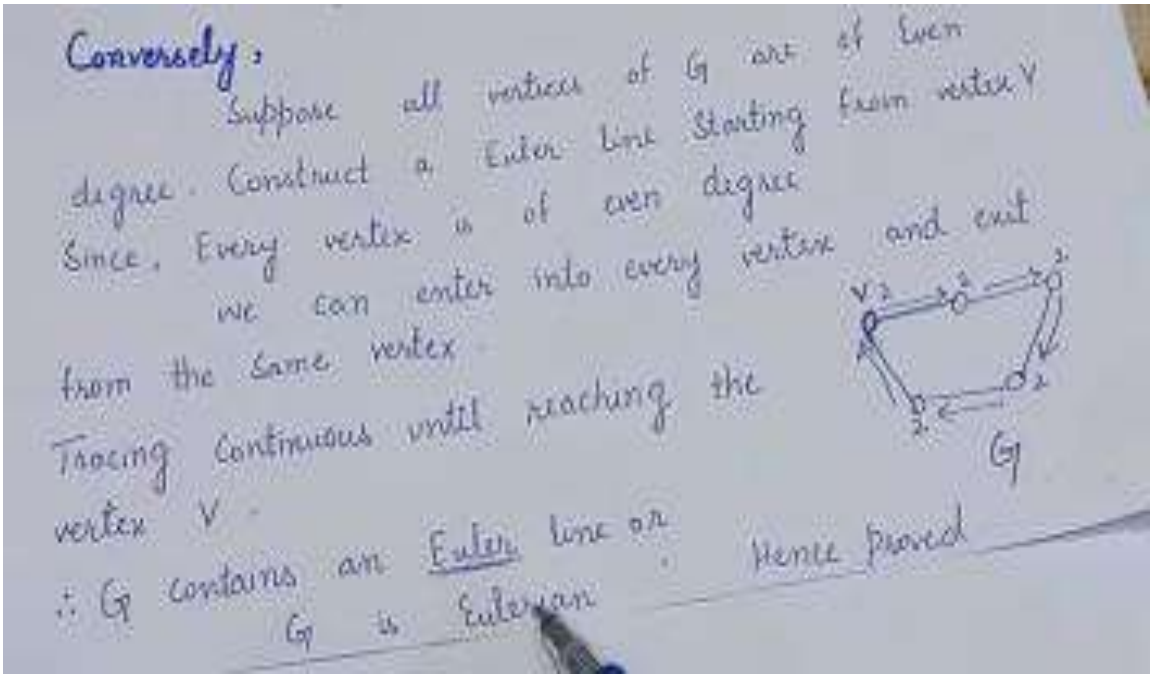
Proof :

Suppose G is an euler graph

∴ G contain an Euler line

In tracing this walk every time, the walk must a vertex V it goes through two new edges By using one edge which we entered, another edge which we are going to leave. From the first vertex we "Exited" and we can "Enter" into that vertex at the last.

∴ Every vertex in a graph G is of Even degree



Conversely :

Suppose all vertices of G are of Even degree. Construct a Euler line Starting from vertex V

Since, Every vertex is of even degree we can enter into every vertex and exit from the Same vertex.

Tracing Continuous until reaching the vertex V.

∴ G contains an Euler line or G is Eulerian. Hence Proved

Theorem: A connected graph G has an Eulerian trail if and only if it has exactly two odd vertices.

Proof: This fact can be proved if we can prove that first, if the connected multi-graph has Euler path exactly two vertices have odd degree and second if the connected multigraph has exactly two vertices of odd degree then it has Euler path. Now, if the graph has an Euler path say from a to z but not Euler Circuit, then it must pass through every edge exactly once. In this scenario, the first edge in the path contributes one to the degree of vertex a, and at all other time when other edges pass through vertex a it contributes twice to the degree of a, hence we can say that degree of a is odd. Similarly, the last edge in the path coming to z contributes one to the degree of z, all the other edges contributes two, one for

entering and one for leaving. Here, also the degree of last vertex, z is odd. All the other vertices other than a and z must have even degree since the edges in those vertices enter and leave the vertex contributing two to the degree of every time the vertices are meet. Hence, if there is an Euler path but not an Euler circuit, exactly two vertices of the graph have odd degree. Secondly, if exactly two vertices of a graph have odd degree and let us consider they are a and z. Now, consider another graph that adds an edge {a,z} to the original graph, then the newly formed graph will have every vertices of even degree. So, there exists Euler circuit in new graph and the removal of new edge gives us the Euler path in the original graph. Hence it exactly two vertices of the graph have an odd degree then the graph has an Euler path but not Euler circuit.

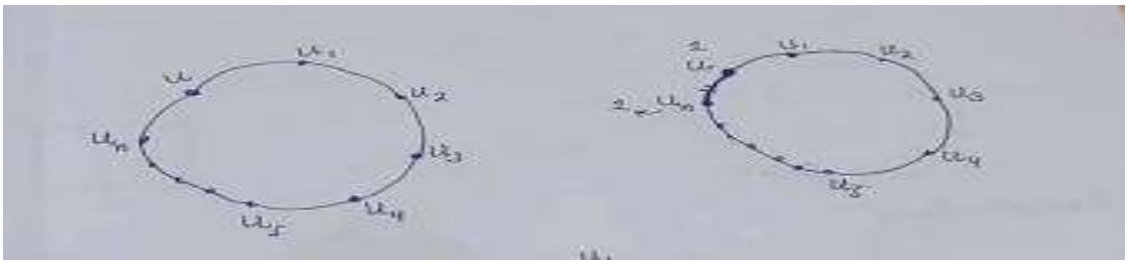
Proof:-

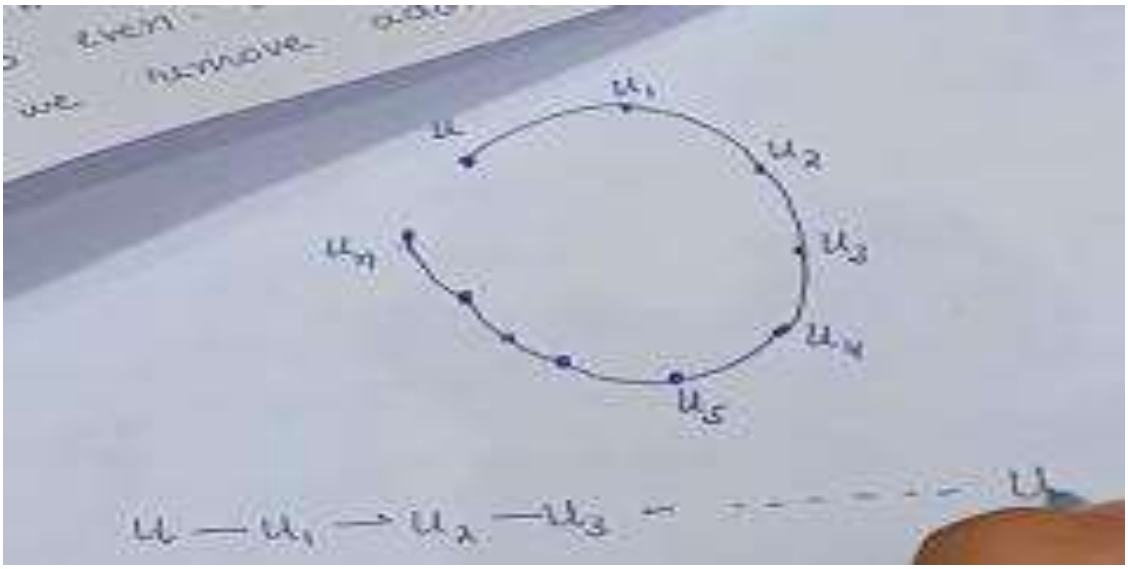
By theorem 6.2 i.e (A Connected graph G is eulerian if and only if the degree of each vertex of G is even)

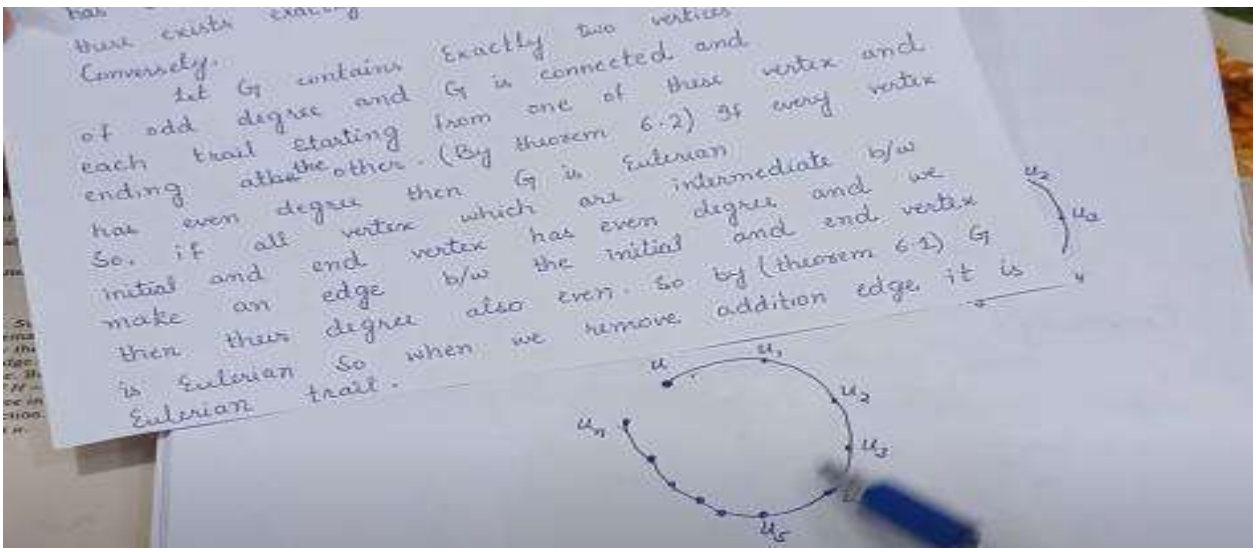
If $u-u$ cycle which is Eulerian then each vertex has even degree.

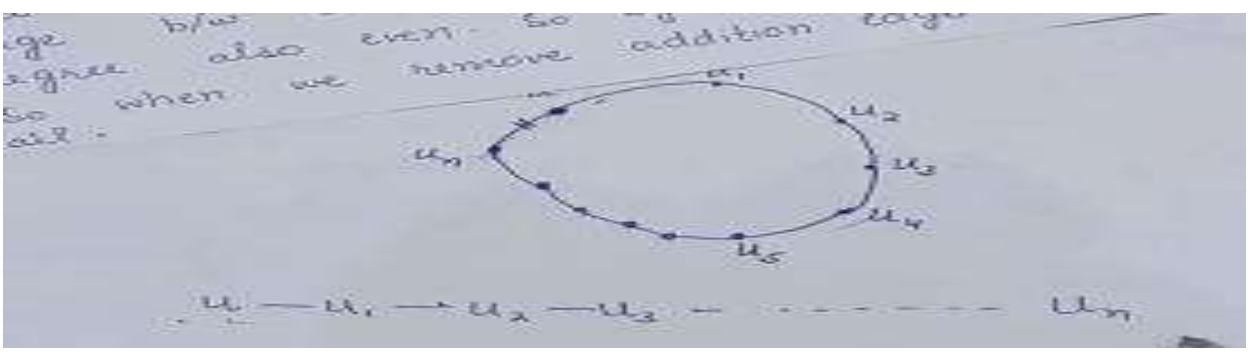
Let $u-u_1-u_2-u_3-u_4-u_5-\cdots u_n-u$ is Eulerian cycle then each vertex has Even degree. now if u_n-u edge remove then initial and end vertex i.e u and u_n has odd degree and all others has even degree. Also $u-u_n$ is Eulerian trail iff there exists exactly two vertices of odd degree.

Conversely.



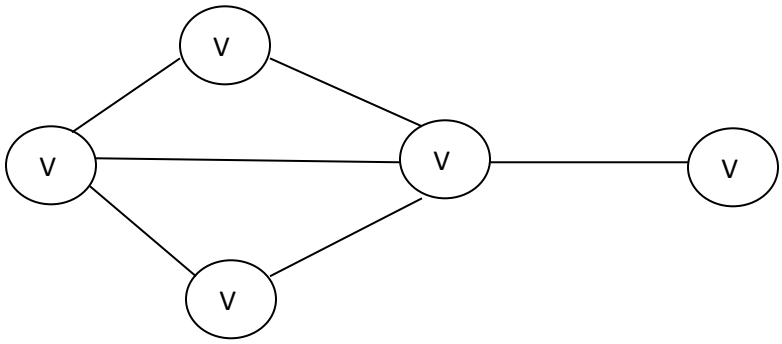






Example: Show that the graph in figure contains an Eulerian trail but no Eulerian circuit

Solution:



G is connected and $\deg(v_1) = 3$, $\deg(v_3) = 4$, $\deg(v_2) = \deg(v_5) = 2$ and $\deg(v_4) = 1$

Since the graph G has exactly two odd vertices, it contains an Eulerian trail but no Eulerian circuit.

Example: Show that the graphs in figure contain no Eulerian circuit.

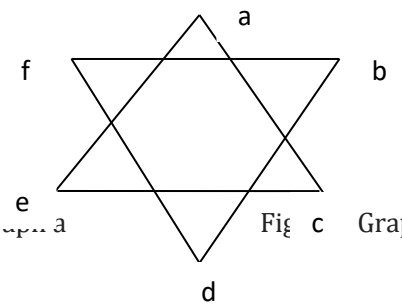


Figure : Graph a

Fig c Graph b

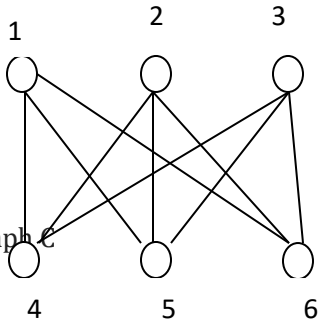


Figure: Graph c

Solution:

a) It is not connected, so it does not contain an Eulerian circuit
b) It is connected but each vertex has odd degree, so it does not contain an Eulerian circuit
c) It is connected but each degree is odd, so, it does not contain an Eulerian circuit

Hamiltonian Graphs

Hamiltonian graphs are named after Sir William Hamilton, an Irish mathematician who introduced the problem of finding a circuit in which all the vertices of a graph appear exactly once.

A cycle that contains every vertex of a graph G exactly once is called Hamiltonian cycle.

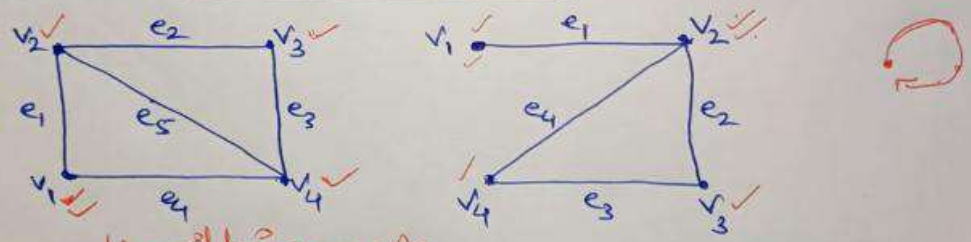
A Hamiltonian path is a simple path that contains all vertices of G.

Now, we summarize two basic concepts of this chapter in the following table

Term	Initial and terminal vertices the same	Must include every edge	Repeated vertices followed
Eulerian Circuit	Yes	Yes	Yes
Hamiltonian Cycle	Yes	No	No

A graph G is called a Hamiltonian graph if it contains a Hamiltonian circuit.

Eg:-



Hamiltonian graph

Hamiltonian Graph-

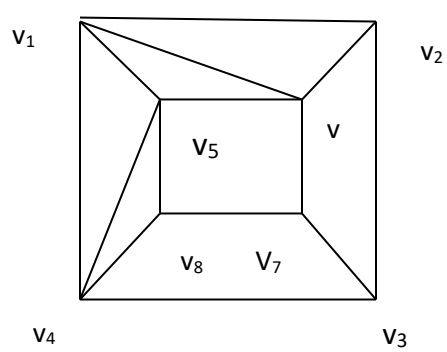
graph → Hamiltonian Path, Hamiltonian circuit

Hamiltonian path- A Hamiltonian path is a simple path that contains all vertices of G where the end points may be distinct.

Hamiltonian Circuit-

A circuit in a graph G that contains each vertex in G exactly once, except for the starting and ending vertex that appears twice is known as Hamiltonian circuit.

Example: Is the graph (cube) given in following figure is an Eulerian or a Hamiltonian cycle



Solution: Clearly, there is no Eulerian circuit since there are some odd vertices but the graph has Hamiltonian cycle ($v_5, v_1, v_2, v_6, v_7, v_3, v_4, v_8, v_5$).

Theorem (without proof)

A simple connected graph with $n>2$ vertices is Hamiltonian if degree of every vertex is at least $n/2$ (This is known as Dirac's Theorem)

Theorem (Without proof)

A connected graph with n vertices is Hamiltonian if for any two non-adjacent vertices u and v, $\deg(u)+\deg(v) \geq n$

(This theorem is known as Ore's Theorem)

It may noted that the theorems stated above provide only sufficient criterion. There are no known simple necessary and sufficient criteria for the existence of Hamiltonian cycle nor is there even an efficient algorithm. For finding such a cycle many theorems exist that establish either necessary or sufficient conditions for a connected graph to have Hamiltonian cycle or path

A few helpful hints for trying to find a Hamiltonian cycle in a graph are given below.

1. If G has a Hamiltonian cycle, then for all $u \in V$. $\deg(u) \geq 2$
2. If $v \in V$ and $\deg(v)=2$ then two edges incident with vertex v must appear in every Hamiltonian circuit for G
3. If $v \in V$ and $\deg(v)>2$ then we try to build a Hamiltonian cycle, once we pass through vertex v, any unused edges incident with v are deleted from further consideration.
4. In building a Hamiltonian circuit for G, we cannot obtain a circuit for a subgraph of G unless it contains all the vertices of G.

Example : Determine, in graph of figure, if there is an Eulerian circuit and or a Hamiltonian cycle.

Example: Give an example of a graph with six vertices which is Hamiltonian but not Eulerian and vice-versa

Weighted Graph or Labelled Graph

A weighted graph is a graph G, in which each edge e is assigned a non-negative real number called weight of e. The weight of a subgraph H of G is the sum of the weights of the edges of the subgraph H

Now we discuss two famous network problems –the shortest path problem and the Chinese postman problem.

The shortest Path Algorithm

Consider a weighted graph G. The length of a path in a weighted graph is the sum of the weights of the edges of this path and the shortest path between the two vertices is the minimum length of the path. There are several different algorithms to find the shortest path between two vertices in a weighted graph. We discuss here one discovered by E.W Dijkstra.

Dijkstra's Algorithm

Step 1: Label the initial vertex of the graph with weight zero

Step 2: Calculate the weights of all vertices adjacent to the initial vertex corresponding to the weights of the edges incident on the initial vertex

Step 3: Label these vertices with smallest possible value of their weights

Step 4: Calculate the weights of all those vertices which are adjacent to the vertices with minimum weights determined in step 3

Step 5: Label these vertices with minimum weight

Step 6: Continue this process until all the vertices of the weighted graph are labeled

Step 7: Trace the path of cumulative minimum weight from the initial vertex to desired vertex

<u>Pseudo code for Dijkstra's Algorithm</u>

DijkstraSP (G: weighted connected simple graph with all weights positive)

{ G has vertices $a = v_0, v_1, v_2, \ldots v_n = z$ and weights $w(v_i, v_j)$ where $w(v_i, v_j) = \infty$ if (v_i, v_j) is not an edge in G

for i=1 to n

$L(v_i) = \infty$

$L(a) = 0$

while $z \notin S$

begin

U = a vertex not in S with L(u) minimum

S = S U {u}

for all vertices v not in S

if L(u) +w(u,v)<L(v) then L(v) = L(u)+w(u,v)

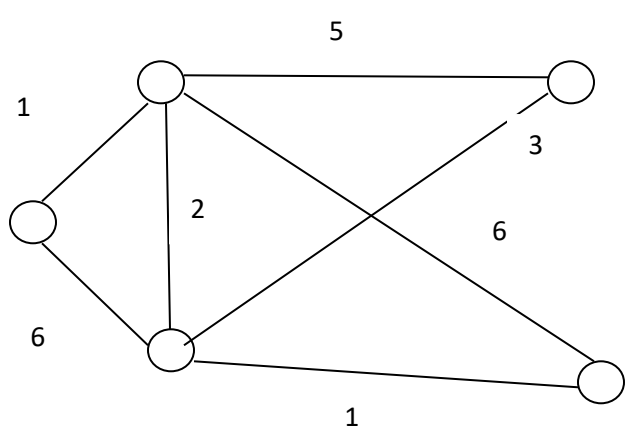
{This adds a vertex to S with minimum label and updates the labels of vertices not in S }

end

{ L(z) = length of a shorted path from a to z }

}

Example: Apply Dijkstra's algorithm to find the shortest path from the vertex a to each of the other vertices of the following weighted graph in figure.



Solution

Since we have to find the shorted path from the vertex a to each of the other vertices then we assign weight 0 to a vertex a and ∞ to all remaining vertices.

Vertex	A	B	c	d	E
Label	0	∞	∞	∞	∞

Now, the vertices adjacent to a are b and c, we calculate weights of b and c and label then with minimum weight.

wt (b) = wt(a)+ wt(a,b) = 0+6

wt (c) = wt(a) + wt(a, c) = 0+1=1

Now, the updated table is

Vertex	**a**	B	C	d	E
Label	0	6	1	∞	∞

Since c has minimum weight so, we select it

Now the vertices adjacent to c are b, e and d

wt(b) = wt(c)+wt(c, b) = 1+2 = 3

wt(e) = wt(c) +wt(c , e) = 1+5 = 6

wt(d) = wt (c)+ wt(c, d) = 1+6 = 7

Now, the updated table is:

Vertex	**A**	**B**	**c**	d	E
Label	0	3	1	7	6

Since b has the smallest weight we select it and replace previous weight by new weight.

Now, vertices adjacent to b are e and d

So,

wt(e) = wt (b) +wt (b, e) = 3+3 = 6

Wt (d) = wt (b) + wt (b, d) = 3+1 = 4

Now, the updated table is:

Vertex	**A**	**B**	**c**	**d**	e
Label	0	3	1	4	6

Since'd' has minimum weight, so it is selected, and its adjacent vertices are b and c

We update weight of b and c if new weight is less than previous weight.

But not change

Hence final updated table is:

Vertex	**A**	**B**	**c**	**d**	e
Label	0	3	1	4	6

Travelling Sales Man Problem (TSP)

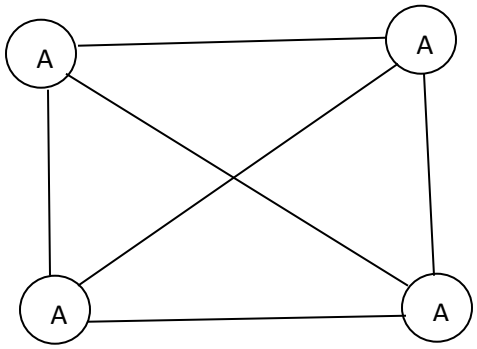
This problem was first formulated in 1930 and is one of the most intensively studied problems in graph theory and optimization. In general-TSP problem can be formulated as follows:

Given a list of cities

Given a list of cities (represented by vertex of graph) and distance between each pair of cities, the problem is to find out the shortest possible route or path that usual each city exactly once and returns to the origin or staring point (city)

The name "travelling sales man problem " is given as it is analogous to the problem like sales man want to visit each of n city exactly one to sale his business and finally returns to his house at the end of journey

Example



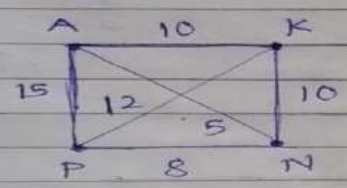
Let a travelling salesman want to visit places A, B, C, D as shown in figure below. Now the problem is : in which order should be visit these places to travel minimum total distance

Travelling Salesman Problem -

This problem is related to Hamiltonian circuit. A salesman is required to visit a number of cities during a trip. Given the distances between the cities, in what order should be travel so as to visit every city exactly once & return home with minimum distance travel

Note - Total number of Hamiltonian circuits in a complete graph on n vertices $= \frac{(n-1)!}{2}$

For the following graph

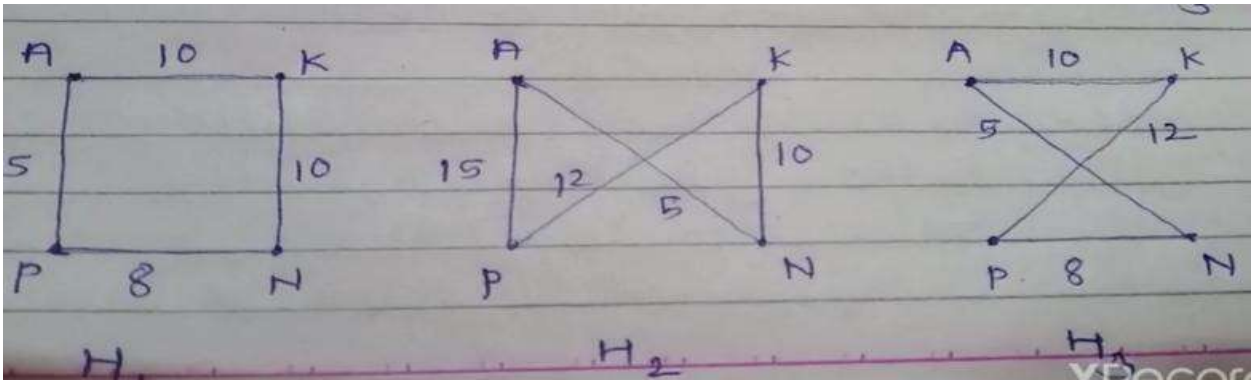


soln-

All Hamiltonian circuits in graph are $\frac{(n-1)!}{2}$

$= \frac{(4-1)!}{2}$

$= 3$



H_1 H_2 H_3

Total weight of the Hamiltonian circuit H_1

$= 10 + 10 + 15 + 8$

$= 43$

Total weight of the Hamitonian circuit H_2

$= 15 + 10 + 12 + 5$

$= 42$

Total weight of the Hamiltonian circuit H_3

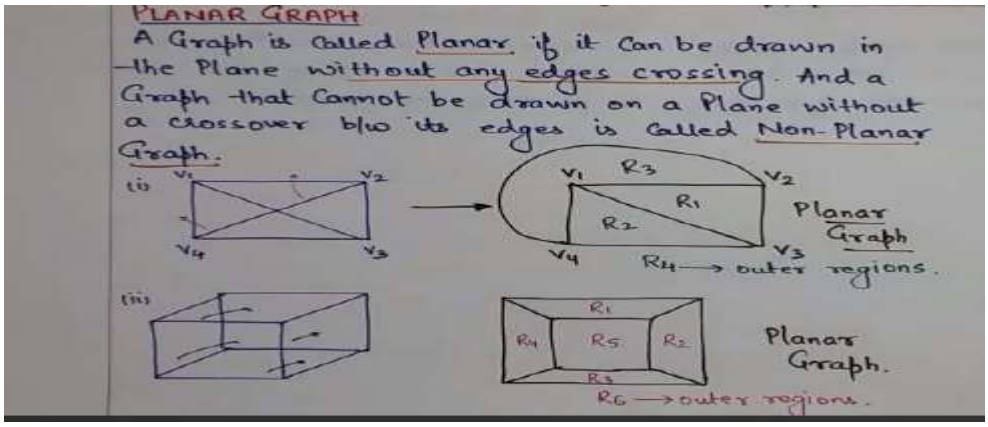
$= 5 + 8 + 12 + 10$

$= 35$

Hence the smallest weight among these Hamiltonian circuit is 35.

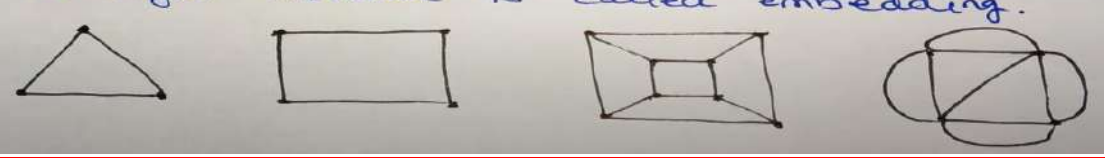
Planner Graph

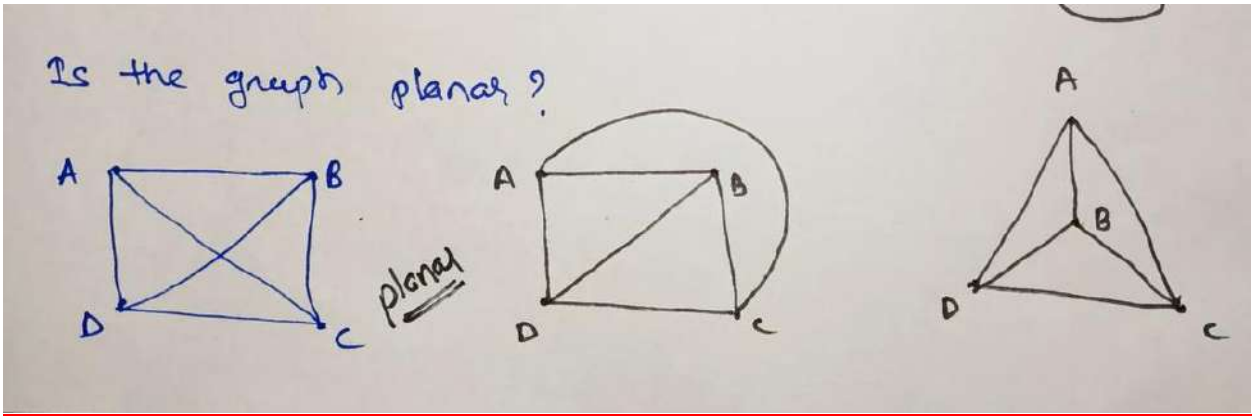
A graph G = (V, E) is said to be planner graph if it can be drawn in plane such that no-intersection of edges exist at a point other than their common end point i.e a graph is planner if it is drawn without crossing edge

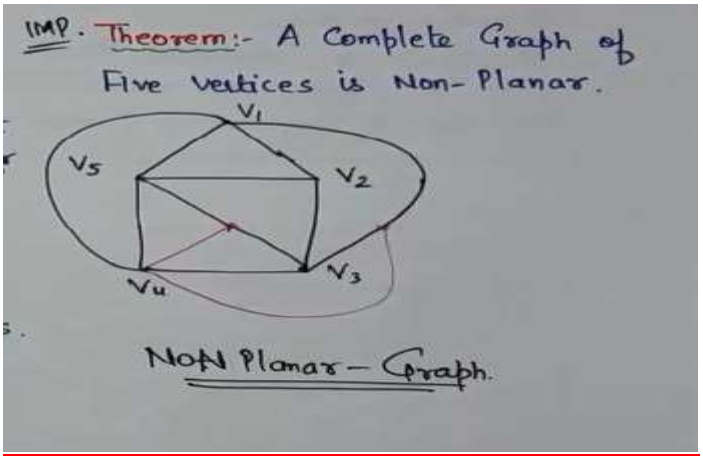


that no edges intersect is called embedding.

Eg:-

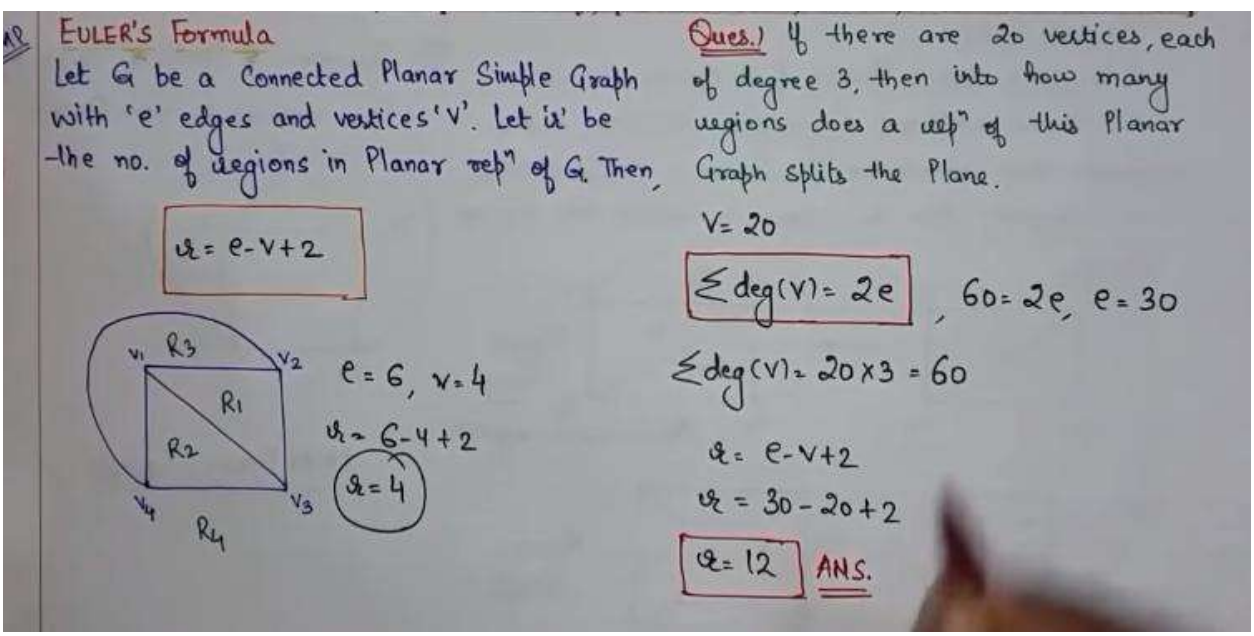






Euler's Formula

Theorem: Let G = (V,E) be a connected planer simple graph with V vertices, e edges and r be the number of regions in planar representation then r = e-V+2



Example 1: Suppose that a connected planer simple graph has 20 vertices, each of degree 4. Into how many regions does a representation of this planner graph splits the plane?

Solution: The graph has 20 vertices, each of degree 4

Therefore V = 20

Sum of degrees of vertices = 4*20 = 80

Since, sum of degree of vertices is equal to twice the number of edges

Therefore 80 = 2*e

Or e = 40

Now, using Euler's formula

r = e-V+2

= 40-20+2

= 22

Therefore, number of regions is 22

Example 2: Suppose that a connected planner graph has 30 edges. If a planner representation of this graph divides the plane into 20 regions, how many vertices does this graph have?

Solution:

Here, the number of edge in the graph e = 30

The number of region in the planner graph = 20

Now, we know that the euler's theorem states that

r = e-v+2

= 30-20+2

=12

Therefore, total number of vertices (v) =12

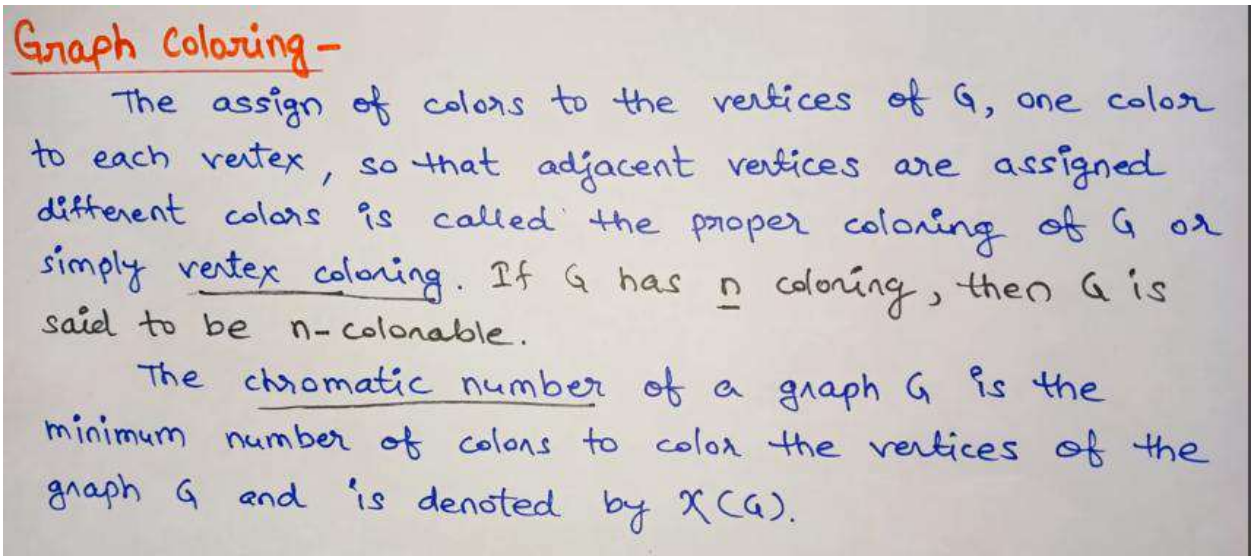
Example 3: Suppose that a connected graph has 30 edges. If a planar representation of this graph divides the plane into 20 regions, how many vertices does this graph have?

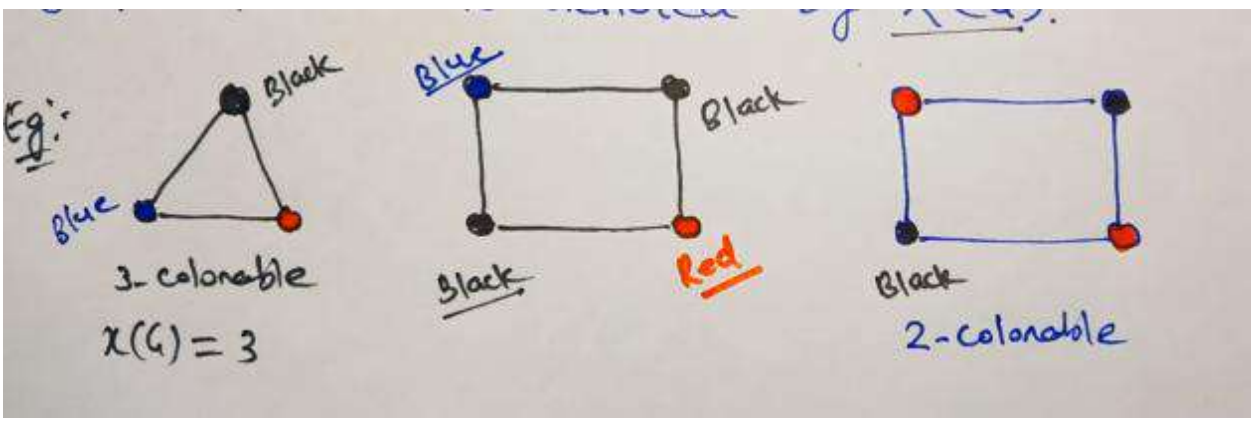
Solution: We have, r = 20, e = 30, so by Euler's formula, we have v = e-r+2 = 30-20+2=12.

So, the number of vertices is 12

Graph Coloring

The map of the world consists of many regions. To design one region with its adjacent another region, different colors are used. One way to ensure that two adjacent regions never have the same color is to use a different on maps with many regions it would be hard to distinguish similar colors. Instead, a smaller number of color should be used whenever possible. For example, we use some method to determine least no of color for coloring map.





Each map in the plane can be represented by a graph. Each region of the map is represented by a vertex. Edge connects two vertices if the region represented by the vertices has a common order. The graph made in this way is known as dual graph.

A coloring of a simple graph is the assignment of a color to each vertex of the graph so that no two adjacent vertices are assigned the same color

A graph can be colored by assigning different colors to each vertex but it is inefficient for graph with large number of vertices. We can color graph with less number of color than number of vertices

The least number of colors needed for coloring a graph is known as chromatic number

Example

Theorem (Four Color Theorem): The chromatic number of planner graph is no greater than four

Example: What is the chromatic number of given graph?

Solution:

Example: What is the chromatic number $k_{m,n}$?

Solution: In complete bipartite graph $k_{m,n}$ every vertices of first position set are adjacent to each 'n' vertices of another partition set. Each vertices of first position set can be colored with single color as the vertices in the same position are upon adjacent to each other. Similarly, all vertices of second position can be colored with another single color. Hence, chromatic number of km,n is equals to 2

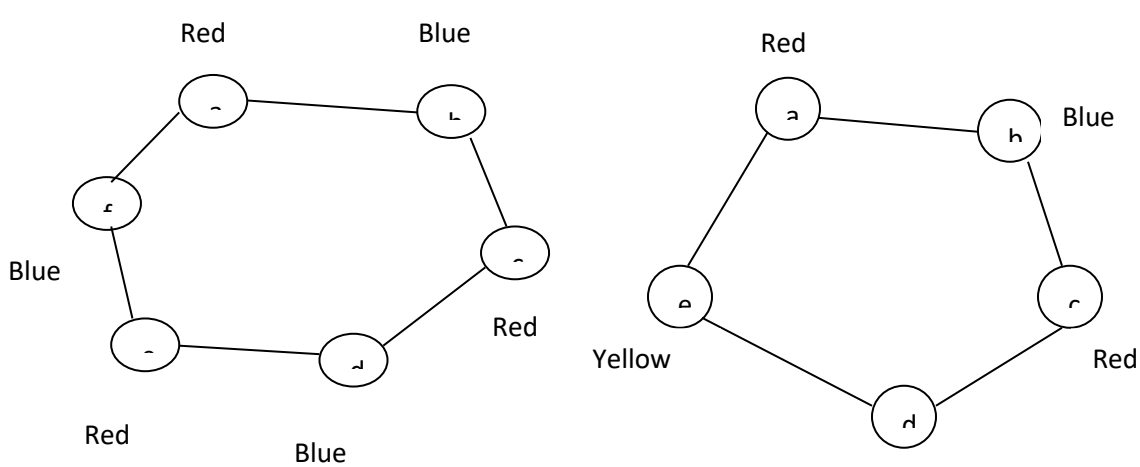
That is $|k_{m,n}| = 2$

Example: What is the chromatic number of k_n?

Solution: In complete graph k_n, there exist an edge between each possible pair of vertices, so no two vertices can be assigned the same color. Hence, chromatic number $k_n = n$ (each vertex has different color).

Example: What is the chromatic number of graph C_n where $n \geq 3$?

Solution: We will first consider some individual cases. To begin, let $n = 6$. Pick a vertex and color it red. Proceed clockwise in the planer depiction of C_6 as shown in following figure. It is necessary to assign a second color say, blue, to the next reached. Continue in the clockwise direction, the third vertex can be colored red. The fourth vertex blue, and fifth vertex red. Finally the sixth vertex which is adjacent to the first and can be colored reed. Hence the chromatic number of C_6 is 2



Blue

Figure: Graph C_7 **Figure Graph C_6**

Next, let n = 5 And consider C_5, Pick a vertex and color it red. Continuing in the clockwise, it is necessary to assign a second color say blue, to the next vertex reached. Continuing in the clockwise direction, the third vertex can be colored red. And fourth vertex can be colored blue. The fifth vertex cannot be colored either red or blue because it is adjacent to the fourth and the first vertex. Consequently, a third color is required for this vertex. Note that we would have also needed three colors if we had colored vertices in the counter clockwise direction. Thus chromatic number of C_5 is 3

Application of graph Coloring

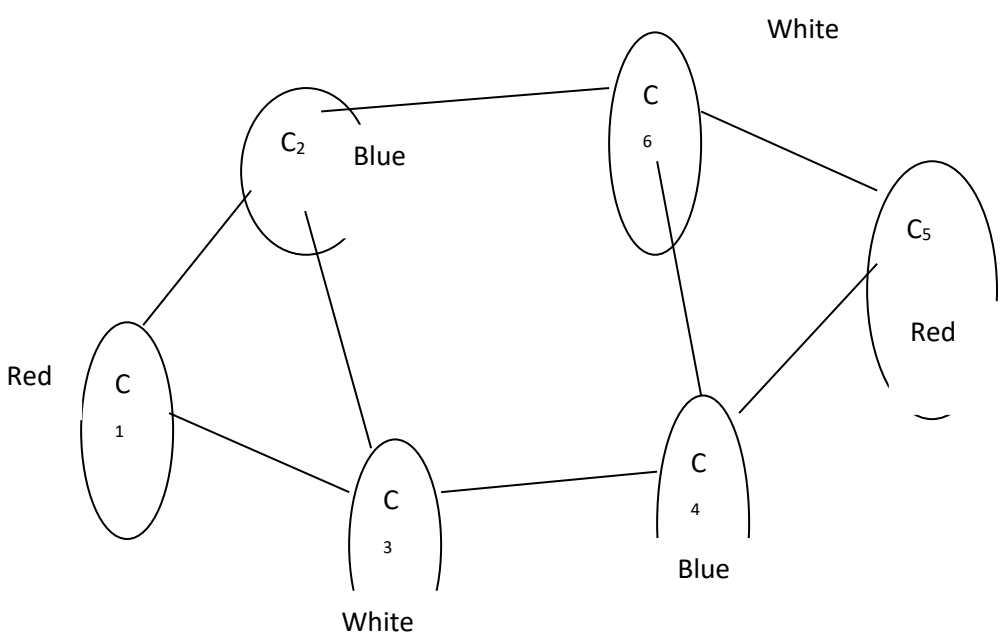
The problem involving scheduling, assignments of resources, can be solved by using concept of graph coloring

Scheduling Final Exam

Problem: How can the final exams at a university be scheduled so that no students has two exams at the same time?

Solution: The problem can be solved using a graph model in which courses represents vertices and there is an edge between two vertices if a common students in the course represented by these vertices. Each time slot for final exam is represented by different colors.

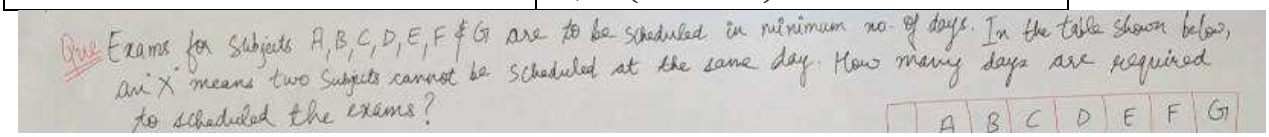
For example, let there are six final courses to be scheduled and course are numbered from c_1 to c_6. Suppose that the course number C_1 & C_2, C_1 and C_3, C_2 and C_3, C_3 and C_4, C_4 and C_5, C_4 and C_6, C_5 and C_6, C_2 and C_6 has common students for exam. Then the graph becomes



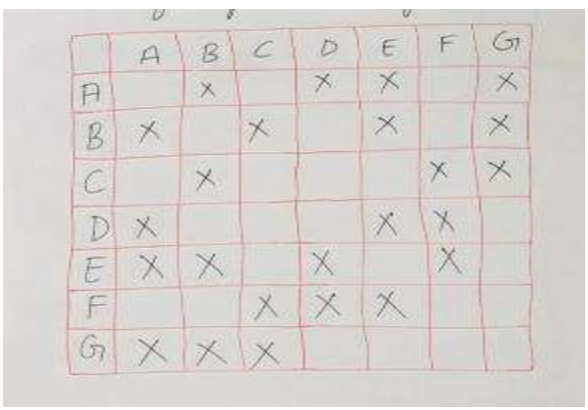
Now, scheduling of exam is determined by coloring this graph.

Therefore, the exam schedule is determined with same color as time period with respective course

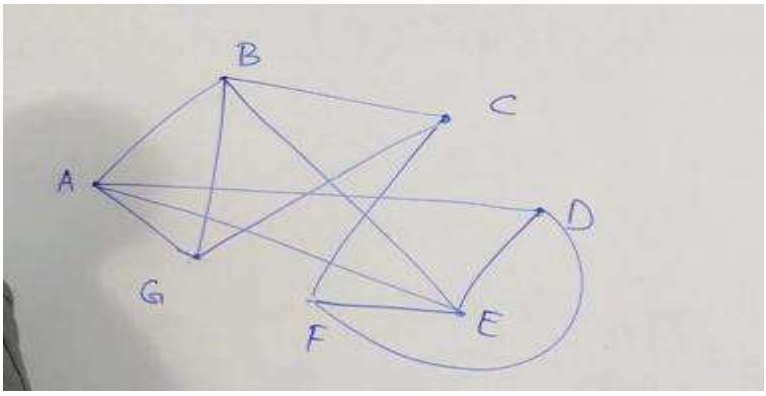
Time Period	Course
I	C_1, C_5 (Red Color)
II	C_3, C_6 (White Color)
III	C_2, C_4 (Blue Color)

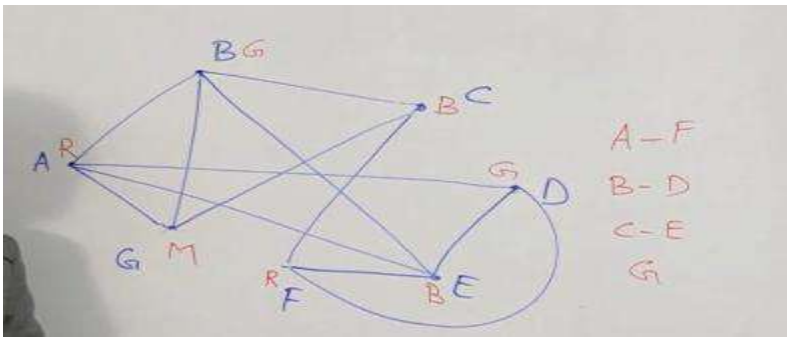

Que Exams for Subjects A, B, C, D, E, F & G are to be scheduled in minimum no. of days. In the table shown below, an X means two Subjects cannot be scheduled at the same day. How many days are required to scheduled the exams?

	A	B	C	D	E	F	G



	A	B	C	D	E	F	G
A		X		X	X		X
B	X		X		X		X
C		X				X	X
D	X				X	X	
E	X	X		X		X	
F			X	X	X		
G	X	X	X				





Directed Graph

The theory of graphs studied so far is concerned with undirected graphs. Most of the concepts and terminology of undirected graphs are similar to directed graphs. Such graphs are frequently more useful in previous dynamic systems such as digital computers or flow systems.

This section gives the basic definitions and properties of directed graphs. Many of the definitions will be similar to those in the preceding sections on undirected graphs.

A directed graph or digraph D = {V,E} consists of two things:

i. A set V = V(D) whose elements are called vertices, points or nodes of D
ii. A set E = E(D) of ordered pairs (u,v) of vertices called arcs or directed edges or simply edges.

A picture of directed graph D is the representation of D in the plane. That is, each vertex v of D is re

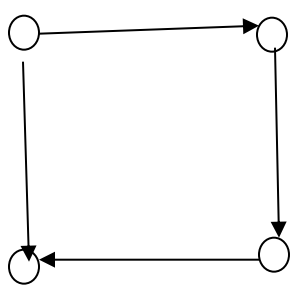
Presented by a dot or small circle and each directed edge e = (u,v) is represented by an arrow or directed curve from the initial vertex u to terminal vertex v.

If edges and or vertices of a directed graph D are labeled with some type of data then D is called a labeled directed graph.

A directed graph D = (V,E) is said to be finite if its set V of vertices and its set E of edges are finite.

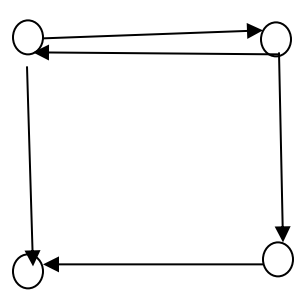
Simple Digraphs

A directed graph D = (V,E) which has neither loops nor multiple edges is called simple directed graph.



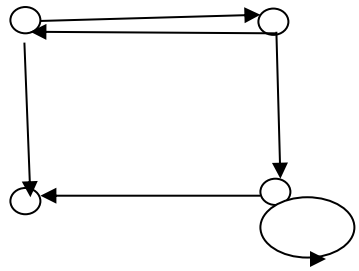
Multiple Diagraphs: In any diagraph, two or more edges are said to be parallel if they have same initial and terminal vertex with same orientation.

A diagraph D = (V, E) which contains some multiple edges is called a multiple directed graph



Pseudograph: In any diagraph, an arc or edge with same initial and terminal vertex is called self loop. A diagraph D = (V,E) which contains self loop is called a pseudo diagraph.

Subgraph: Let H be a directed graph with vertex set V(H) and edge set E(H). Similarly, let G be directed graph with vertex set V(G) and edge set E(G). Then H is said to be subgraph of G, written as H ⊆G, if V(H) ⊆V(G) and E(H) ⊆E(G).

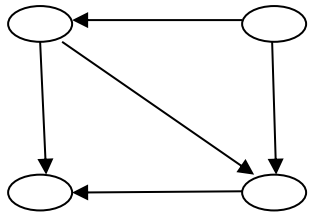


Subgraph: Let **H** be a directed graph with vertex set **V(H)** and edge set **E(H)**. Similarly let G be a directed graph with vertex set **V(G)** and edge set **E(G)**. Then **H** is said to be subgraph of **G**, written as **H ⊆G** if **V(G) ⊆V(G)** and **E(H) ⊆ E(G)**.

Adjacent Vertices: The vertices u and v are said to be adjacent vertices if there is a directed edge e = (u,v) or e = (v, u). Note that the edges (u,v) and (v, u) are different in a directed graph.

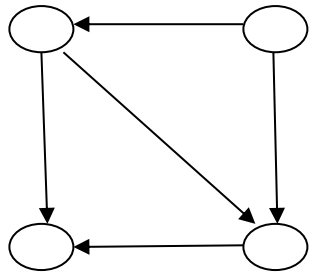
The edge e said to be incident on each of its end points u and v.

In degree and out degree of vertex: Suppose that D = (V,E) is a directed graph. The in-degree of v, written as indeg(v), is the number of edges incident towards v and the outdegree of a vertex v of D, written as outdeg(v), is the number of edges directed away from v.



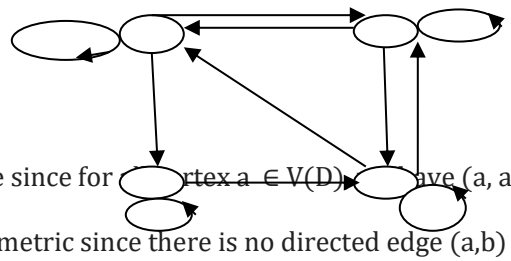
Indegree of vertex u is **1** and its outdegree is **2**

Source and Sink vertex: In a diagraph D = (V,E), a vertex v with zero indegree is called a source vertex and a vertex v with zero outdegree is called a sink vertex. For example, we consider a directed graph D of four vertices u, v, w and x in figure below.



In figure, the vertex v has indegree 0, so it is a source and there are no vertices with outdegree zero, so there are no sink. The indegree sequence of the directed graph D in figure is (2, 2 , 1, 0) and outdegree sequence of D is (2, 1, 1, 1)

Example: Determine whether the directed graph **D** in figure is a reflexive, symmetric and transitive diagraph



Solution:

Digraph D is reflexive since for ~~vertex a ∈ V(D)~~ ave (a, a) ∈ E(D)

Digraph D is not symmetric since there is no directed edge (a,b) for all directed edge (b, a) in D

Digraph D is not transitive since for (a, b) and (b,c) in D there is no (a, c) in D

Paths: Let D be a directed graph. The concepts of paths, simple paths, trials and cycles carry over from non-directed graphs G except that the directions of edges must agree with the direction of the path.

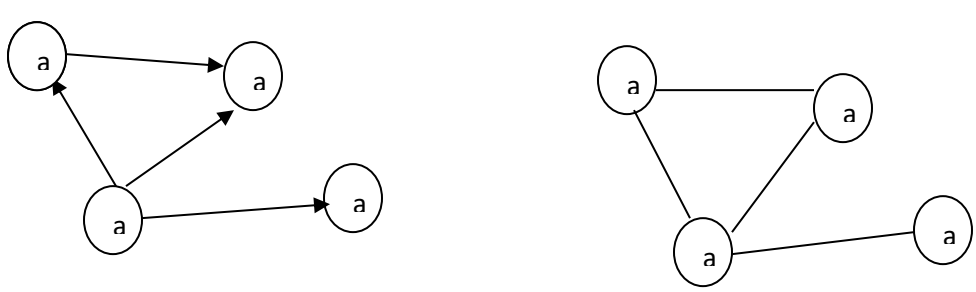
Specifically:

i. A directed path in D is an alternative sequence of vertices and directed edges, say, P = (v_0,v_1 ,v_2……v_n) such that each edge ei begins at vi-1 and ends at vi

ii. The length of the path P in n, its number of edges
iii. A simple path is a path with distinct vertices
iv. A trial path is a path with directed edges
v. A closed path has the same first and last vertices
vi. A spanning path contains all the vertices of D
vii. A cycle is a closed path with distinct vertices except first and last
viii. A circuit is a closed path containing at least three edges

Underlying Graph

Given a digraph D, we can obtain a graph G from D by removing all the arrows or directories from the edges. If the graph G has the same vertex set as that of the digraph D and corresponding to each directed edge in D associated with the ordered pair of vertices (a,b), there is an edge in G associated with the pair (a,b) then G is called the underlying graph D. For example, figure (b) is the underlying graph of directed graph D in figure (a).

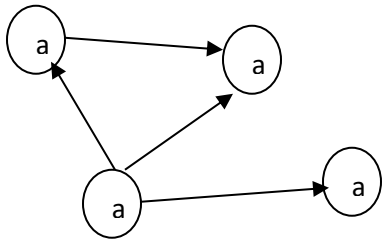


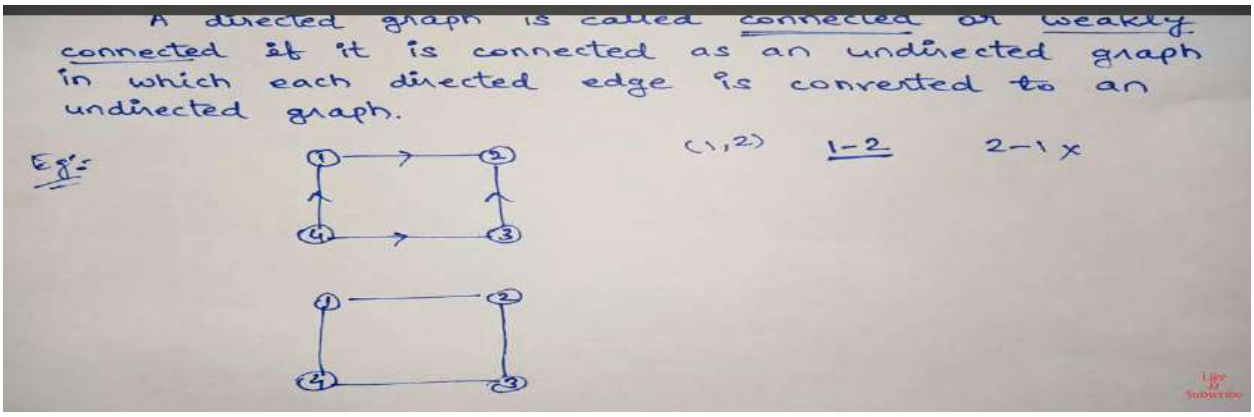
(b)

(a)

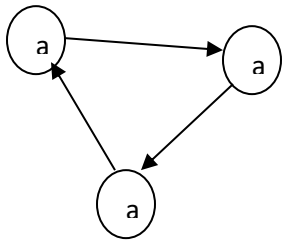
Weakly Connected Graph

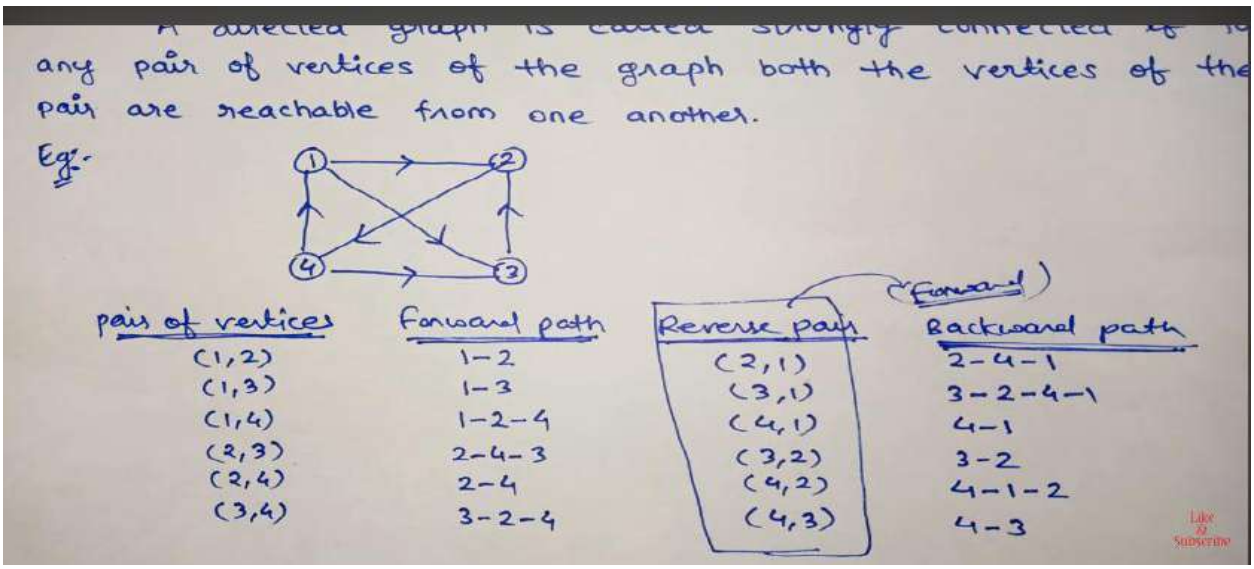
A digraph D is said to be weakly connected if its underlying graph is connected. For example, the digraph D in figure is weakly connected but the digraph D in figure is not weakly connected.





Strongly Connected Graph : A directed graph or diagraph D is said to be strongly connected graph if for any pair of vertices vi and vj in D, there is a direct path from vi to vj or from vj to vj (not necessarily both) for any pair of vertices vi and vj then D is said to be unilaterally connected.





Theorem: The sum of the out degrees of vertices, the sum of indegrees of vertices and number of edges in a directed graph are equal to each other.

Proof: When the indegrees of the vertices are summed, each edge is counted exactly once since every edge goes to exactly one vertex. Thus

$$\sum_{i=1}^{n} in - \deg(vi) = q$$

Similarly, when the outdegrees are summed, each edge is counted exactly once every edge goes out of exactly one vertex.

Then $\sum_{i=1}^{n} out - \deg(vi) = q$

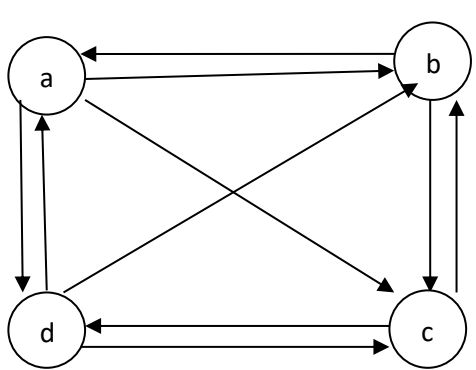
Thus

$\sum_{i=1}^{n} in - \deg(vi) = q = \sum_{i=1}^{n} in - \deg(vi)$

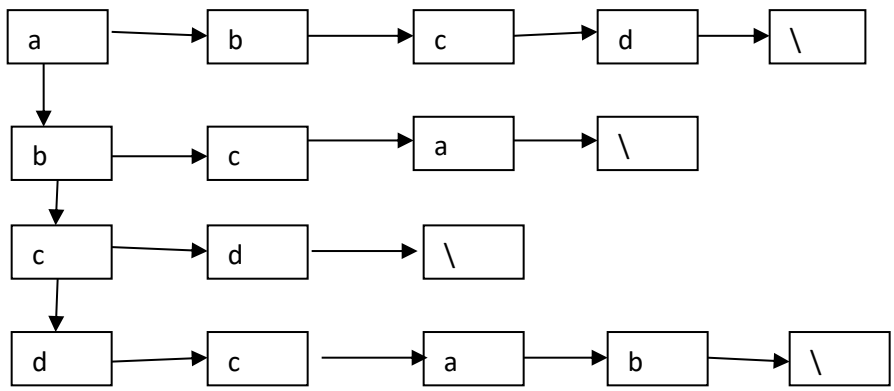
Representation of Diagraphs: There are number ways of representing graphs. Some popular ways are:

i. Adjacency Matrix
ii. Incidence Matrix
iii. Adjacency List (Linked List)

Example: Find the adjacency matrix representation, incidence and adjacency List representation of following graph



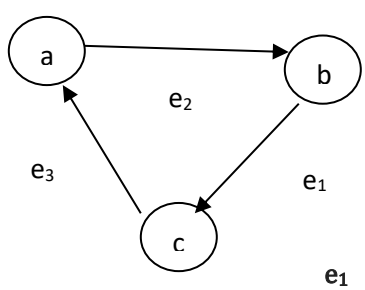
a) Adjacency List Representation



b) Adjacency Matrix Representation

$$\begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

c) Incidence Matrix Representation



$$\begin{array}{c} \\ a \\ b \\ c \end{array} \begin{array}{c} \begin{array}{ccc} e_1 & e_2 & e_3 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \end{array}$$

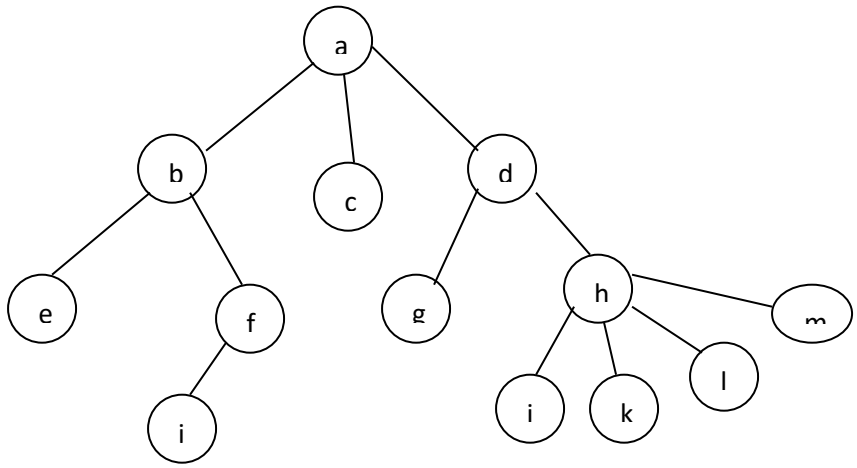
Tree

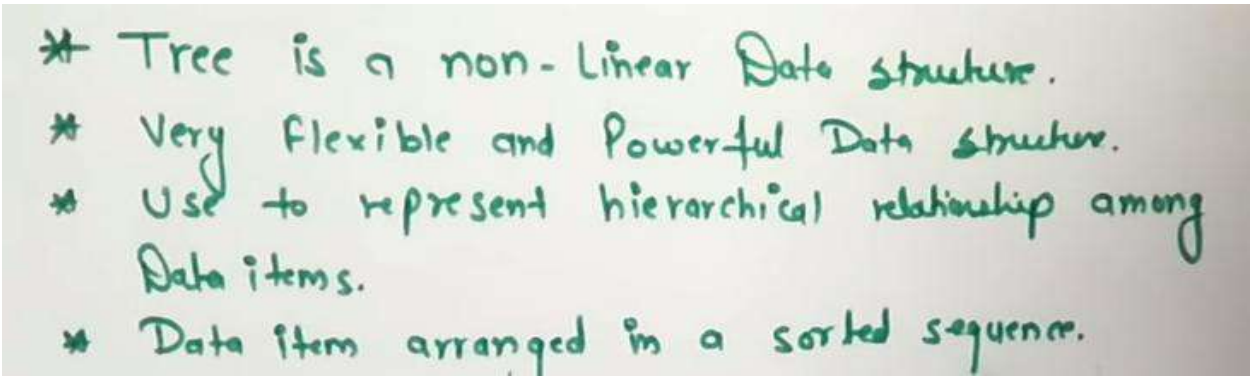
A tree is a collection of nodes connected by directed (or undirected) edges. A tree is non-linear data structure, compared to arrays, linked lists, stacks and queues which are linear data structures. A tree can

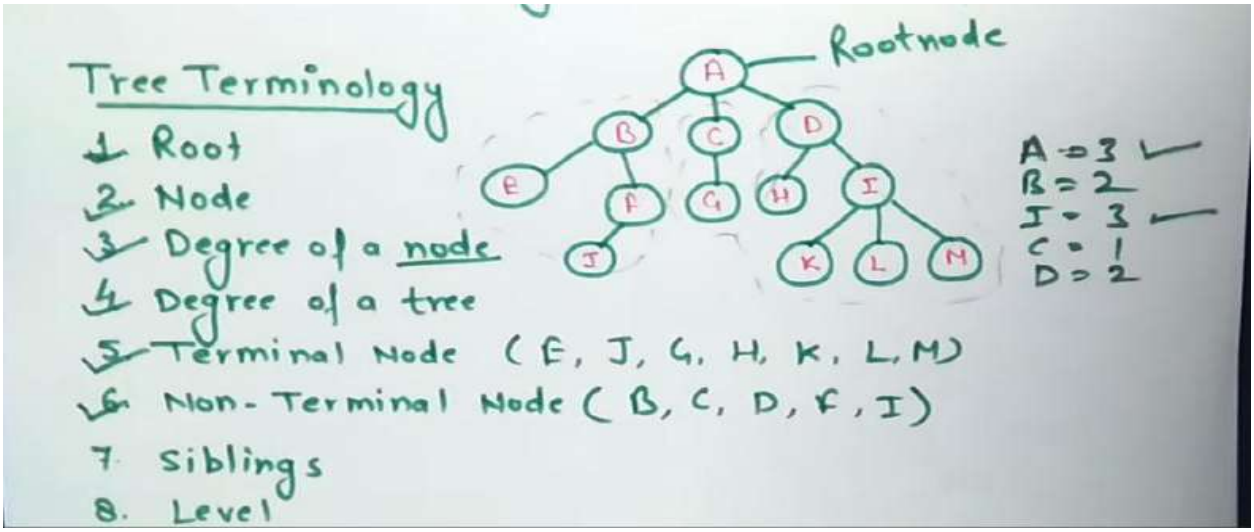
be empty with no nodes or a tree is a structure consisting of one node called the root and zero or one or more subtrees. A tree has following general properties:

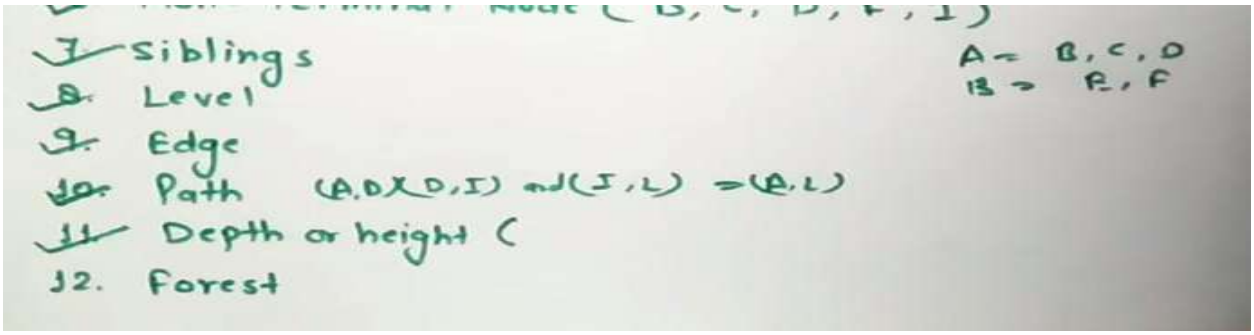
- One node is distinguished as a root
- Every node (exclude a root) is connected by a directed edge from exactly one other node. A direction is : parent-child.

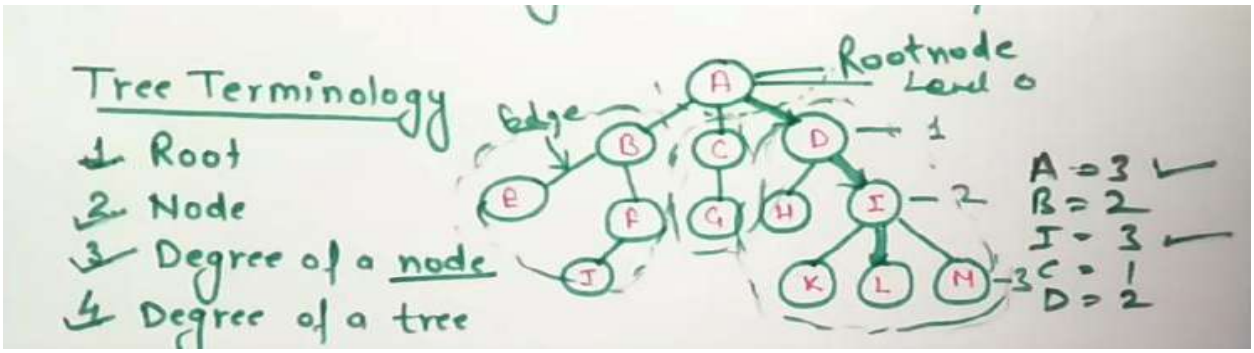
A tree occurs in situations where many elements are to be organized into some sort of hierarchy. In computer science, tree are useful in organizing and storing data in a database, used to construct efficient algorithms for locating items in a list.











Tree terminologies

- **Node:** Each data item in a tree is called a node. It is the basic structure in a tree. It specifies the data and links (branches) to other data items. There are 11 nodes in the above tree
- **Root:** It is the first data item (node) lies at the top in hierarchical arrangement of data items. In the above tree, 'A' is the root
- **Degree of a node:** It is the number of sub trees of a node n in a given tree. For example, the degree of node **a** is 3

- **Degree of a tree:** It is the maximum degree of nodes in a given tree. In the above tree, the degree of node **a** is 3 and another node **h** is also have degree 4. In the whole tree, this value is maximum. So, the degree of above tree is 4
- **Terminal node:** A node without degree zero is called a terminal node or lead node. In the above tree, node j, k, and l are terminal node
- **Non-terminal node :** A node (except the root node) whose out degree is not zero is called non-terminal node. Non terminal nodes are the intermediate nodes in traversing the given tree from its root node to the terminal nodes (leaves).
- **Siblings:** The children nodes of a given parent node are called siblings. They are also called brothers. In the above tree, e and f are brothers
- **Level/depth:** The number of edges that need to be followed while traversing a tree from toot to that node is always at level 0. Then its immediate children are at level 1 and their immediate children are at level 2 and so on up to the terminal node.
- **Edge:** It is the connecting line of two nodes or relation between two nodes of tree.
- **Path:** Path in a tree is the sequence of consecutive edges from the source to the destination node of tree. In the above tree, the path between a and j is given by the node pairs (a,d),(d,h),(h,j)
- **Forest:** It is a set of disjoint trees. In a given tree, if we remove its root, it becomes a forest.

Applications Codes

A prefix code is a special type of coding system in each code satisfies a "prefix property". Prefix property means there is no code were in the system that is prefix of any other code word in the system.

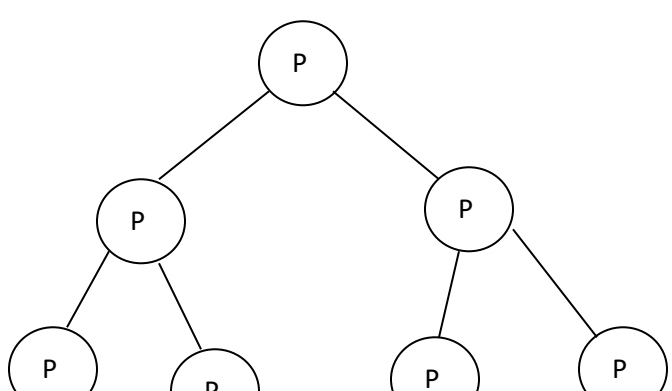
A prefix code can be represented using binary tree where the symbols are leaf node and edges are labeled with code words.

Tree as models in computer System

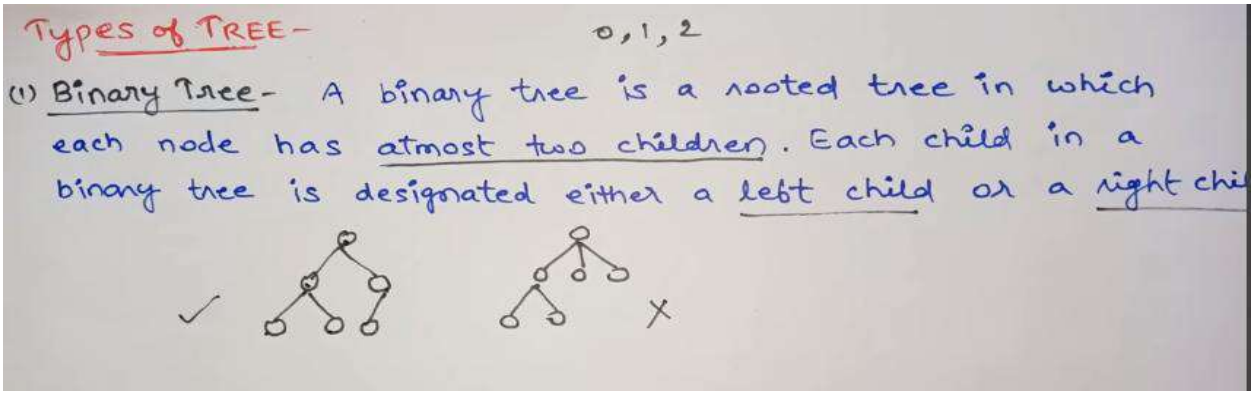
Files in computer memory can be organized into directories or folders in which folder contains both files and sub-folders. In file system, root directory or folders contain file in the system. So a file system can be represented by a rooted tree, where the root represents main folder, internal vertices represents sub-folders and leaf nodes represents ordinary files or empty folders.

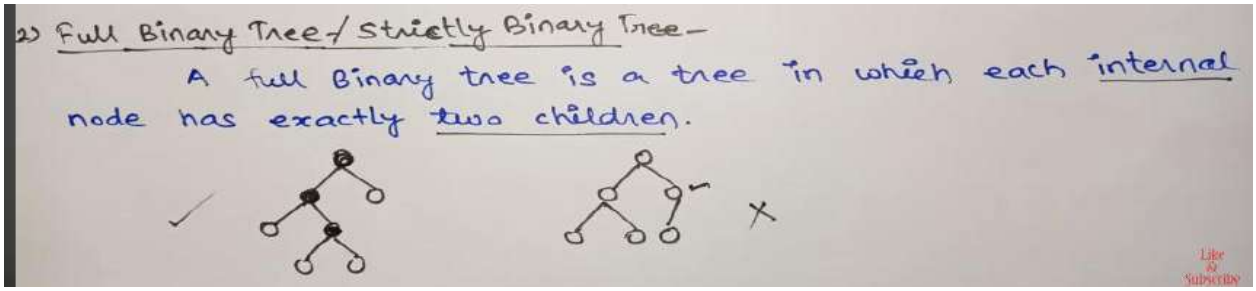
Tree as a model for parallel Processing

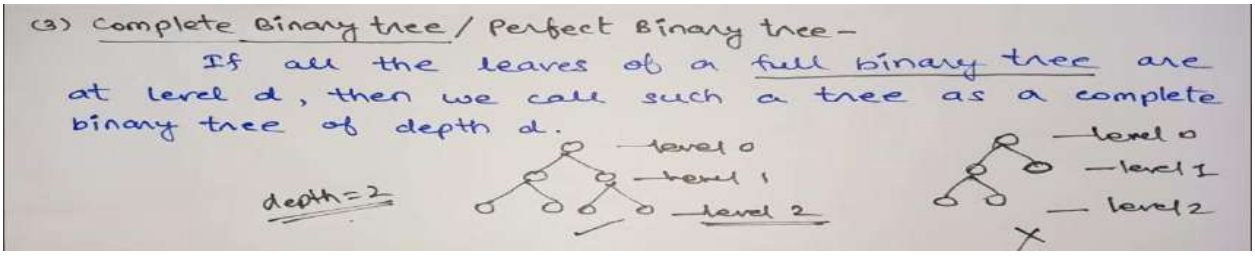
A parallel processing system consists of many processing devices (CPU). These processors are interconnected with each other to complete a task in parallel computing fashion. Tree can be used to represent the connection between these processors. Here, a processor is represented by vertex and the edges between these vertices are used to represent connection between them. A tree connected network for seven processor as shown below.

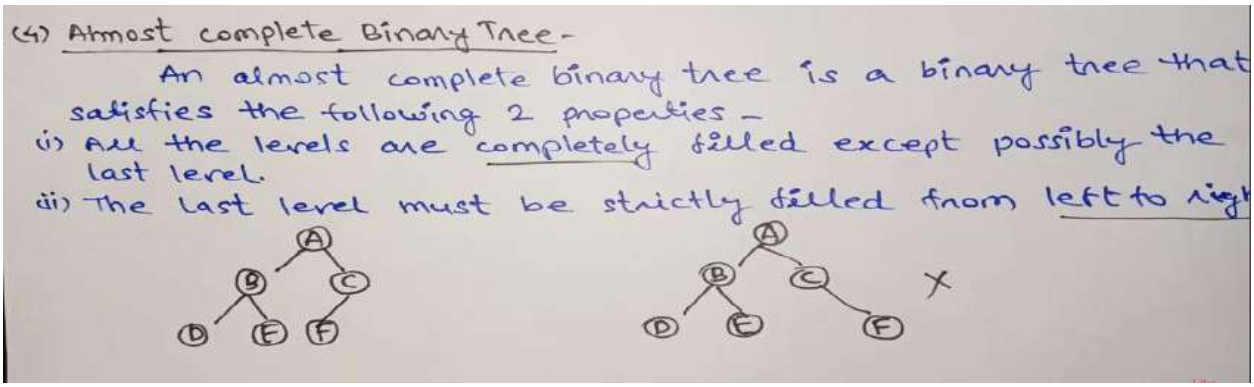


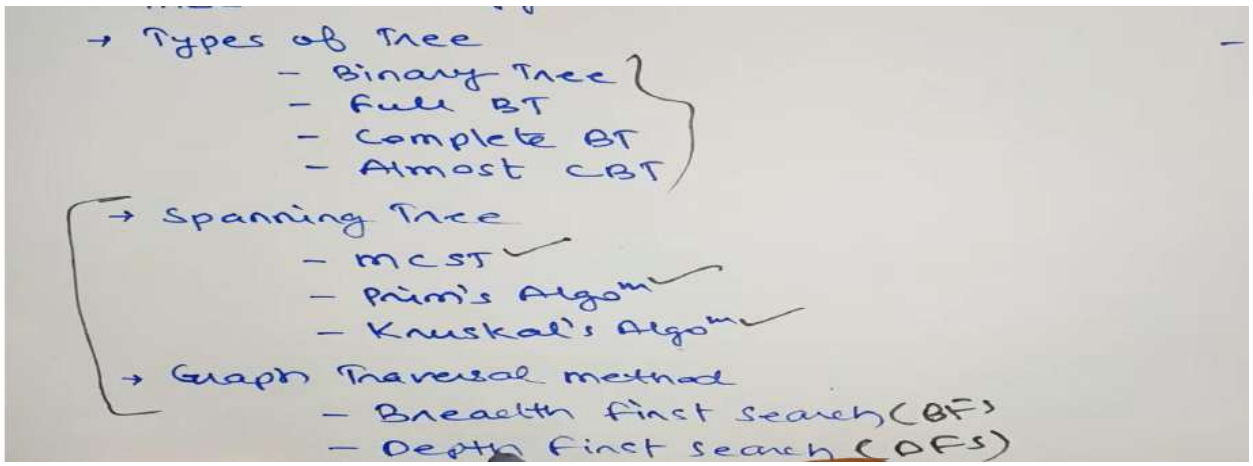
A tree connected network











Tree Traversal

Tree traversal is one of the most common operations performed on tree data structures. Traversing a binary tree is way in which each node in the tree is visited exactly once in a systematic manner. We know that the order of traversal of nodes in a linear list is the from first to last, however there is no such "natural " linear order for the nodes of a tree.

We have the following three orderings or methods for traversing or methods for traversing a non-empty binary tree.

- Preorder Traversal
- Inorder Traversal
- Postorder Traversal

All these traversal techniques are defined recursively

Preorder Traversal

In this technique, first of all we process the root R of the binary tree T. Then, we traverse the left subtree T_1 of R in preorder (which means that we traverse root of subtree T_1 first and then its left subtree). After visiting left subtree of R, then we take over right subtree T_2 of R and process all nodes in preorder.

Steps

1. Start
2. If root==NULL then return
3. Visit root
4. Traverse the left subtree in preorder
5. Traverse the right subtree in preorder
6. stop

Inorder Traversal

In this traversal technique, first of all we process the left subtree T_1 of the root R in order then we process the root and at last the right subtree T_2 of R

Steps

1. Start
2. If root ==NULL then return
3. Traverse the left subtree in order
4. Visit root
5. Traverse the right subtree in post order
6. Stop

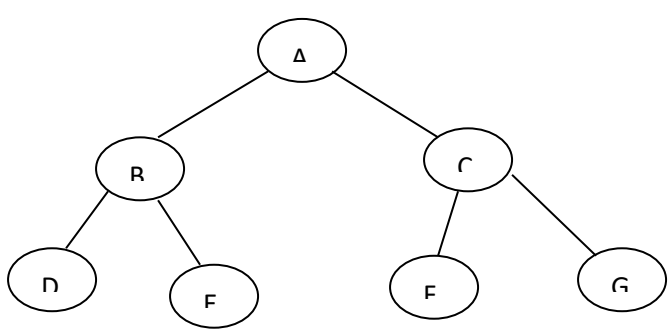
Postorder Traversal

In this technique first of all we process the left subtree T_1 of R in Postorder, then right subtree T_2 of R in Postorder and at the last, the root R.

Steps

1. Start
2. If root ==NULL then return
3. Traverse the left subtree in order
4. Traverse the right subtree in post order
5. Visit the root
6. Stop

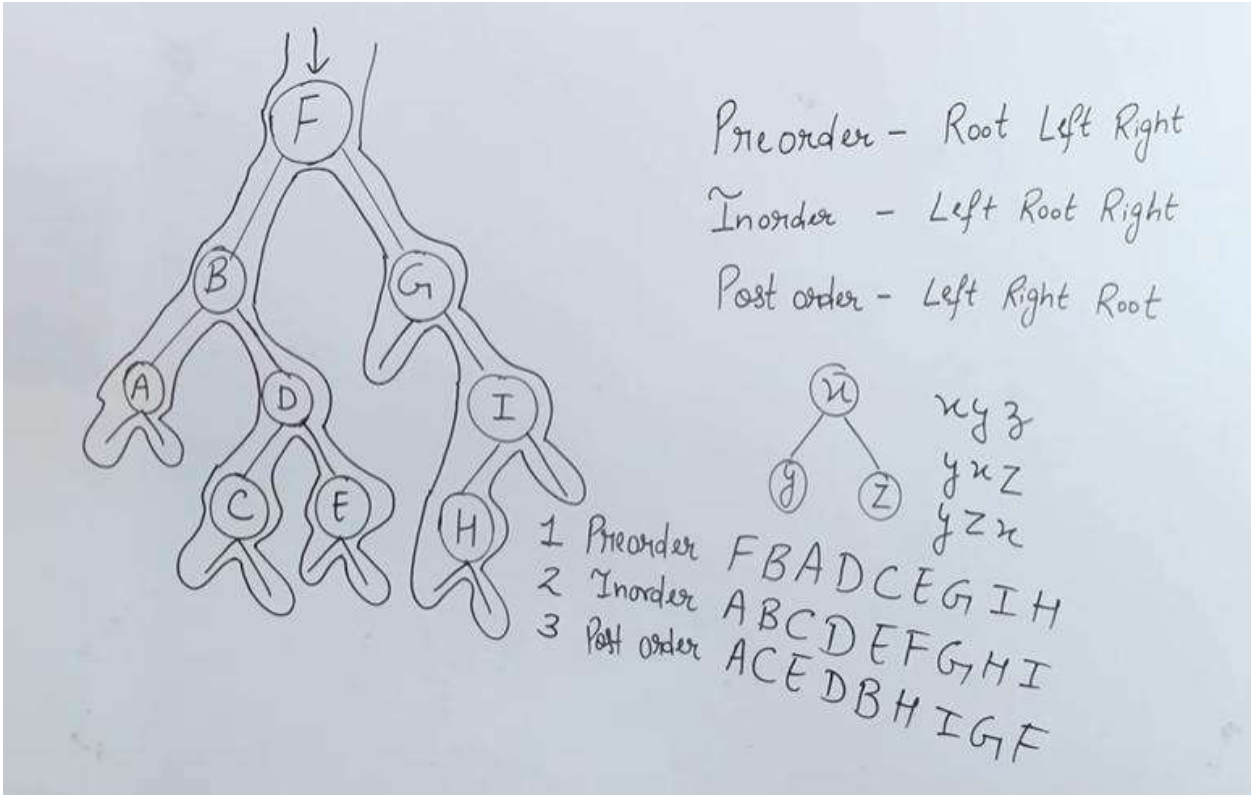
Example: Find preorder, inorder and Postorder traversal of following tree



Preorder Traversal: A B D E C F G

Inorder Traversal: D B E A F C G

Postorder Traversal: D E B F G C A



Spanning Tree: Let G = (V, E) be any connected graph then any subgraph T = (V, T') of given graph G consists of all the vertices and some(or all) of edges in G, which is acyclic is called spanning tree of given graph.

Example:

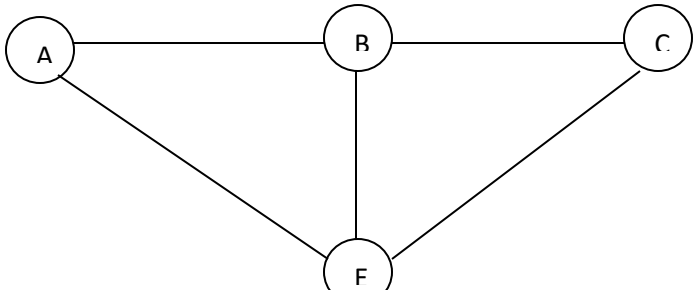
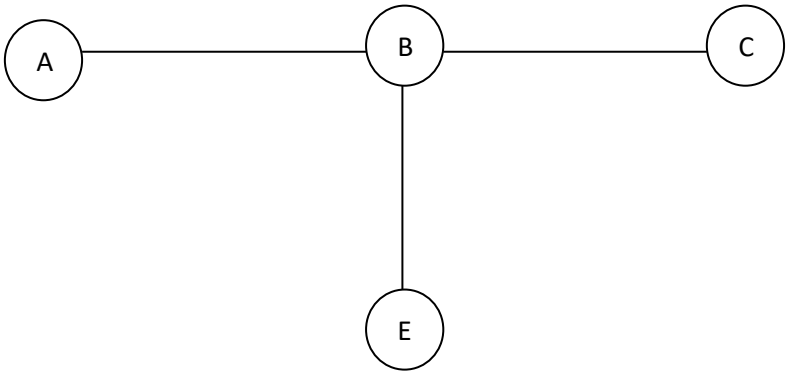
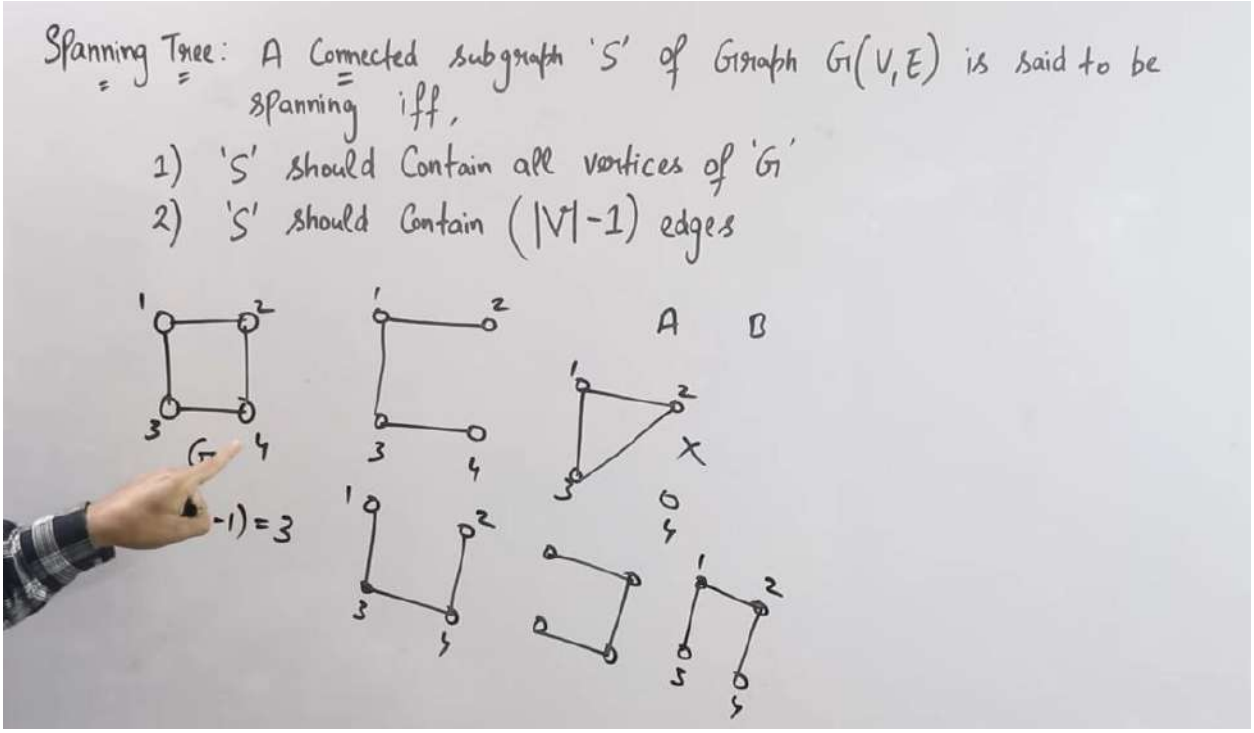
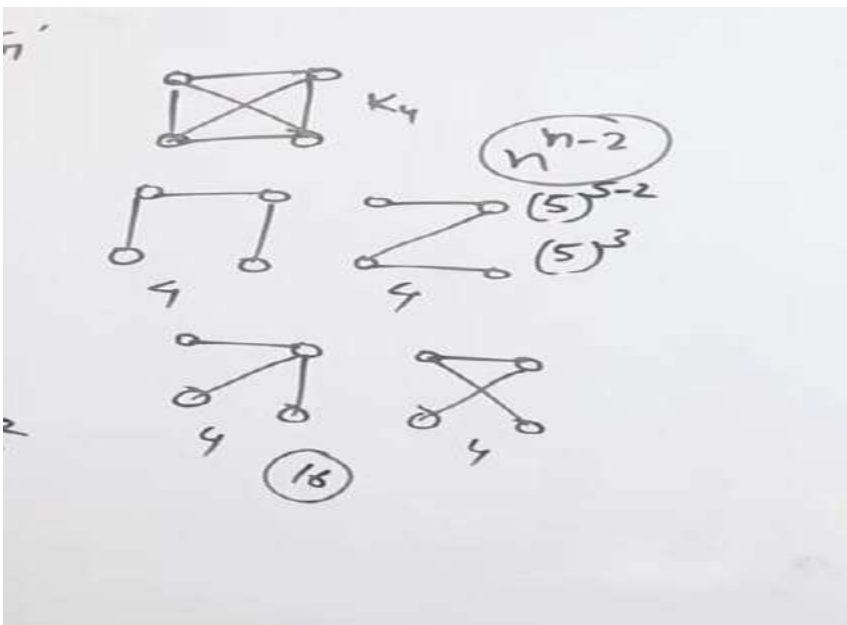


Figure: Graph G = (V,E)







Minimum spanning tree: Let G be a connected weighted graph. The weight of a spanning tree of G is the sum of the weights of the edges which are included on that spanning tree. Thus, a minimum spanning tree of G is a spanning tree of G with minimum possible weight among all possible spanning trees of that graph. Consider a figure which shows six cities and cost of lying railways links between certain pair of cities. We want to set up railways link between the cities at minimum costs.

There are several algorithms for determining the minimal spanning tree of given weighted graphs. Here we discuss only Kruskal's Algorithm

Kruskal's Algorithm

The algorithm involves the following steps

Step 1: List all the edges of G with non-decreasing order of their weights

Step 2: Select an edge of minimum weight (if there are more than one edge of minimum weight then arbitrarily choose one of them). This will be the first edge of T

Step 3: At each stage, select an edge of minimum weight from all the remaining edges of G if it does not form a cycle with the previously selected edges in T. Then add the edge to T

Step 4: Repeat step 3 until n-1 edges have been selected

Psedudocode for Kruskal's Algorithm

```
{
        T = {V} // set of vertices
        E =  set of edges sorted in non-decreasing order of their weight
        While (|T|<n and E!=NULL)
        {
                Select edge (u,v) from the edge E in order
                Remove (u,v) from E
                If edge(u,v) does not create cycle in T
                T = T U edge (u,v)
        }
}
```

Example: Find the minimum spanning tree of given graph using Kruskal's Algirthm

Binary Tree

A binary tree is a finite set of elements that is either empty or portioned into three disjoint subsets. The first subset contains the single elements called root of the tree, the other subsets are called left and right sub tree of original tree. The binary tree is so named because each node can have at most two descendants.

Types of binary trees

Strictly Binary Tree

It is a binary tree with non-empty right and left sub trees. In other words, it is binary tree with every node N has either 0 or 2 tree. The strictly binary tree is also known as extended 2 Tree or simply 2 –tree. Sometimes nodes with 2 children's are known as internal nodes and nodes with 0 children are known as external nodes.

Complete Binary Tree

It is a special type of strictly binary tree where all the leaves of the tree reside at the same level. Using the depth we always say a complete binary tree of depth d where all leaves with be at level d

Binary Search Tree

The application of binary tree is searching and sorting. By enforcing certain rules on the values of the elements stored in a binary tree, it could be used to search and sort.

Binary search tree is a tree in which value of each node in the tree is greater than the value of node in its left (if exists) and it is less than the value in its right child (if it exists).

As name suggests, binary search tree (BST) is used for searching purpose. For every node N in the BST, the following property will be true.

- The data at left node will be smaller than data at node N
- The data at right node will be larger than data at node N

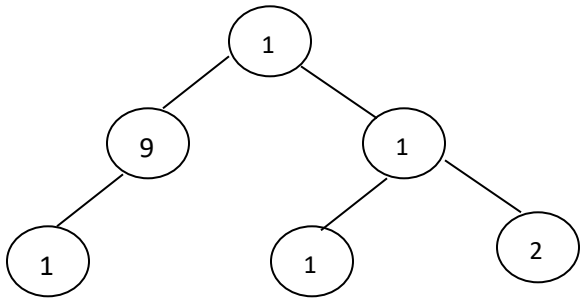
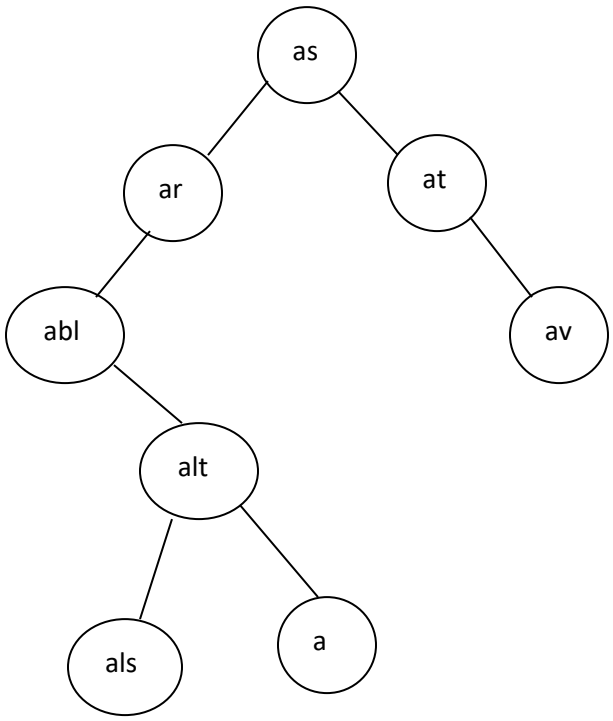


Figure: Binary Search Tree

Example: Draw a BST for the following words: ask, art, atc, able, alto, also, avoid, ant

Solution

The first key in the list act as root key for node in the tree



Expression Tree

Binary trees are used to represent arithmetic expressions. An expression tree is a binary tree used to represent mathematical expression in which all the leaf node contains operands such as constants or variables and non leaf node contains mathematical operators.

Example: Use binary expression tree to represent the expressions

a. a-b
b. (a+b)*c
c. (a+b)*(c-d)
d. ((a+b0*c)+(d/e)
e. (a+b)*(c+(d/e))

M-ary Tree

A rooted tree T = (V,E) in which every non-terminal vertex has no more than m-children is called m-ary tree.

Full m-ary Tree: A m-ary tree T = (V,E) where every internal vertex has exactly m children is called full m-ary tree.

An m-ary with m=2 is called binary tree

Example:

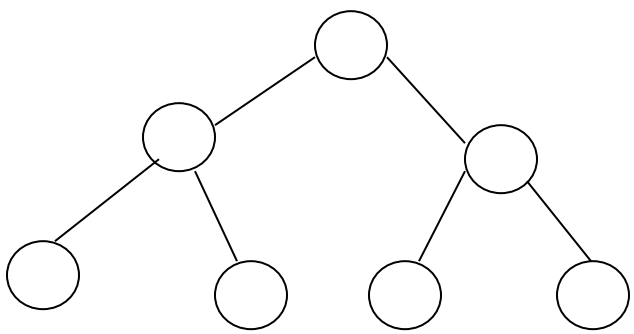


Figure : Full 2 ary tree

Theorem: A full m-ary tree with 'I' internal vertices contain n = mi+1 vertices.

Proof: Let T = (V,E) is a full m-ary tree with 'I' internal vertices except the root are child of internal vertices.

Since, by definition of full m-ary tree each internal vertex has exactly m children and hence total vertices except root is equal to m*i

Hence full binary tree T has total number of vertices (n) = m*i+1 including root

Theorem

A full m-ary tree with

i. n vertices has i = (n-1)/m internal vertices and l = [(m-1)*n+1)]/m leaves
ii. i internal vertices has n = mi+1 vertices and l = (m-1)*i+1 leaves
iii. l leaves has = (ml-1)(m-1) vertices and i = (l-1)/(m-1) internal vertices

Proof: A full m-ary tree with i internal vertices has

n = mi+1 vertices……(i)

and n = i+l [Total vertices = no of internal vertices + leaf vertices

From eqn(i)

n = mi+i……(ii)

Or mi = n-1

Or i = (n-1)/m proved

Again putting the value of I in eqn(ii), we get

n = (n-1)/m+l

or l = n-(n-1)/m

or l = (m*n-n+1)/m

or l = (n(m-1)+1)/m

(ii) We know that

n = mi+1

and n = i+l

and hence

mi+1 = i+l

or l = i*(m-1)+1 Proved

(ii) We know that

n = mi+1……(1)

and n = l+1……..(2)

Solving these two equations we get

mi+1 = l+i

or mi-i = l-1

or i*(m-1) = l-1

i = $\frac{(l-1)}{(m-1}$ proved

Again putting value of (i) in (1)

n = m $\frac{(l-1)}{(m-1}$ +1

or n= $\frac{(ml-1)}{(m-1}$ Proved

Theorem

Let G(V,E) be a loop free undirected graph. Then G is a tree if there is a unique path between any two vertices of G

Proof: Let G = (V,E) be a graph. Since there is a path between each pair of vertices u and v, G must be connected. Thus, to show G is a tree it remains to show that G has no cycles. Since G is loop free, it has no cycles of length 1. Suppose that G has a cycle of length greater than one, say

C = (v_1, v_2, v_3....v_n, v_1)

Then any two distinct vertices of the cycle C are joined by two paths which contradict the fact that there is unique path between any two vertices of G. Hence G, has no cycles and so it is tree

Or

Proof:

1. G is connected because there is a unique path between any two vertices.
2. G has no cycles because a cycle would imply the existence of at least two different paths between some vertices.
3. Thus, G is a connected acyclic graph, which is the definition of a tree.

Theorem

A tree with n vertices has exactly n-1 edges

Proof:

Let G = (V.E) be a tree with n vertices. We use induction method to prove it. Let n = 1. i.e. has one vertex, since it has no loops, the number of edges in G is 0. This establishes that it is true for n=1

Let it be true for n. Now we wish to show it is true for n+1. Let G be a tree with n+1 vertices and let u be a vertex of degree n. If we remove such a vertex and the edge incident on it, then sub graph G-u is still connected and has no cycles. Hence G-u is a tree. However G-u has n vertices, so by induction it has n-1 edges. Since G-u has exactly one edge less than that of G, it follows that G has n edges. So summing n+1 vertices of g we got n edges. This completes the proof.

Theorem

In any tree G, there are at least two pendant vertices

Proof

Let G = (V,E) be a tree and v_1 be a vertex of degree 1. If there is a vertex v_2 adjacent to v_1 with degree 1 then we are done. If not, we select v_3 adjacent to v_2. Continuing in this way, we get a sequence of vertices v_1,,v_2,v_3……v_n. Since a tree has no cycle, this sequence terminates for some k. ie there exists an adjacent vertices with degree 1. Thus v_1 and v_k are our desired vertices each of degree 1.

OR A tree with only one vertex (also known as a singleton tree) has no pendant vertices. However, if a tree G has two or more vertices, then it must have at least two pendant vertices. This is because in a tree, all vertices except the root have degree at most 2, and if a vertex has degree 0, then it is not part of the tree. Therefore, if a tree has n vertices, the root has degree at least 1, and there are n-1 edges, so there must be at least n-1 vertices of degree 1, and hence at least two pendant vertices.

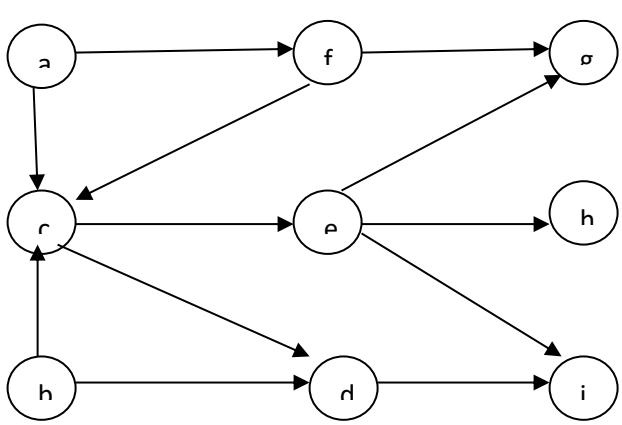
Network Flow

Definition of Transport Network

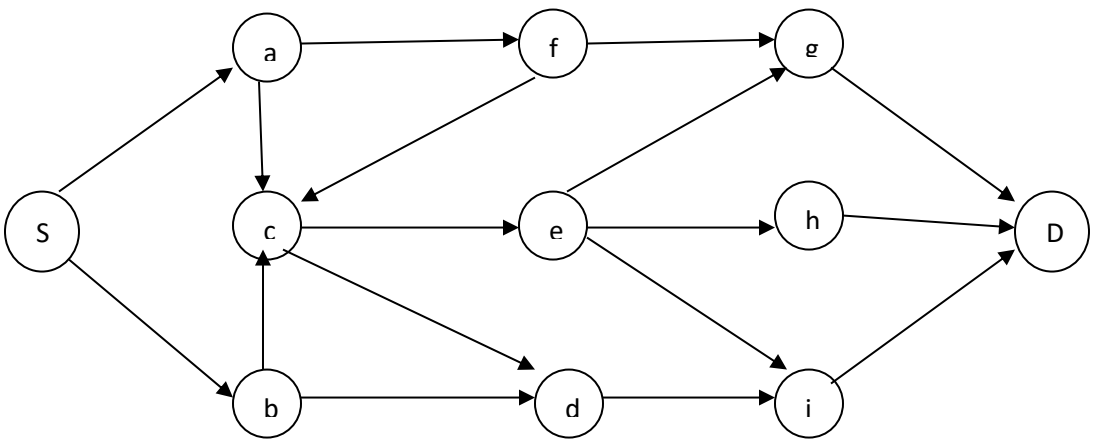
In graph theory, a flow network (also called transport network) is a directed graph where each edge has a capacity and each edge receives a flow. Here, capacity implies the maximum rate at which something flows through the edge. The amount of flow on an edge cannot exceed the capacity of the edge. All vertices except the source denoted by S and sink denoted by D are called intermediate vertices. A flow must satisfy the restriction that the amount of flow into an intermediate vertex equals the amount of flow out of it, unless it is a source S, which has only outgoing flow, or sink D, which has only incoming flow. A flow network can be used to model traffic in a road system, fluids in pipes, currents in an electric circuits, or anything similar in which something travels through a network of nodes.

Note: In most cases, there is a single source and single destination but sometimes there are multiple sources and multiple destinations. In such cases, we can create the equivalent transport network by combining these multiple source into single source with additional edge

Example



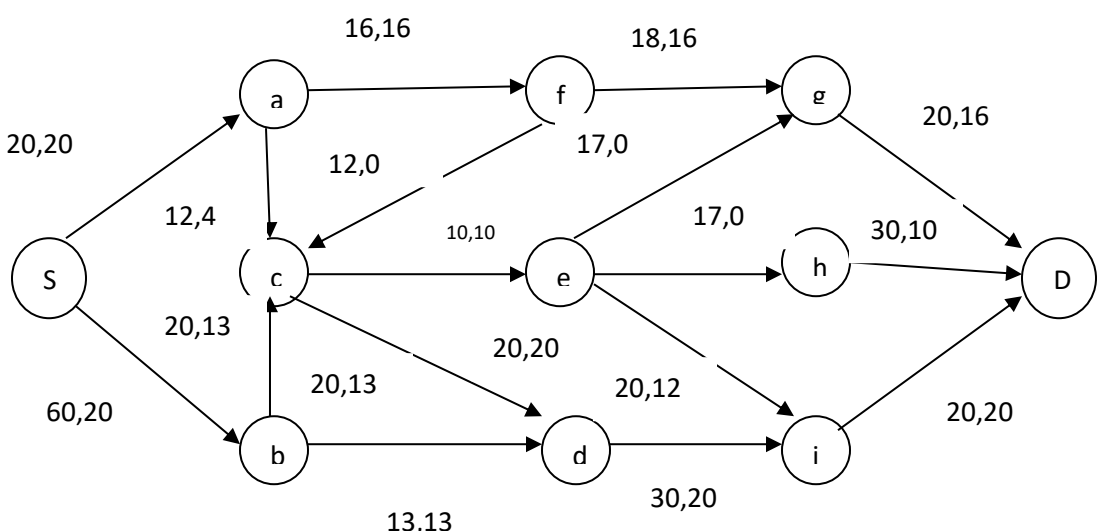
Then, the equivalent transportation network with single source S and single destination D is as shown below:



Flows: The material (or oil) transport along any edge is known as flow

Let us label edges with two no, first label represents the capacity of edge and second represent the amount of flow through that edges. The criteria are:

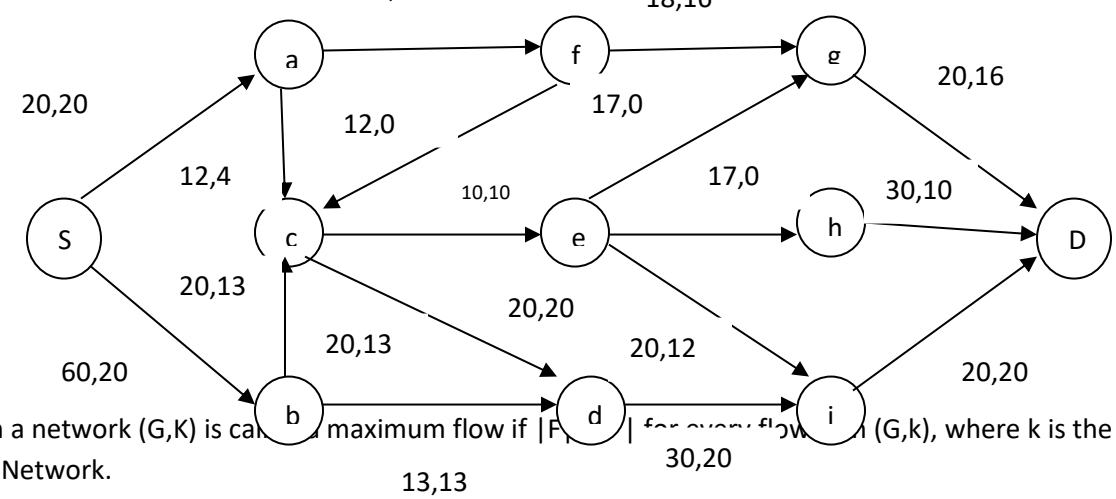
- The amount of flow can't exceed the capacity of that edge
- For every node (except S and D), the amount of flow flowing into matrix v (**incoming flow**) must be equal to the amount of flow flowing out of vertex v



Network Flow Problem

Given a transport graph G = (V,E) where each edge e is associated with its capacity C(e)>0 and the two special node: source node and S and sink or destination node D then the network flow problem states that what is the maximum total amoun 16,16 ossible to car 18,16 source node S to Destination node D

Example



A flow F in a network (G,K) is ca maximum flow if |F | | for every flow (G,k), where k is the capacity of Network.

Computing Max Flow: Concept and Process

To find the maximum flow in the given transport network |F|≥|F'|, the following steps are necessary.

- Identify the augmenting path
- Increase floe along that path
- Repeat the above two steps still the maximum flow is obtained

Augmenting Path

- An augmenting path P is simple path from S to D with unsaturated edge
- We build augmented path with only forward unsaturated edge
- Augmenting path with some backward edge

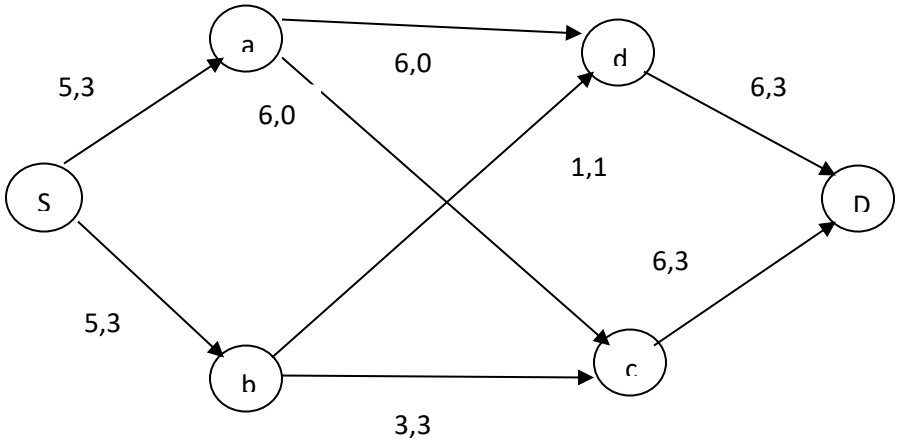
In backward edge Augmenting Path (A.P), the backward edge must have positive flow and of course, the forward edge must be unsaturated.

When an edge (d—>c) is taken in opposite direction (c→d), is called backward edge

Two basic Ways to Increase the Value of Flows

1. If an edge is not being used to capacity, try to send more flow through it
2. If an edge is working against us by sending some flow back toward the source, we could try to reduce the flow along this edge and redirect in a more practical direction

Example:



In the above given network, there is only one augmented path with only forward edge. This path is S→a→d→D

Now, we increase flow along path S→a→d→D

Now,

k(S,a)-F(S,a) = 5-3=2

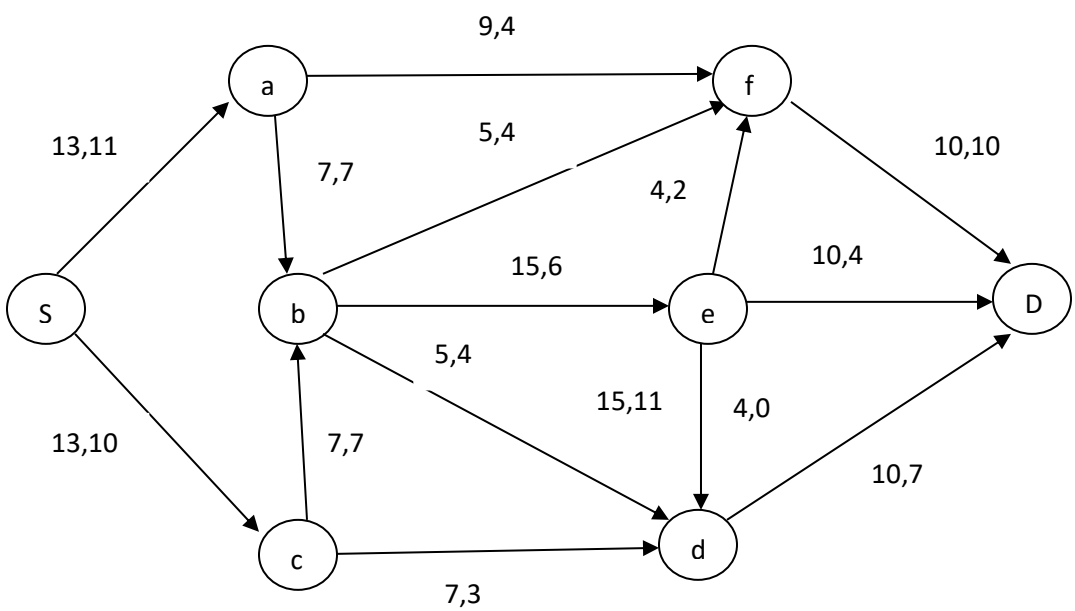
k(a,d)-F(a,d) = 6-0 = 6

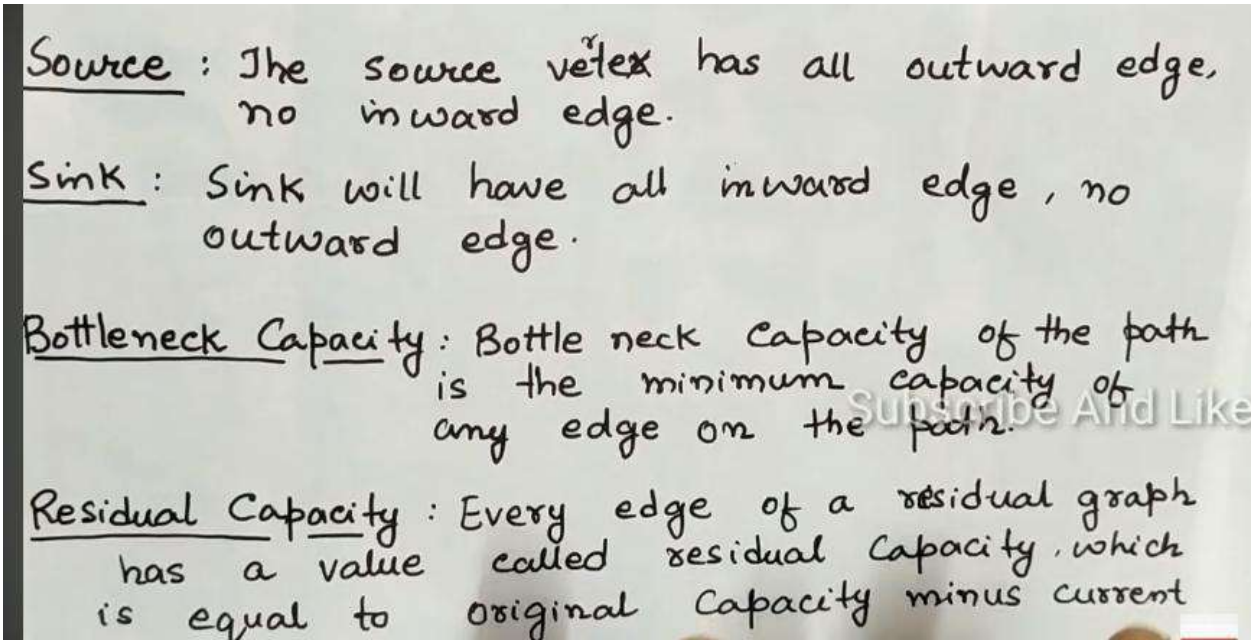
k(d,D)-F(d,D)= 6-3=0

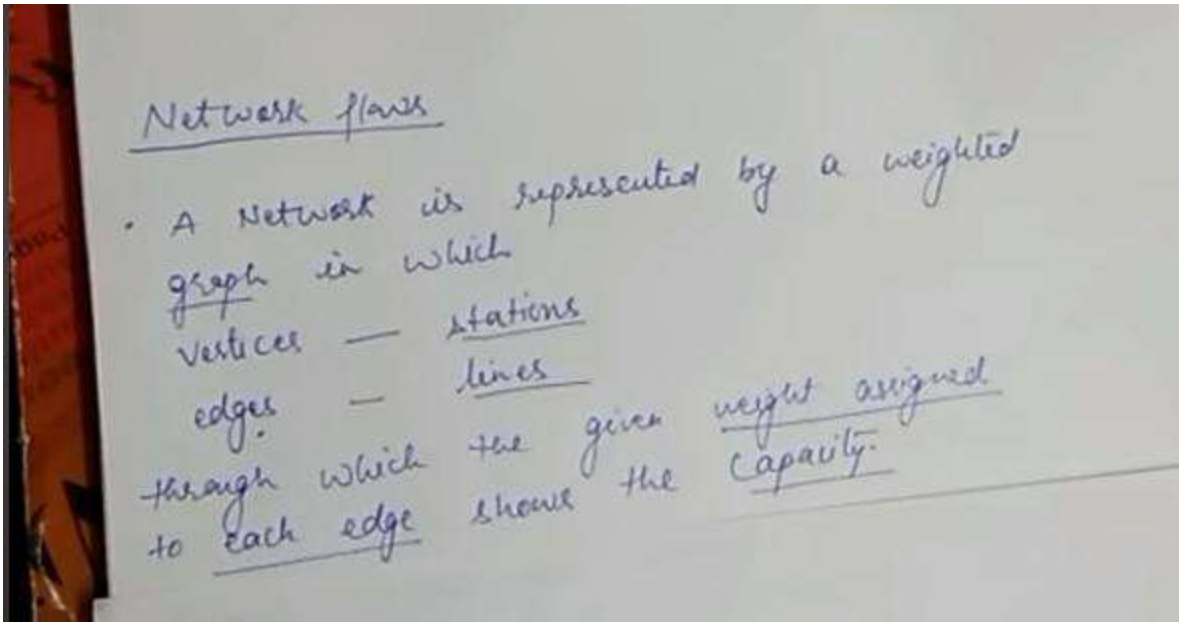
Min (2,6,3) = 2

Increase flow by 2 along path S→a→d→D

Example 2: Find a maximum flow for the network below




Source : The source vetex has all outward edge, no inward edge.
Sink : Sink will have all inward edge, no outward edge.
Bottleneck Capacity : Bottle neck capacity of the path is the minimum capacity of any edge on the path.
Subscribe And Like
Residual Capacity : Every edge of a residual graph has a value called residual capacity, which is equal to original capacity minus current


Network flows
• A Network is represented by a weighted graph in which
vertices — stations
edges — lines
through which the given weight assigned to each edge shows the capacity.

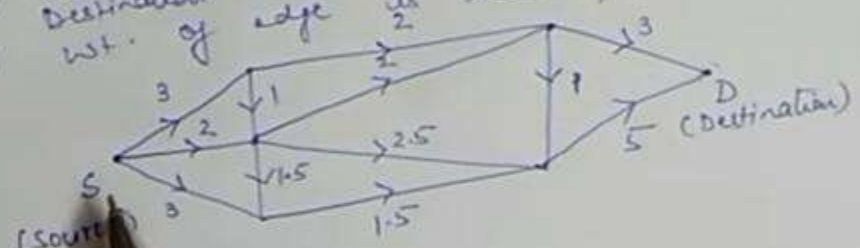
possible per unit of
that line.

- A Network has satisfies following properties:-

a) Contain one & only one vertex – no incoming edge
Source.

b) Contain one & only one vertex – no outcoming edge.
Destination

c) wt. of edge is real positive no.



Network flows

Flow $f(v_i, v_j)$

a) $0 \leq f(v_i, v_j) \leq c(v_i, v_j)$

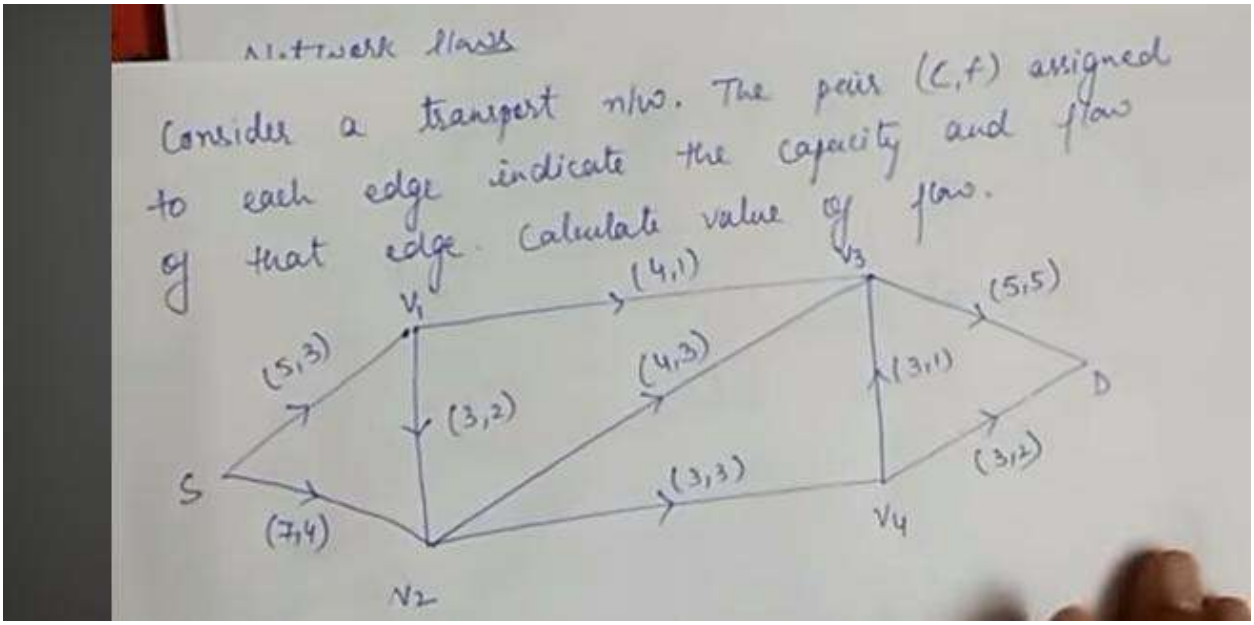
b) $\sum_{\forall i} f(v_i, v_j) = \sum_{\forall k} (v_j, v_k)$

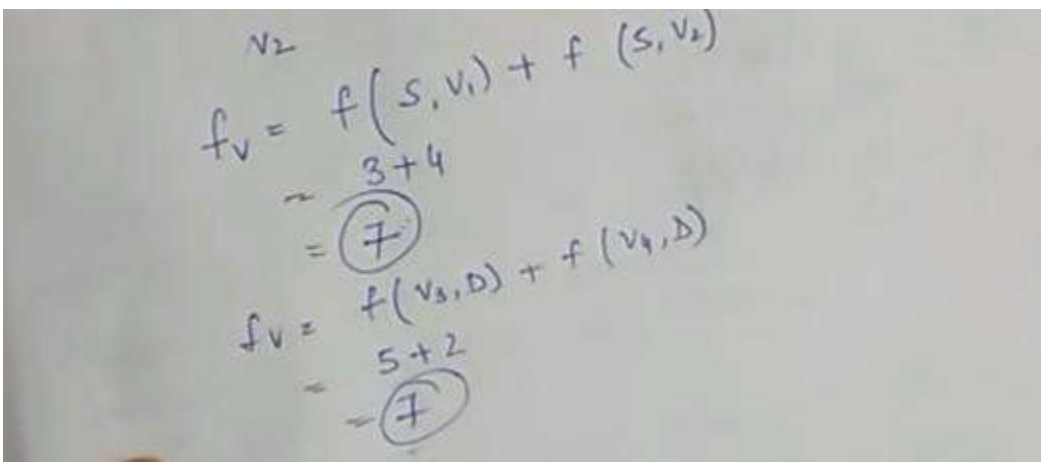
c) $f(v_i, v_j) = 0$

Value of the flow :-

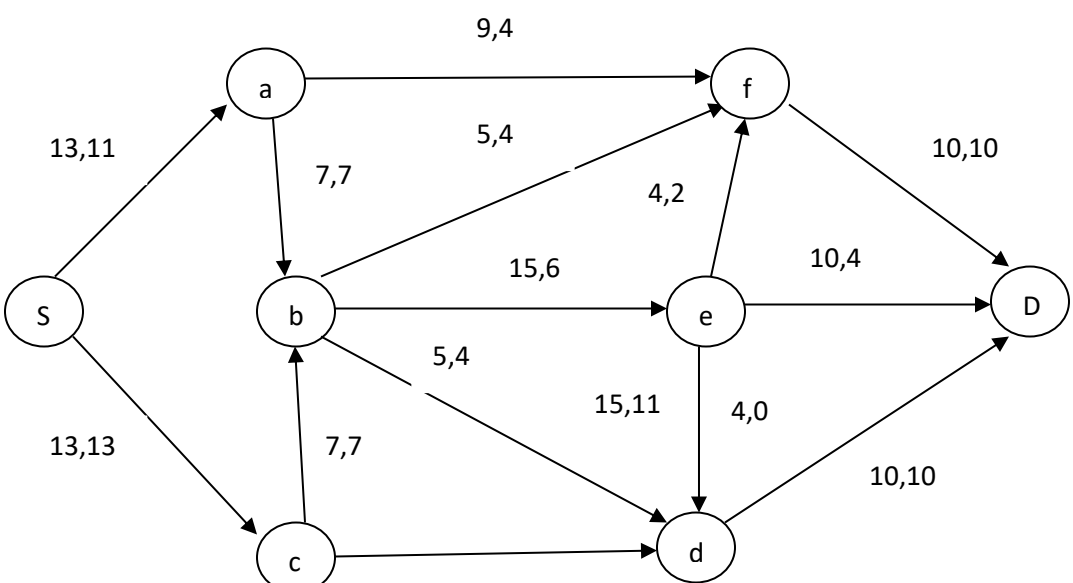
$$f_v = \left[\sum_{\forall i} f(s, v_i) - \sum_{\forall i} f(v_i, s)\right]$$

$$= \left[\sum_{\forall k} f(v_k, D) - \sum_{\forall k} f(D, v_k)\right]$$



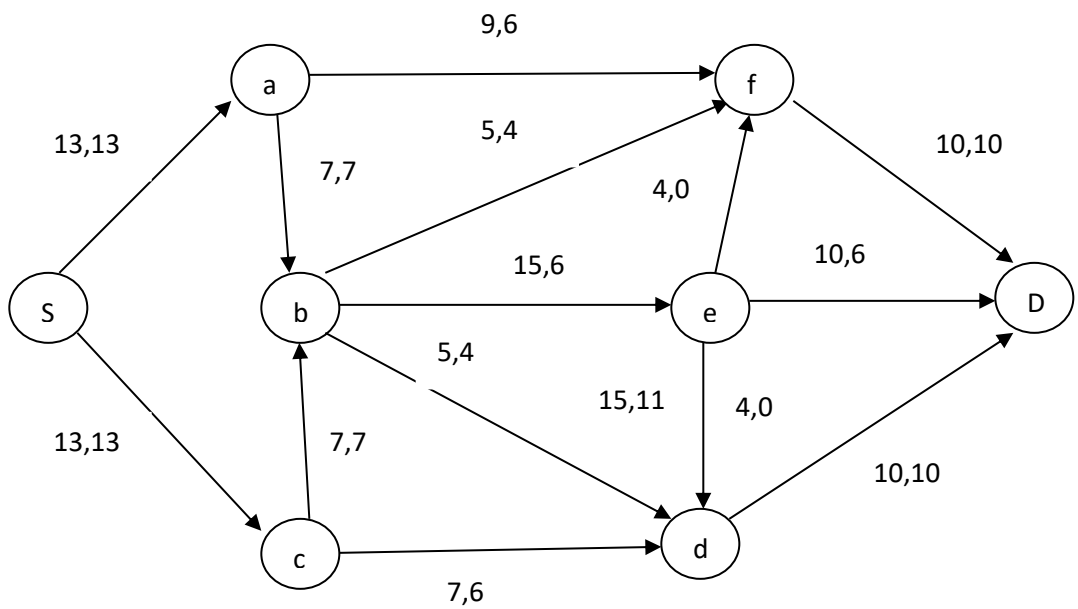


Solution: Here first we look for an augmented path where, if possible, all edges are forward (they have to be unsaturated). The path P1: S→c→d→D is such a path. The minimum slack of the edges along p1 is 3. By increasing the flow by 3 along the edges of p1, we obtain the flow F1 as shown below



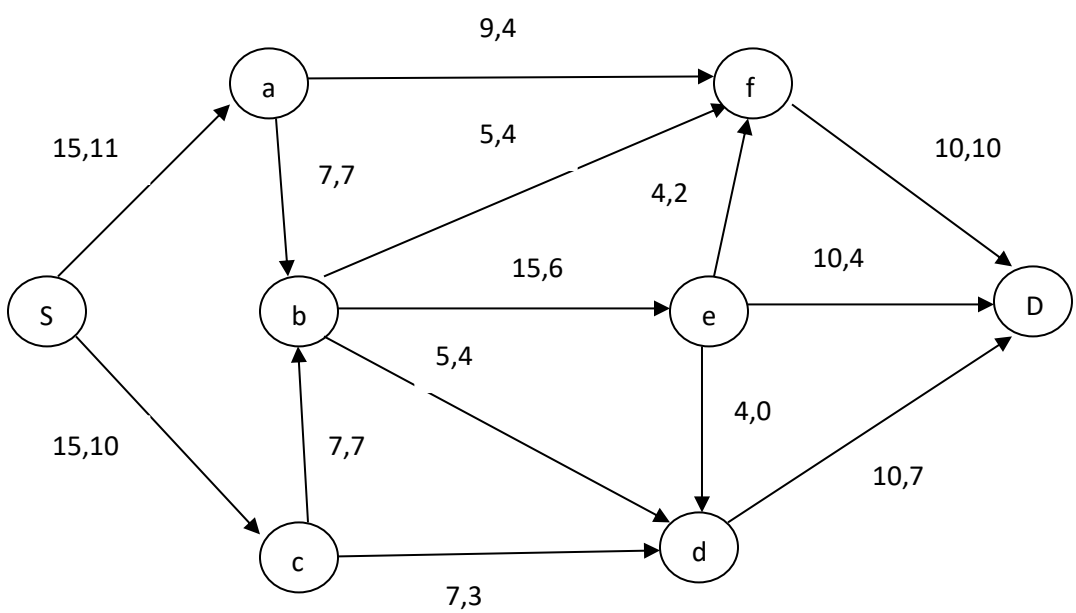
Now, there are no directed S-D paths with unsaturated edges. Thus, we look for augmenting paths with some backward edges. Note that the backward edges must have positive flow and the forward edges must be unsaturated.

Now, we augment the flow by 2 along the path S→a→f→e→D (with backward edge from f to e) to obtain the flow as in figure below



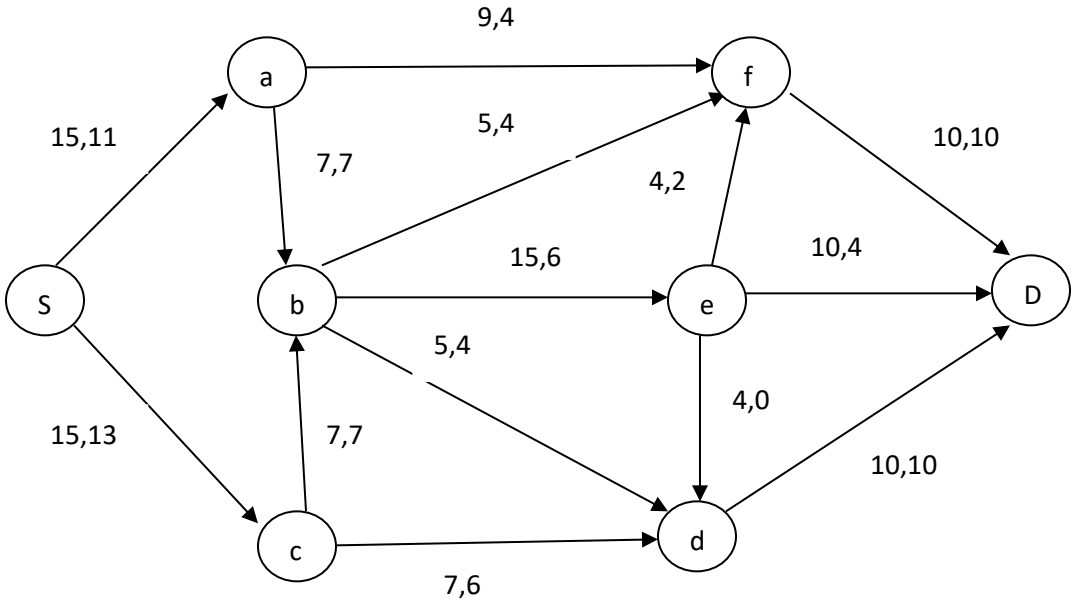
There are no more augmenting paths from S to D because both edge (S,a) and (S,c) has become saturated. Thus, the maximum flow is 13+13= 26 (sum of outgoing flow at source node) or 10+6+10 = 26 (sum of incoming flow at sink node)

Example: Find a maximum flow for the network shown below



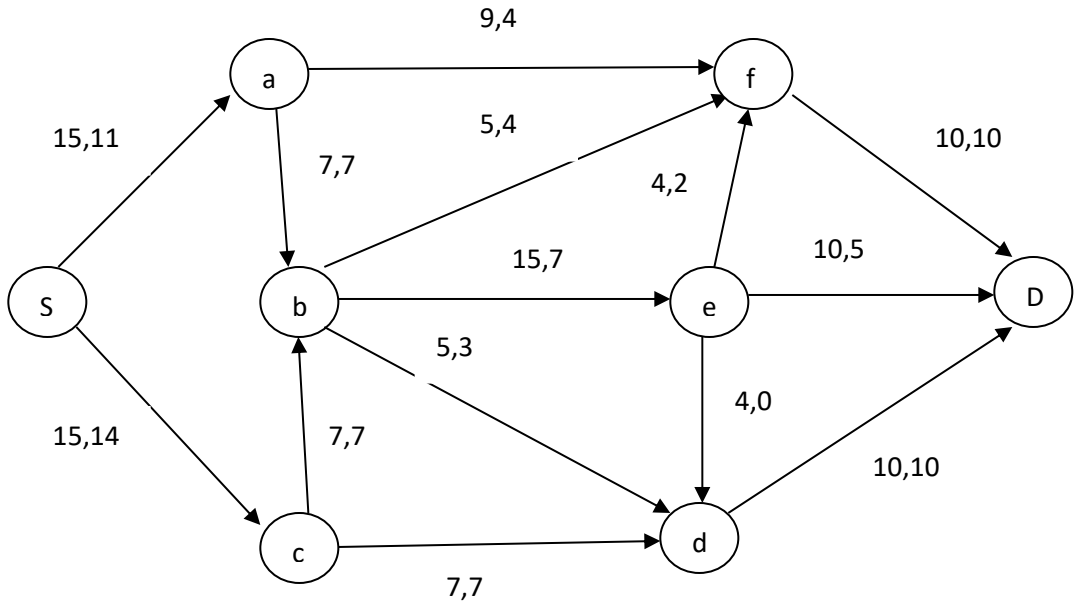
Solution: First we try to find out the augmenting path with all forward edge if possible. The path P1: S→c→d→D is such path.

The minimum slack (the value that can be augmented) along path P1 is 3. After increasing by 3, we have the flow network as :

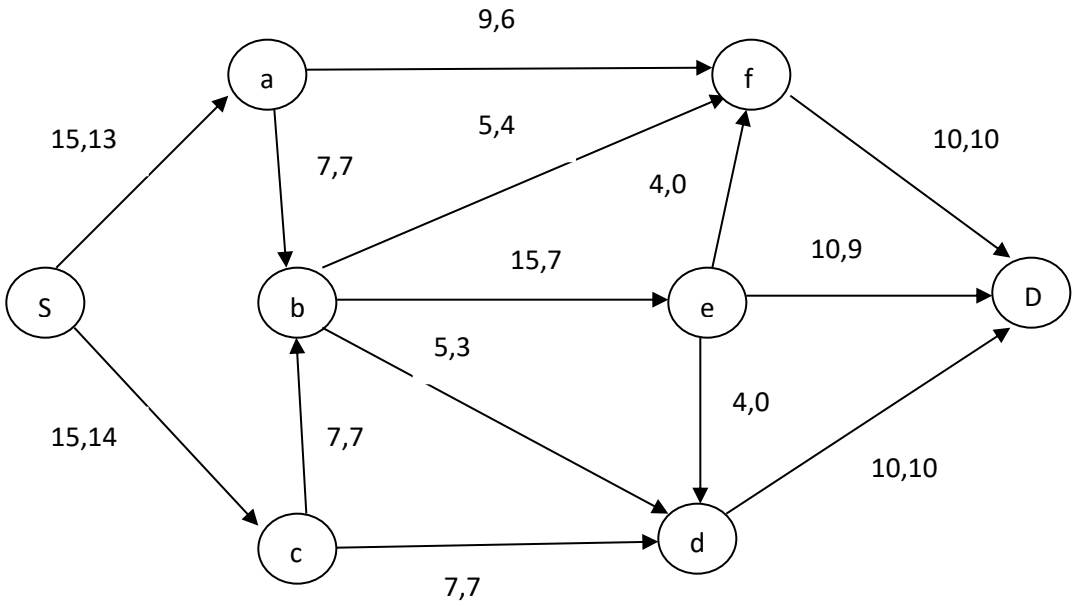


Now, there is no directed augmenting path. So, we look for augmenting path with some backward edge.

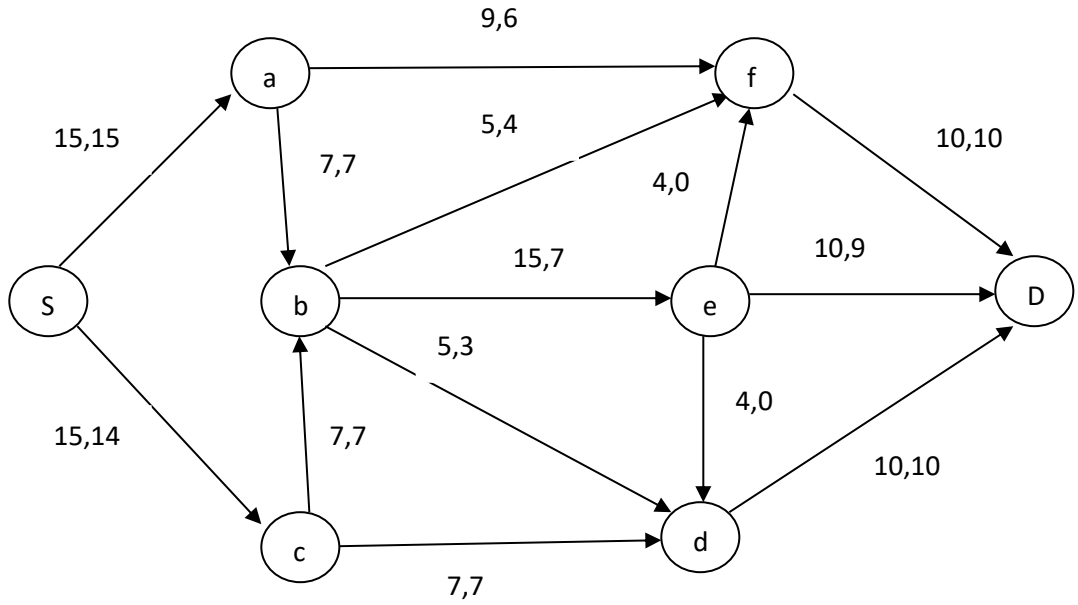
The path P2: S→c→d→b→e→D is such path with minimum slack 1. So after augmenting this path with flow 1, the network is:



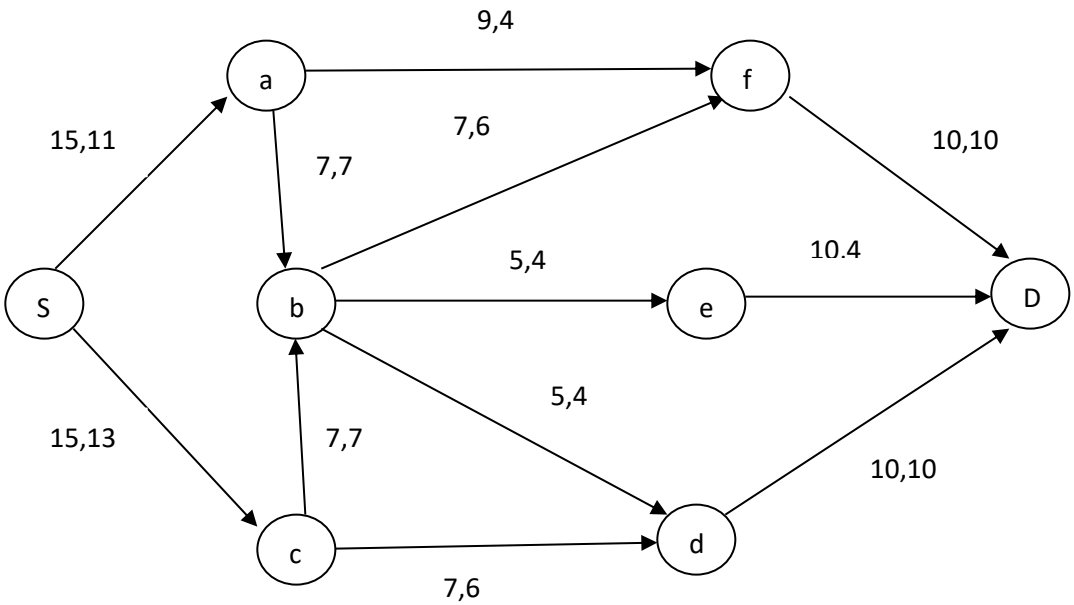
Now, another augmenting path with backward edge is p3: S→a→f→e→D, where the minimum slack is 2 (since we cannot make backward edge negative) the flow network is:



Now, another augmenting path with backward edge is P4: S→a→f→b→e→D where the minimum slack is 2, the flow network is now



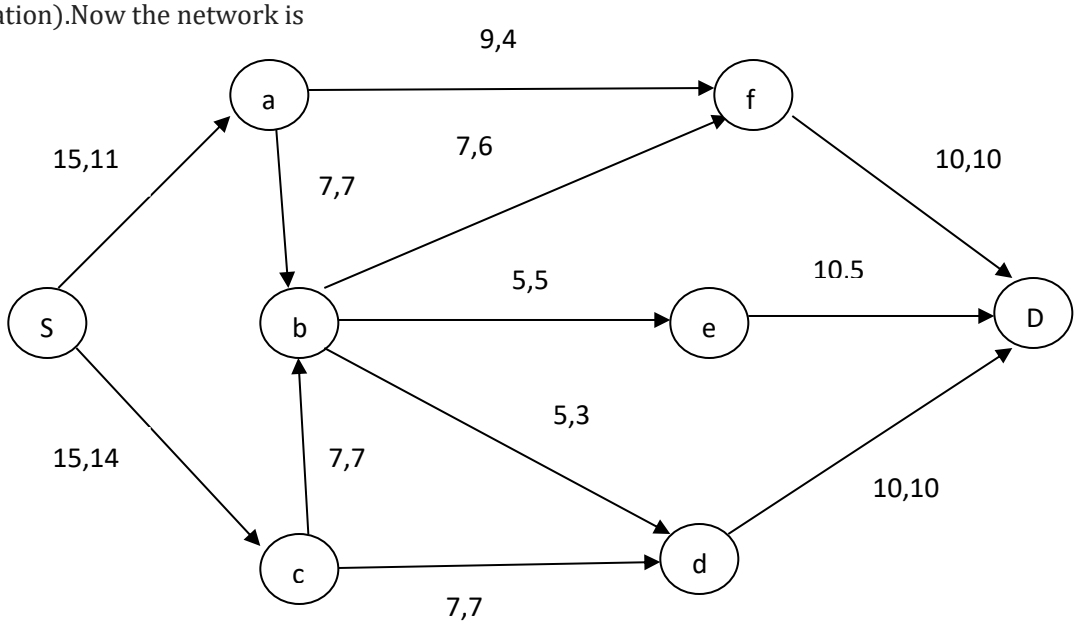
Example: Find a maximum flow for the network below



Solution: In the given network, there is no augmenting path with only forward edge. So we have to find (if any) augmented path with some backward edge. One of such path is:

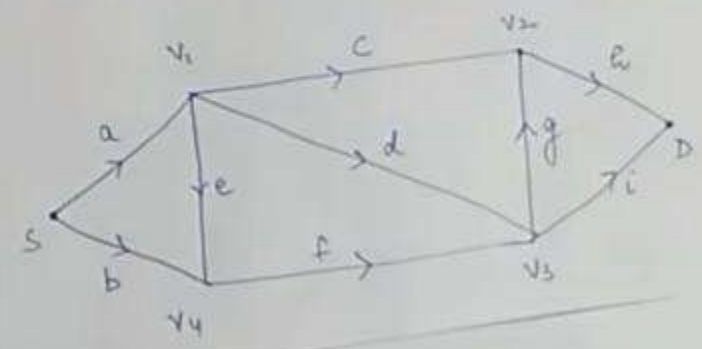
S→c→d→b→e→D with (d,b) as backward edge.

This is an augmenting path since we can increase flow by 1 along the path because the minimum slack along that path is 1. [Note that in backward edge, we subtract the flow by 1 to maintain the rule of flow conservation).Now the network is



Cut Set in a N/W Flow & its Capacity

- A cut set separates the source & sink
- It is denoted by $(X, \bar{X})$



V_4

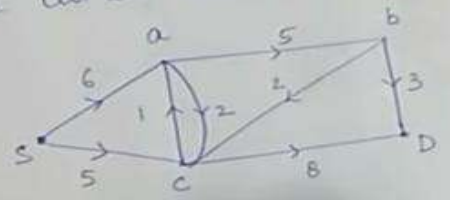
- Capacity of the cut-set denoted by $c(X, \bar{X})$

$$c(X, \bar{X}) = \sum_{\substack{v_i \in X \\ v_j \in \bar{X}}} c(v_i, v_j)$$

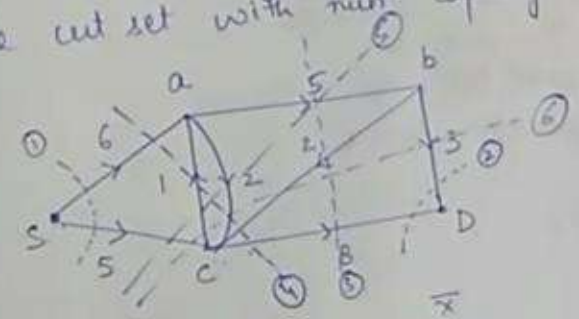
- Cut $(X, \bar{X})$ is min. if there is no cut $(Y, \bar{Y})$ in the n/w such that

$$c(X, \bar{X}) > c(Y, \bar{Y})$$

Q. In given transport n/w, find all cut-set that separate source from sink. Also find out the cut set with min. capacity.



separate source from sink
the cut set with min. capacity



① $(S,a) + (S,c)$
$6 + 5 = 11$

② $(b,D) + (C,D) = 11$
$3 + 8$

③ $(a,b) + (b,C) + (C,D)$
$5 + 1 + 8 = 13$

S.No	X	$\bar{X}$	Cut $(X, \bar{X})$
1.	$\{S\}$	$\{a,c,b,D\}$	11
2.	$\{S,a,c,b\}$	$\{D\}$	11
3.	$\{S,a,c\}$	$\{b,D\}$	13
4.	$\{S,c\}$	$\{a,b,D\}$	
	$\{S,a\}$	$\{b,c,D\}$	
		$\{c,D\}$	

	X	$\bar{X}$	Cut $(X, \bar{X})$
	$\{S\}$	$\{a,c,b,D\}$	11
	$\{S,a,c,b\}$	$\{D\}$	11
	$\{S,a,c\}$	$\{b,D\}$	13
	$\{S,c\}$	$\{a,b,D\}$	15
4.	$\{S,a\}$	$\{b,c,D\}$	12
5.	$\{S,a,b\}$	$\{c,D\}$	12
6.			

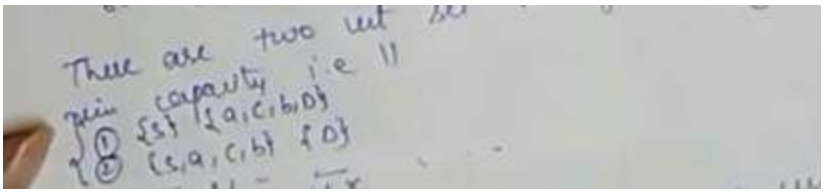
④ $(S,a) + (C,a) + (a,c) + (b,c) + (c,D)$
$6 + 1 + 8 = 15$

⑤ $(S,c) + (C,a) + (a,c) + (a,b)$
$5 + 2 + 5 = 12$

Max flow min-cut Theorem

- In a given transport n/w the max value of flow i.e fmax from source to sink is equal to min. value of the capacities of all the cut in that separate source from sink.

i.e. $f_{max} = \min C(X, \bar{X})$



S- D Cut or Cut

A cut of transport network is a set of edges whose removal will divide the network into two halves X and X' where

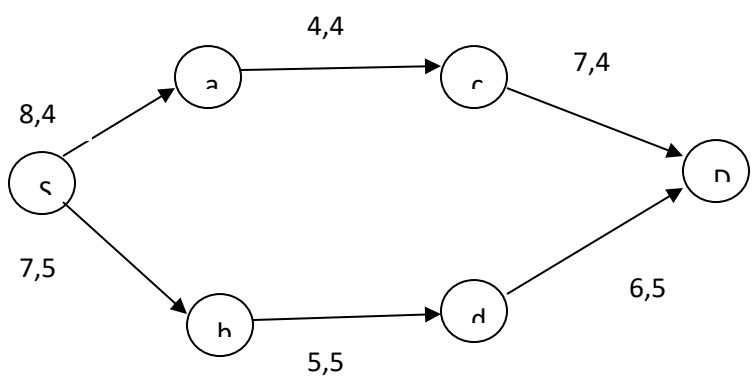
- Source vertex S ∈ X
- Sink vertex D ∈ X'

It is denoted by (X, X')

Capacity of S-D Cut

The capacity of S-D cut (X, X') is defined as sum of all capacities of edges from X to X'. It is denoted by K(X,X')

Example: Find all the S-D cuts in the given Transport Network.



Solution: All the possible S-D cut are:

X	X'	Cuts(X,X')	K(X,X')
S	{a, b, c, d, D}	{(S, a),(s, b)}	15
{S, a}	{b, c, d, D}	{(S, b),(a, c)}	11
{S, b}	{a,b,c,d,D}	{(S,a),(b,d)}	13
{S, a, c}	{b, d, D}	{(S, b),(c, D)}	14
{S, a, b}	{c, d, D}	{(a, c),(b, d)}	9
{S, b, d}	{a, c, D}	{(S, a),(d, D)}	14
{S, a , c, b}	{d, D}	{(c, D),(b, d)}	13
{S, a, b, d}	{c, D}	{(a, c),(d, D)}	10
{S, a, c, b, d}	{D}	{(c, d),(d, D)}	13

The capacity of minimal cut is 9

Maximal Flow and Minimal Cuts

Suppose that (G, k) is a flow network with source S and sink D, where G represents the graph and k represents the capacity of the edges. Suppose that X is a set of vertices such that S ∈ X but D ∉ X. Let X' denotes the complement of X in V. Then the set (X, X') of all edges from a vertex in X to a vertex in X' is called an S-D cut

If C is any set of edges in a transport network (G,k), then the capacity of C is the sum of the capacities of the edges of C. thus, the capacity of k (C) is defined as:

K (C) $= \sum_{e \in C} k(e)$

The capacity of an S-D cut, k (X, X') is the sum of all capacities of the edges from X to '. There may be edges from X' to X but are not exerted into the computations of k(X, X'). We call an S-D cut (X, X') a minimal cut if there is no S-D cut (Y, Y') such that

K(Y,Y')<k(X.X')

Computing Minimal Cut from Max Flow

Let Vs be the set of vertices reached by augmenting path from the source S and V_D is the set of remaining vertices then the cut (Vs, V_D) is the minimal cut

One simple way but inefficient way to find the minimum cut is to simply list all possible cuts and select the smallest. However, the number of possible cuts is extremely large, it is impossible to list all possible cuts in a network.

A better approach is to make use of max flow min cut theorem

The minimum cut is actually simple to find after max is computed by Ford-Folkerson algorithm

The minimal cut is actually simple to find after max flow is computed by Ford-Folkerson algorithm

Simply mark the edges that carrying a flow equal to their capacity and look for a cut that consists only of marked edge and no other edges

Theorem (max flow min cut)

Statement: In any flow (transport network) network, the value of any maximal flow is equal to the capacity of a minimal cut

Proof: We know that for a given flow network with flow F and capacity of cut K, we have

Value of F$\leq$ capacity of k

Now, optimizing this flow network such that there are no F-augmenting paths, we have

Value of F = capacity of cut k

Let F* be maximum flow and k* be minimum cut for the network. Then for some flow F and cut k, we have

Capacity of k = value of F $\leq$ capacity of k*

But no cut can have capacity less than minimum cut, we have

Value of F = capacity of k*

Also, we have

Value of F* $\leq$ capacity of k* = value of F

But no flow can be greater than the maximum flow

Therefore value of F* = Capacity of k*

Hence proved

[Relations]

Discrete Structures (CSc 511)

Samujjwal Bhandari

Central Department of Computer Science and Information Technology (CDCSIT)

Tribhuvan University, Kirtipur,

Kathmandu, Nepal.

Representing Relations

Listings of ordered pairs (read yourself)

Matrix Representation

The relation with finite sets can be represented using the matrix (zero one matrix). Let A be a set (a_1, a_2, …a_n) and B be the set (b_1, b_2, …, b_m), where elements are listed in some arbitrary order we represent relation from A to B by matrix M_R = [m_{ij}], where

$$m_{ij} = \begin{cases} 1 \text{ if } (a_i, b_j) \in R. \\ 0 \text{ if } (a_i, b_j) \notin R. \end{cases}$$

Example:

Represent the relation {(1,1), (1,2), (1,3), (2,2), (2,3), (3,2), (3,3)} on the set {1,2,3) with matrix, where elements of the set is listed in increasing order.

Solution:

$$M_R = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Identifying properties

Reflexive: If all the diagonal elements are 1 i.e. all m_{ij} = 1 whenever i = j, then the relation represented by the matrix is reflexive **(is above matrix reflexive? Yes)**.

Symmetric: If m_{ij} = 1 in the matrix then m_{ji} = 1 must be true and if m_{ij} = 0 then m_{ji} = 0 is also true. (it means that the relation represented by a matrix is symmetric if and only if the matrix is equal to its transpose). **(What about above matrix? No)**.

Antisymmetric: If m_{ij} = 1 and i ≠j, then m_{ji} = 0 or, in other words, either m_{ij} = 0 or m_{ji} = 0 when i ≠j. **(What about above matrix? No)**.

Note: Read composition of relations

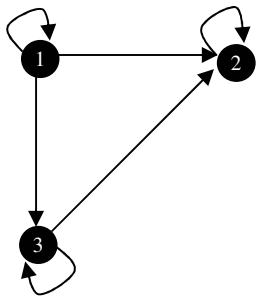
Directed Graph Representation

A directed graph, or digraph is a set of vertices V together with the set of edges. The vertex a is called initial vertex of the edge (a, b), and the vertex b is called the terminal vertex of this edge.

Example:

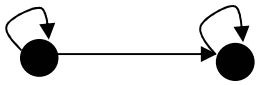
Draw the directed graph for the relation given in above example (example in matrix representation relation is {(1,1), (1,2), (1,3), (2,2), (2,3), (3,2), (3,3)}).

Solution:

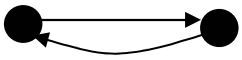


Identifying properties

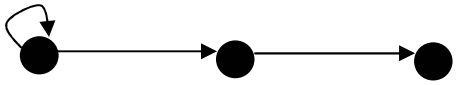
Reflexive: If every vertex has edge from the vertex to itself.



Symmetric: If for every edge of one direction there is another edge in opposite direction joining same two vertices as of first edge.



Antisymmetric: If no two distinct vertices have an edge going in both directions.



Closures of Relations

Let R be a relation on a set A. R may or may not have some property P, like symmetry, reflexivity, etc. If there is a relation S with property P containing R such that S is a subset of every relation with property P containing R, then S is called the closure of R with respect to P.

Reflexive closure

For any relation R on A, reflexive closure of R is formed by adding to R all pairs of the form (a, a) with $a \in A$, not already in R i.e. $R \cup D$, where $D = \{(a, a) \mid a \in A\}$ is the diagonal relation on A.

Example:

Let R be the relation on the set {1, 2, 3, 4} containing the ordered pairs (1,2), (1,3), (2,2), (2, 4), (3,1), (3,2), (3, 4), and (4, 4) find reflexive closure.

Solution:

We have R = {(1,2), (1,3), (2,2), (2, 4), (3,1), (3,2), (3, 4), (4, 4)} and

D = {(1,1), (2,2), (3,3), (4,4)}

So, $R \cup D$ = {(1,1), (1,2), (1,3), (2,2), (2, 4), (3,1), (3,2), (3,3), (3, 4), (4, 4)} is the reflexive closure of R.

Symmetric closure

If all ordered pairs of the form (b, a) is added to the relation R on A, where (a, b) is in the relation, that are not already in R, then the newly formed set after addition is symmetric closure of R i.e. for the relation R on A $R \cup R^{-1}$ is symmetric closure of relation R, where $R^{-1} = \{(b, a) \mid (a, b) \in A\}$.

Example:

Find symmetric closure for the relation given in above example.

Solution:

We have R = {(1,2), (1,3), (2,2), (2, 4), (3,1), (3,2), (3, 4), (4, 4)} and

R^{-1} = {(2,1), (3,1), (4,2), (1,3), (2,3), (4,3)}

So, $R \cup R^{-1}$ = {(1,2), (1,3), (2,1), (2,2), (2,3), (2,4), (3,1), (3,2), (3,4), (4,2), (4,3), (4,4)} is the symmetric closure of R.

Transitive closure

A **Path** from a to b in the directed graph G is a sequence of edges (x_0, x_1), (x_1, x_2), … (x_{n-1}, x_n), in G, where n is a nonnegative integer, and x_0 =a and x_n = b. In other words a sequence of edges where the terminal vertex of one edge is same as the initial vertex in next edge in the path. This path is denoted by $x_0, x_1, \ldots, x_n$ and has length n. A path of length $n \geq 1$ that begins and ends at the same vertex is called a circuit or cycle.

Let R be a relation n a set A. The connectivity relation R* consists of the pairs (a,b) such that there is a path of at least one from a to b in R. In notation we write

$$R^* = \bigcup_{n=1}^{\infty} R^n.$$

Theorem 1: (Prove yourself)

The transitive closure of a relation R equals the connectivity relation R*.

Theorem 2:

Let M_R be the zero-one matrix of the relation R on a set with n elements. Then the zero-one matrix of the transitive closure R* is

$M_{R^*} = M_R \vee M_R^{[2]} \vee M_R^{[3]} \vee \ldots \vee M_R^{[n]}$.

Suggestion: Try to read an algorithm using the above theorem.

Warshall's algorithm

Given a set S with n elements $v_1, v_2, \ldots, v_n$ where elements are listed in arbitrary order and R is a relation on a set S. If a, $x_1, x_2, \ldots, x_{m-1}$, b is a path, then $x_1, x_2, \ldots, x_{m-1}$ are called interior vertices of the given path. More clearly, all the vertices of a path that appears somewhere but not as a first or last vertices in a path are interior vertices (If first vertex is met again in the path then it can be interior vertex, similarly if the last vertex has been met already then it can be interior vertex). Warshall's algorithm uses the concept of interior vertices of a path.

Lemma 1:

Let $W_k = [w_{ij}^{[k]}]$ be the zero one matrix that has a 1 in its $(i, j)^{th}$ position if and only if there is a path from v_i to v_j with interior vertices from the set $\{v_1, v_2, \ldots, v_k\}$. Then $w_{ij}^{[k]} = w_{ij}^{[k-1]} \vee (w_{ij}^{[k-1]} \wedge w_{ij}^{[k-1]})$, whenever i, j, and k are positive integers not exceeding n.

Algorithm:

```
Warsahll(W: n by n 0-1 matrix )
{
for(k =0;k<n;k++)
        for(i =0;i<n;i++)
                for(j =0;j<n;j++)
                        w_ij = w_ij ∨( w_ik ∧ w_kj)
//W = [w_ij] is the result
}
```

What is the complexity of above algorithm?

Equivalence Relations

Equivalence Relations

A relation R on set A is called equivalence relation if it satisfies the three properties namely, reflexive, symmetric, and transitive. The two elements related by equivalence relations are called equivalent.

Equivalence Classes

Given R, an equivalence relation on set A, the set of all elements that are related to an element a of A is called the equivalence class of a. The equivalence class of a with respect to R is denoted by $[a]_R$ or [a] when only one relation is in consideration. In notational term we can write $[a]_R = \{b \mid (a,b) \in R\}$. If $c \in [a]_R$, then c is called a representative of equivalence class $[a]_R$.

Equivalence Classes and Partitions

Given a set A, a partition of A is a collection P of disjoint subsets whose union is A i.e.

$\forall B \in P, B \subseteq A$;

$\forall B,C \in P, B \cap C = \phi$, or $B = C$; and

$\forall x \in A \exists B \in P$ such that $x \in B$;

Theorem 1:

Let R be an equivalence relation on a set S. Then the equivalence classes of R form a partition of S. Conversely, given a partition $\{A_i \mid i \in I\}$ of the set S, there is an equivalence relation R that has the sets Ai, $i \in I$, as its equivalence classes.

Example:

Define the relation R on the set A of positive integers by $(a,b) \in R$ iff a/b can be expressed in the form 2^m, where m is an arbitrary integer.

a) Show that R is an equivalence relation.
b) Determine the equivalence classes under R.

Solution:

a)

For all $a \in A$, $a/a = 1 = 2^0$, where m = 0 hence R is reflexive. If $(a,b) \in R$ then we have $a/b = 2^m$ such that $b/a = 2^{-m}$. So we have $(b,a) \in R$ whenever $(a,b) \in R$ hence R is symmetric. Take $(a,b) \in R$ and $(b, c) \in R$ then we can write $a/b = 2^m$ and $b/c = 2^n$ so we have $a/c = 2^{m+n}$, where m+n is an arbitrary integer so that $(a,c) \in R$ hence R is transitive. For the facts above we showed that R is an equivalence relation.

b)

Some equivalence classes may be

[1] = {1,2,4,8,… …}

[3] = {3,6,12,24,48, … …}

[5] = {5,10,20,40, … …}

[7] = {7,14,28,56, … …}

… …

We can generalize these classes as

$[p] = \{2^m * p \mid$ m is nonnegative integers and p is either 1 or prime greater than 2$\}$.

Partial Orderings

A relation R on a set A is called **partial order** or partial ordering if it is reflexive, antisymmetric, and transitive. A set A together with the partial order R is called a partially ordered set, or a poset, denoted by (A, R).

Example:

Show that a relation ≤ "less than or equal" is partial order on the set of integers.

Solution:

We know $\forall a \in Z$, $a \leq a$, hence ≤ is reflexive. If $a \leq b$ and $b \leq a$, then $a = b$, hence ≤ is antisymmetric, and if $a \leq b$ and $b \leq c$, then $a \leq c$, hence ≤ is transitive. It follows that ≤ is a partial ordering on the set of integers Z and (Z, ≤) is a poset.

In a poset (S, R), the elements a and b of a poset are comparable if either aRb or bRa. When neither aRb nor bRa, then a and b are incomparable. If every two elements of a set S are comparable, then S is called a **totally ordered** or **linearly ordered set** or a **chain**, and R is called **total order** or a linear order.

(S,R) is a **well ordered set** if it is a poset such that R is a total order and such that every nonempty subset of S has a least element.

Lexicographic order

Given two posets (A_1,L_1) and (A_2,L_2). The lexicographic ordering L on $A_1 \times A_2$ is defined by specifying that one pair is less than a second pair i.e. $(a_1, a_2)L(b_1, b_2)$ if $a_1L_1b_1$ or both $a_1 = b_1$ and $a_2L_2b_2$. Similarly we can extend this definition to Cartesian product or more than two sets.

Lexicographic Ordering of Strings

Suppose $a_1a_2 \ldots a_m$ and $b_1b_2 \ldots b_n$ are strings on a partially ordered set S and suppose two strings are not equal. If t is the minimum of m and n, then we define lexicographic ordering that the string $a_1a_2 \ldots a_m$ is less than $b_1b_2 \ldots b_n$ if and only if

$(a_1a_2 \ldots a_t) < (b_1b_2 \ldots b_t)$, or

$(a_1a_2 \ldots a_t) = (b_1b_2 \ldots b_t)$ and $m < n$.

Example #1:

Find the lexicographic ordering of the 3 tuples (1,1,2), (1,2,1)

Solution:

(1,1,2) < (1,2,1) because first elements of both the 3 tuples are same so if we look forward then the second element of first 3 tuple is less than the second element of second 3 tuple.

Example #2:

Find the lexicographic ordering of the strings *quack, quick, quicksilver, quicksand, quacking*.

Solution:

quack < *quick* [strings differ in third position and a < i]

quick < *quicksilver* [letter of the first string agrees but second string is longer]

quicksand < *quicksilver* [strings differ in seventh position and a < i]

quacking < *quicksand* [strings differ in third position and a < i]

Similarly others are

quack < *quacking, …..*

So the ordering is

quack < *quacking* < *quick* < *quicksand* < *quicksilver*

Hasse Diagrams

A partial ordering on a finite set can be represented using the pictorial notation as follows:

Construct the directed graph of a relation.

Remove all the loops (since it is clear that partial order is reflexive so every vertex has a loop)

Remove all the edges that is requires due to the transitivity

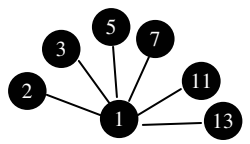
Arrange each edge so that its initial vertex is below its terminal vertex and remove all arrows from the edges since edges point in upward direction only.

The diagram formed using above steps contains sufficient information to find the partial ordering. This diagram is called **Hasse diagram**.

Example #1:

Draw the Hasse diagram for divisibility on the set {1,2,3,5,7,11,13}

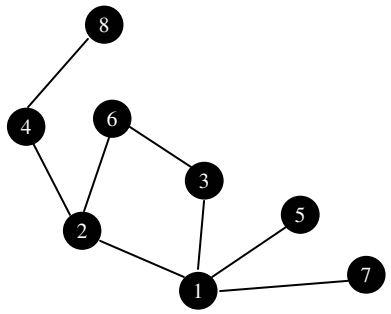
Solution:



Example #2:

Draw the Hasse diagram for divisibility on the set {1,2,3,4,5,6,7,8}

Solution:



Maximal and Minimal Elements

Some Definitions:

Maximal Elements: An element of a poset is maximal if it is not less than any elements of the poset i.e. a is maximal in the poset (S,≤) if there is no b ∈ S such that a < b. In Hasse diagram maximal elements are top elements.

Minimal Elements: An element of a poset is minimal if it is not greater than any elements of the poset i.e. a is minimal in the poset (S,≤) if there is no b ∈ S such that b < a. In Hasse diagram minimal elements are bottom elements.

Greatest Element: An element in a poset that is greater than every other element i.e. a is the greatest element of the poset (S,≤) if b ≤ a for all b ∈ S. The greatest element may exist or may not exist in a poset.

Least Element: An element in a poset that is less than every other element i.e. a is the least element of the poset $(S,\leq)$ if $a \leq b$ for all $b \in S$. The least element may exist or may not exist in a poset.

Upper Bound: If u is an element of S such that $a \leq u$ for all elements $a \in A$, then u is called an upper bound of A, where $A \subseteq S$.

Lower Bound: If l is an element of S such that $l \leq a$ for all elements $a \in A$, then l is called a lower bound of A, where $A \subseteq S$.

Least Upper Bound: l_u is the least upper bound of the subset A if $a \leq l_u$ whenever $a \in A$ and $l_u \leq u$ whenever u is an upper bound of A.

Greatest Lower Bound: g_l is the greatest lower bound of the subset A if g_l is a lower bound and $l \leq g_l$ whenever l is a lower bound of A.

Example:

Answer these questions for the poset $(\{3,5,9,15,24,45\},|)$

a) Find the maximal elements
b) Find the minimal elements
c) Is there a greatest element?
d) Is there a least element?
e) Find all upper bounds of {3,5}
f) Find the least upper bound of {3,5} if exists
g) Find all lower bounds of {15,45}
h) Find the greatest lower bound of {15,45}, if exists.

Solution:

a) 24 and 45 are maximal elements.
b) 3 and 5 are minimal elements
c) No greatest element since there is no element in a set is divisible by all the elements in a set.
d) No least element since no element in a set divides all the elements in a set.
e) Since 15 and 45 are divisible by both 3 and 5 and they are follow the relation we have 15 and 45 as upper bounds of {3,5}

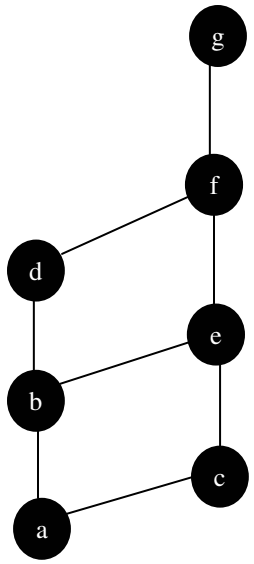
f) Since among the upper bounds 15 is the least, 15 is least upper bound.
g) The elements 3, 5 , and 15 divides the elements 15 and 45. Hence 3, 5, and 15 are lower bounds
h) Amongst the lower bounds 15 is the greatest lower bound.

Lattices

A partially ordered set in which every pair of elements has both the least upper bound and the greatest lower bound is called **lattice**.

Example #1:

Identify whether the poset given by following Hasse diagram is lattice or not?



Solution:

Every pair of elements for e.g. (a,b), (a,c), (a,e), (b,e) …. Has least upper bound and greatest lower bound so the poset given by above Hasse diagram is a lattice.

Example #2:

Determine whether the poset (Z, ≥) is a lattice

Solution:

Suppose x and y are two integers. If we have a relation $x \geq y$ for all $x,y \in Z$, then we can

say that x is the least upper bound and y is the greatest lower bound. Conversely $y \geq x$ for all $x,y \in Z$, then we can say that y is the least upper bound and x is the greatest lower bound. These conditions hold for all elements in Z, hence poset $(Z, \geq)$ is a lattice.

Topological Sorting

A total ordering $\leq$ is called **compatible** with the partial ordering R if $a \leq b$ whenever aRb. Construction of a compatible total ordering from a partial ordering is called topological sorting.

Lemma 2:

Every finite nonempty poset $(S, \leq)$ has a minimal element.

Proof:

Take an element a_0 from the set S. if a_0 is not minimal, then there is an element a_1 with $a_1 < a_0$. If a_1 is not minimal there is an element a_2 with $a_2 < a_1$. this process is continued until all the elements in the set are tested for and if it goes up to nth element of the set which is the last one from the set then it will be the minimal element.

Algorithm:

toposort(S: finite poset)

{

k =1;

while (S ≠ ϕ)

{

a_k *= a minimal element of S // from lemma 1 it is sure*

S = S – { a_k *};*

k = k+1;

}

return {$a_1, a_2, \ldots, a_n$*} //compatible total ordering of S.*

}

Example:

Find a compatible total order for the divisibility relation on the set {1,2,3,6,8,12,24,36}

Solution:

From the set we can identify that 1 is the minimal element so, 1 is selected. Next, minimal of {2,3,6,8,12,24,36} are 2 and 3, so we can choose any one element. Lets choose 2. Similarly, minimal of {3,6,8,12,24,36} is 3 and 8, so choose 8. Again select minimal element i.e. 3 from the set {3,6,12,24,36}. Continuing like this we have one of the total ordering as

$1 \prec 2 \prec 8 \prec 3 \prec 6 \prec 12 \prec 24 \prec 36.$

Self Studies

Read chapter 7 of your textbook such that you can cover all the read materials in the class.